

残酷共学 · 增量笔记

自 2026-08-08 00:20 BJT 后新增
100 篇

本次增量 · 带索引

点击目录条目可跳转

数据来源：intensivecolearn.ing 活动公开打卡

生成时间：2026-08-08 23:00 (BJT)

目录

主题章节 → 笔记索引 (点击可跳转)

01 AMM机制 (17 篇)	7
笔记 001: Day3: 认识 DEX 内、DEX 间、三角套利、稳定币套利等机会结构，并理解机会产生的原因	7
笔记 002: Day 3-4 补记：Robinhood 链套利——从"看别人赚钱"到"自己测出机会"。	8
笔记 003: Uniswap V2	9
笔记 004: Day 4 链上套利残酷共学	9
笔记 005: day4	10
笔记 006: Day 4 - AMM、流动性与价格冲击	10
笔记 007: D4 (8/8) 三角套利 + 原子套利 + 闪电贷全套 (互动推演 + 真实链上拆解)	11
笔记 008: Day 4：同交易对不同手续费池的候选价差筛查。	11
笔记 009: eth/0x4c5bad456b77f78bca98b3983737f5b5dd85f0ee02dc	12
笔记 010: ## 做了什么	12
笔记 011: 链上套利残酷共学 · Day 4 打卡目标 (2026-08-08)	16
笔记 012: 假设同一个 ETH 在两个 DEX 的池子里价格不同：	16
笔记 013: 问题一：什么是 AMM、订单簿、流动性深度、价格冲击和区块确认？它们为什么是理解链上套利必须掌握的基	
笔记 014: 今天看了Uniswap的白皮书，从V2的 $x \cdot y = k$ 的模型，看到V3的tick类的CLMM模型	27
笔记 015: Day 4 集中流动性、可执行深度与股票代币机会观察	27
笔记 016: 链上套利 (On-Chain Arbitrage) 是指利用区块链网络中不同去中心化交易平台 (DEX)、	28
笔记 017: Day 4：Pool 基础状态	29
02 LI.FI与跨链 (27 篇)	30
笔记 018: 链上套利残酷共学 Day 1：建立统一套利判断框架	30
笔记 019: 今天主要做了一个初始的成本计算器，很多参数都没有扩展，目的是只做验证。	35
笔记 020: 核心架構與運作機制	36
笔记 021: 今天建立统一净收益表 \research/opportunity-log.csv\，把 D3 的 1	36
笔记 022: 2026-08-08	36
笔记 023: https://github.com/ZeroNullNone/onchain_arbitrage_	37

笔记 024: 已跑完24小时数据，先说结论：	37
笔记 025: 本次从 LI.FI 拉取了三天的跨链报价数据(每次间隔30 分钟), 定时向 LI.FI (ht	38
笔记 026: Day 4：两链预置库存与事后调仓	39
笔记 027: 今日完成	39
笔记 028: 链上套利残酷共学打卡 - 2026-08-08	40
笔记 029: 笔记	41
笔记 030: 【8月8日 学习打卡】链上套利 Day 4：监控系统升级与自动交易	42
笔记 031: LI.FI 对套利者（尤其是跨链套利）非常有用，它是跨链资金调度和执行的核心基础设施工具。对套利者的	42
笔记 032: 链上套利共学 Day 4 LI.FI Agent 集成学习：5 端点最小集 + /tools 桥链对	43
笔记 033: 模块 3 第 2 天：Hermes 信号工作流骨架	44
笔记 034: Day 4 从交易复盘走向支付路由	45
笔记 035: 链上套利残酷共学 · Day 4 打卡（2026-08-08）	45
笔记 036: 2026-08-08 — Day 4: 真实案例净收益计算（跨链 DEX 价差 × 成本模型）	47
笔记 037: /v1/quote (GET) /v1/routes (POST)	48
笔记 038:	
链上发现一个价差，只是套利流程的开始。最后能不能执行，要看资金从哪里来、走哪条路、多久到达、实际收到	49
笔记 039: 今天按照昨天确定的方向，完成了第一个 LI.FI 只读报价观察器。我没有连接钱包、没有签名，也没有发	49
笔记 040: Day 4 从 Route 监控到可实现盈亏的套利全流程	50
笔记 041: Day 4 · integrator 陷阱（实测 2026-08-07，8/8 打卡）	51
笔记 042: 链上套利共学 · Day4 打卡笔记（08/08）	52
笔记 043: Day 4 2026.08.08 Solana 三角套利: 从发现到实盘	53
笔记 044: Day 4 学习笔记（2026-08-08）	54

03 MEV与抢跑（5 篇） 55

笔记 045: MEV-Flashbot-Sandwich-RS：从 Mempool 到 Bundle 的 MEV	55
笔记 046: markdown	58
笔记 047: 学习笔记：以太坊区块构建系列（Greenfield Capital 译文）第 1 课	62
笔记 048: @brucexu_eth 用 Hermes 制作的交互式教学网站\	62

笔记 049: 	64
04 做市与LP (3 篇)	73
笔记 050: Day 4 : 全自动套利系统架构 v0.2	73
笔记 051: 在網上看到了Polymarket的套利文章，不過因為台灣網域被封的原因，我使用predict的資料進	75
笔记 052: (本打卡由 Hermes Agent 辅助整理并提交。内容全部来自我的研究和实践，Agent 仅负责	76
05 套利框架 (36 篇)	76
笔记 053: 2026-08-08 跨链 RWA 套利监控日志	77
笔记 054: Boros 规模数据与亲身实测(截至 2026-08 初)	77
笔记 055: 共学第4天。今天研读了两份链上地址调研报告,收获很大。	78
笔记 056: 理解执行成本模型	79
笔记 057: 今天第三天，继续啃套利基础。	79
笔记 058: Day 4 (8/8) 成本模型 I：显性成本，一分不漏	80
笔记 059: - 今天把链上套利重新理解成“读取状态 → 按顺序执行状态转换 → 回到同一计价资产 → 扣除全部成 . . .	80
笔记 060: 今天主要学习了 Hermes Agent 在研究流程中的应用，开始思考如何把以前依靠人工搜索、记录和	80
笔记 061: 肥尾分布和杠铃策略	81
笔记 062: 【Day 4 打卡】2026/8/8	82
笔记 063: D4 打卡 (8/8) · 三层套利认知闭环 + Solana 主线双实现 + 武器库成型	83
笔记 064: 链上套利残酷共学 · Day 3 心得	84
笔记 065: 这是我的所思所想	84
笔记 066: 今天收住了“独立发现价差”这一阶段，也迈出了实时读取链上数据的第一步。	85
笔记 067: 调研 RWA Perpetual Arbitrage :	86
笔记 068: 一、套利的本质：寻找市场“不平衡”	86
笔记 069: 【Day 4 2026-08-08】 M1 屏幕价差 vs 净收益	87
笔记 070: 永续合约对永续合约	87
笔记 071: 【Day 8 打卡 · 2026-08-08】用 Hermes 构建研究与信号工作流	88
笔记 072: 2026年8月8日 链上套利学习笔记	88
笔记 073: 今日学习心得：从工具报价到真实 Swap 交易还原	88

笔记 074: Day 4 打卡：USDL 稳定币赎回套利案例	89
笔记 075: Day 4 从报价到可执行：拆解一笔 DEX 套利交易的真实利润	93
笔记 076: LP 不是套利	94
笔记 077: Day 4 打卡（2026-08-08）	95
笔记 078: 现货和合约差20%，我差点把方向搞反	96
笔记 079: 链上套利学习 Day 4 笔记	96
笔记 080: 今天状态有点累，所以没有继续硬啃很多概念，只抓住了一个最关键的变化：我开始把页面上的报价看成一张“询” . . .	104
笔记 081: 链上套利残酷共学 2026-08-08 打卡	104
笔记 082: 今日打卡 把套利想法变成可证伪的实验	104
笔记 083: 今天继续深挖链上套利的 MEV 部分：	106
笔记 084: 2026-08-08 共学 Day 4 投入约 1 小时	106
笔记 085: 今日共学打卡 Gate CrossEx × 跨所资金费率套利	107
笔记 086: 链上套利残酷共学 Day 2 学习笔记	107
笔记 087: 太好了，这份聊天记录简直是“小白防坑教科书”。里面全是真实的市场声音，没有一句废话。	108
笔记 088: Day 4（8/8）打卡	110
06 学习计划（4 篇）	110
笔记 089: Day 4 打卡 套利策略类型全景对比与可行性矩阵	110
笔记 090: Day 4 · 报价对比（agent 开发工程师视角）	113
笔记 091: 2026-08-08 8 月 6 日、8 月 7 日补录说明	115
笔记 092: Day 4 并发配对报价：从单腿样本走向闭环成本	115
07 实盘与数据（4 篇）	116
笔记 093: 今天做了什么：搭建“候选机会证据记录”模板并归档 D1-D3（OPP-001~004，全部“已验证”→	116
笔记 094: 2026年8月8日 Codex 学习笔记	117
笔记 095: 学习订单簿的作用，相比K线OHCLV	120
笔记 096: 今天把研究重点进一步收敛到资金费率套利，并尝试将前几天讨论的策略转化为可监控、可计算的信号。	121
08 风险与安全（4 篇）	121

笔记 097: 探讨在 cex-dex 套利中我一直存疑的问题，Day 3：【DEX 自动交易，如何保证钱包安全】	121
笔记 098: Day 4 — 闪电贷套利实战	122
笔记 099: 01.5 BEP20 余额、转账、授权与事件 学习笔记	123
笔记 100: 第四天学习记录：今天把 Robinhood Chain 上 0xd712 套利体系从历史交易观察推进	126

笔记 001: Day3: 认识 DEX 内、DEX

间、三角套利、稳定币套利等机会结构，并理解机会产生的原因

作者：JWiang | 时间：2026-08-07 18:48 UTC | 字数：1940 | 评分：80

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day3: 认识 DEX 内、DEX 间、三角套利、稳定币套利等机会结构，并理解机会产生的原因

一、DEX 间套利 (DEX-to-DEX Arbitrage)

1、定义是指：同一条链上，同一种资产，在不同 DEX 上出现短暂价格差，通过连续买卖获取收益。

例如：Ethereum 链：

DEX	ETH价格
Uniswap	3000 USDC
SushiSwap	3012 USDC

理论套利路径：

USDC
↓
Uniswap 买 ETH
↓
SushiSwap 卖 ETH
↓
USDC

如果扣除所有成本后仍有利润，就存在套利机会。

2、为什么价格会不同？

价格差通常来自四类原因：

① 不同流动性池独立定价：每个 DEX 都维护自己的流动性池。例如：

Uniswap：100 ETH、300000 USDC

Sushi：300 ETH、910000 USDC

由于储备不同，价格自然不同。

② 大额交易导致价格瞬时偏离

有人刚刚在 Uniswap 买了大量 ETH。结果 ETH 被买走。ETH 更少。价格立即上涨。

但是：Sushi 还没发生交易。价格仍然保持原来。于是：套利窗口出现。

③ LP（流动性提供者）结构不同

不同协议：Fee Tier 不同 LP 数量不同 流动性集中程度不同

因此：价格恢复速度也不同。

④ 区块确认延迟

区块链不是实时数据库。交易：用户提交 - Mempool - Validator - Block 不同协议更新状态存在时间差。

3、DEX 间套利流程

发现价格差 ↓ 确认两个池子的深度 ↓ 计算 Gas ↓ 计算滑点 ↓ 模拟成交价格 ↓ 判断是否还有利润 ↓ 发送交易

真正套利者：不是看：Price A Price B

而是：Execution Price A Execution Price B

二、三角套利 (Triangular Arbitrage)

1、定义：利用三个交易对之间汇率不一致完成循环交易。

例如：交易对：ETH / USDC、BTC / ETH、BTC / USDC

形成：USDC - ETH - BTC - USDC

如果：最终得到：1010 USDC，初始：1000 USDC，利润：10 USDC

2、为什么会出现？

因为三个市场更新速度不同。

例如：ETH 涨了。BTC 没同步。于是：BTC/ETH 汇率错误。机器人立即修正。

3、套利判断公式

设：USDC→ETH 汇率：R1

ETH→BTC R2

BTC→USDC R3

则套利条件： $R1 \times R2 \times R3 > 1$

如果：等于：1，没有套利。如果小于1，亏损。如果大于1，理论存在套利。

但是必须继续扣：Gas、DEX Fee、Slippage 之后才能判断。

4、为什么机器人快？

因为他们实时扫描几千个交易池。任何： $R1 \times R2 \times R3$ 发生变化，立即发送交易。所以，三角套利生命周期通常毫秒级。

三、稳定币套利 (Stablecoin Arbitrage)

1、为什么稳定币也有套利？

稳定币：目标：1 USDC $\approx$ 1 USD，但实际上会：0.998、1.002、0.995

原因：供需变化。

2、Curve 为什么重要？普通 AMM 适合 ETH、BTC 等波动资产。

Curve：专门稳定币。因为：采用：StableSwap。比普通恒定乘积模型，它会在价格接近

1:1

时提供更低滑点，因此更适合稳定币之间的大额兑换。这也是为什么很多稳定币套利首先发生在 Curve 这样的协议。

3、脱锚 (Depeg)

例如：USDC：1 ↓ 0.92

原因：市场恐慌。

套利者：买：0.92 - 赎回：1美元 - 利润：0.08

现实中还需要考虑：是否能够赎回、流动性是否足够、是否存在信用风险

四、常见稳定币套利

DEX 与 DEX: Curve - Uniswap

CEX 与 DEX: Binance - Curve、不同链 Ethereum - Arbitrum

笔记 002: Day 3-4 补记：Robinhood 链套利——从“看别人赚钱”到“自己测出机会”。

作者：akerjia | 时间：2026-08-08 00:13 UTC | 字数：562 | 评分：59

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 3-4 补记：Robinhood 链套利——从“看别人赚钱”到“自己测出机会”。

1. 地址调研 (8/7)：深挖 0xD712...9B96 高频套利 bot——它通过代理合约做 WETH→中间币→USDG→WETH 三角循环，一笔交易内原子执行，拼不拢就整体回滚只亏 gas (约 3 美分)。关键认知：Robinhood 链 gas 极低，策略是“高失败率 + 小额高频”，用便宜失败换抢跑速度。

2. 只读扫描器 v0 落地 (8/8)：按调研结论先做只读工具——读池状态 (Uniswap v4.1 storage + v3 风格 CLPool)、算三角闭环理论收益、记录机会，不签名不交易。跑通真实扫描。

3. 抓到第一个理论机会：TA (meme 币) 跨池定价分歧 12.8%——v4 TA/WETH 池 13 分钟被 8 笔交易砸低 10%，USDG/TA 池闲置没跟上。WETH→TA→USDG→WETH 路线 0.004 WETH 理论毛利 +9.1%，扣 gas 净赚约 \$0.70。但金额越大滑点越狠 (0.05

WETH 即亏损)，且已有其他 bot 在抢——理论毛利 $\neq$ 实际利润，定位是积累样本。

4. 下一步：扩展中间币池（已发现 11 个候选）、cron 定时扫描积累机会样本，暂不实盘。

笔记 003: Uniswap V2

作者：xsh | 时间：2026-08-08 00:41 UTC | 字数：330 | 评分：57

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Uniswap V2

v2概念中，交易对称称为Pair（v3命名为Pool，因为一个交易对只有一个Pool且费率固定为0.3%）

Contract Name: UniswapV2Pair

合约：UniswapV2Router02，UniswapV2Pair，UniswapV2Factory

Event：event Swap(address indexed sender, uint amount0In, uint amount1In, uint amount0Out, uint amount1Out, address indexed to)

event Sync(uint112 reserve0, uint112 reserve1)

笔记 004: Day 4 | 链上套利残酷共学

作者：HashFlow | 时间：2026-08-08 04:20 UTC | 字数：1345 | 评分：89

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 4 | 链上套利残酷共学

日期：2026.08.08（周六）

主题：成本模型 + 净利润过滤

核心产出：15 项完整成本模型

已实现（6 项）

| # | 成本 | 来源 |

| :-: | :----- | :----- |

| ① | Swap 费 | pool.fee() — Uniswap V3 合约 |

| ② | Gas Base Fee | eth_gasPrice — EIP-1559 |

| ③ | Gas Priority Fee | eth_maxPriorityFeePerGas — EIP-1559 |

| ④ | Gas 用量 | 硬编码 180000，待替换 eth_estimateGas |

| ⑤ | 滑点 | Quoter V2 在 Base 公开 RPC 不可用 |

| ⑥ | Builder 支付 | 存根，待接入 Flashbots relay API |

未实现（9 项，已标注来源）

MEV 竞价溢价、链下基础设施、失败尝试、资金/对冲/订单流成本、ERC20 Approve、ETH $\leftrightarrow$ WETH、闪电贷手续费
成本来源验证

访问了 Bruce 的「套利机制实验室」(arb-demo.brucexu.xyz)，五个模块全部遍历：

MEV 价值流确认了成本归因模型（searcher/builder/proposer/LP/L1）

明确了 "Builder payment、swap fee 和 gas 是从 Searcher 流出的内部支付"

明确了 "净利润还要扣除链下基础设施、失败尝试、资金、对冲和订单流成本"

实战验证

用 WETH/USDC 22 个池子跑净利润过滤：

最低价买入 (iziswap, \$1,906.58, 0.3% fee)

最高价卖出 (uniswap, \$1,921.04, 1.0% fee)

毛利润 \$7.59 $\rightarrow$ 仅 Swap 费就 \$13.08 $\rightarrow$ 净亏损 \$5.50

不需要滑点和 MEV 竞价——仅协议费一项就足以让这笔交易亏损

关键认知

1. DEX 间套利的摩擦成本是结构性的，不是偶然的——协议费系统性地大于价差

2. 成本不是 3 项而是 15 项，但其中 6 项已足以判断多数交易不可行

3. Base L2 的 Gas 极低 (eth_gasPrice $\approx$ \$0)，不是 bug——OP Stack 的 L2 费用模型确实如此

下一步

Day 5（周日）：从假滑点到真实滑点——探索 V3 tick 模拟或 Quoter 替代方案

笔记 005: day4

作者：lunateng | 时间：2026-08-08 07:02 UTC | 字数：1288 | 评分：80

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

day4

Robinhood Chain 地址：高频三角循环套利

机器人在链下持续读取各个池的价格、流动性、手续费和 Gas，把池子建模成一张兑换图，然后搜索从 WETH 出发、最终回到 WETH 的正收益闭环。

WETH

→ 中间币

→ USDG

→ WETH

Aave 地址：闪电贷清算套利

1\ 闪电贷提供交易本金

机器人不需要提前持有数千枚 LINK。只要整个交易结束前还款，就能使用 Aave 或 Balancer 的流动性。

“无本金”不等于零成本，它仍需承担闪电贷费、Gas、兑换成本和竞价成本。

2\ 捕捉清算折价

当借款人 Health Factor 低于1，机器人替其偿还部分债务，并获得价值更高的抵押品。

利润不是来自普通币价上涨，而是来自协议给清算人的奖励。

3\ Bebop PMM RFQ 精确回补

机器人没有直接在公开 AMM 中逐池兑换，而是向专业做市商获取 RFQ 报价。

Bebop 返回的 LINK 数量与“闪电贷本金 + 费用”精确一致；剩余价值则用 WETH 表达。这能降低大额公开兑换的滑点，并减少路径暴露。

4\ BuilderNet 区块竞价

清算机会是公开的，任何机器人都可以调用。真正稀缺的是：谁先进入有效区块。

该交易把0.5366204 WETH全部解包成ETH，支付给当块的BuilderNet fee recipient，相当于将可见毛收益拿去竞拍排序。

| 维度 | Robinhood Chain | Aave清算 |

| :--- | :----- | :----- |

| 利润来源 | 多个池之间的价格不一致 | 借款人的清算折价 |

| 起始资本 | WETH或原生ETH | 闪电贷 |

| 交易路径 | 三角或多池循环 | 借款→清算→RFQ→还款 |

| 执行场所 | Uniswap v4/v3、CLPool | Aave、Spark、Balancer、Bebop |

| 排序竞争 | 可能抢跑/backrun，尚未确认 | 明确需要抢先清算 |

| 单笔特点 | 利润小、Gas低、频率高 | 金额大、机会少、竞争激烈 |

| 失败结果 | 整笔回滚，只亏Gas | 整笔回滚，只亏Gas |

| 盈利证据 | 两笔样本可直接算净利润 | 毛差额可见，真实退款不可见 |

| 核心壁垒 | 低延迟多池报价与低成本试单 | 仓位索引、RFQ和Builder渠道 |

笔记 006: Day 4 - AMM、流动性与价格冲击

作者：MountainE | 时间：2026-08-08 07:17 UTC | 字数：543 | 评分：57

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 4 - AMM、流动性与价格冲击

今天理解了 AMM、储备量、恒定乘积和滑点之间的关系。在简化的恒定乘积模型中， $x \cdot y = k$ ， x 和 y 是两种资产的储备量，交易会改变储备比例和成交价格。

我用一个简单例子记录价格冲击：

- 初始池子有 100 ETH 和 200,000 USDC，则 $k = 20,000,000$ ，初始即时价格约为 2,000 USDC/ETH。
- 如果用 USDC 买入 10 ETH，忽略手续费，新的 ETH 储备量为 110，新的 USDC 储备量为 $20,000,000 / 110 = 181,818.18$ 。
- 这笔交易需支付约 18,181.82 USDC，平均成交价约为 1,818.18 USDC/ETH，和交易前的即时价格有明显差异。

这个例子说明：交易规模相对池子储备越大，价格冲击通常越大；流动性越深，同等规模下的滑点通常越小。因此，套利不能只看两个池子的标价，还要把交易规模、价格冲击、手续费和 Gas 一起计算。

今日产出：完成了一篇 AMM 成本示意笔记，核心公式为 $x y = k$ 。

明日计划：对比订单簿与 AMM 的成交机制，并记录区块确认、报价有效期和状态变化对套利执行的影响。

笔记 007: D4 (8/8) 三角套利 + 原子套利 + 闪电贷全套 (互动推演 + 真实链上拆解)

作者：peanutwen00 | 时间：2026-08-08 07:59 UTC | 字数：1122 | 评分：70

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

D4 (8/8) 三角套利 + 原子套利 + 闪电贷全套 (互动推演 + 真实链上拆解)

【上午：三角套利】

1) 理论推演：三角套利 = 同一 DEX 内 $A \rightarrow B \rightarrow C \rightarrow A$ 循环，隐含汇率乘积 $\neq 1$ 时存在机会。亲手推出“低价池买、正常池卖、转一圈变多”的完整逻辑。

2) 关键发现：初始三池价格一致时无套利空间（乘积=1）；池子太浅时价格冲击爆炸（0.25 ETH 池被 0.99 ETH 买入推价 25 倍）。用公式验证了“价差 + 池深 + 规模”三要素。

3) 真实链上验证（Uniswap V2 WBTC/WETH/USDC 三池实时数据）：发现 0.198% 真实价差，但 3 次 swap 手续费 0.9% > 价差 → 转一圈反而亏。活体演示“屏幕价差 ≠ 净收益”——机器人已把大价差抹平到低于手续费。

【下午：原子套利】

4) 概念：原子套利 = 事件驱动（池子被大额兑换砸歪）→ 闪电贷借钱 → 一笔交易内“借-买-卖-还”。与三角套利区别：三角是结构性机会，原子是偶发事件机会。

5) 最优规模推演：模拟 58,000 vs 64,000 价差场景，算出最优借款额约 10 万 USDC（净利 +5,185），借 58 万直接亏 6.8 万——价格冲击两头吃，存在“甜蜜点”。

6) 真实交易拆解 (tx 0x9762a7cb...)：Balancer Vault 闪电贷借 1.21 亿 USDC，多池兑换校准价格后归还，完整展现“借-买-卖-还”原子结构。完美呼应文章“大型原子套利源于个别用户的价格冲击兑换”。

【闪电贷专项】

7) 费用真相：不是时间利息，是固定比例手续费（只借一个区块）。Balancer/Uniswap 免费，Aave 0.05%-0.09%。费率差决定生死：1.21 亿借款 Balancer 0 成本 vs Aave 6 万刀。

8) 去哪借：不是专门合约，是 Aave/Balancer 等协议内置功能；必须自己写合约实现回调接口（executeOperation/receiveFlashLoan）或用聚合器（慢，抢不到机会）。

9) 原子性约束：必须一笔交易内借还，没还上整笔回滚（唯一损失 Gas，Flashbots 连 Gas 都不扣）。

【结论】三角套利和 stat arb 已被机器人垄断（0.198% 价差被手续费吃掉）；原子套利竞争分散是散户最现实切入点，但核心门槛是“会写合约 + 机器级速度”。

产出：知识库新增/更新 amm-principles 验证、flash-loan-basics 大扩展（费用/来源/速度/约束），两笔真实链上交易拆解。

笔记 008: Day 4：同交易对不同手续费池的候选价差筛查。

作者：daohuang1982-beep | 时间：2026-08-08 09:12 UTC | 字数：646 | 评分：72

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 4：同交易对不同手续费池的候选价差筛查。

今天只读比较了 Ethereum 主网 Uniswap V3 WETH/USDC 的 0.05% 与 0.3% 两个池。两个页面的报价方向不同，我先统一为 1 WETH = ? USDC：0.05% 池为 1,916.93 USDC，0.3% 池由 1 USDC = 0.0005242 WETH 倒数换算为约 1,907.668829 USDC，页面毛价差约 0.485%。

若只扣除 0.3% 和 0.05% 两腿名义手续费，理论余量约为 0.134%，每 1,000 USDC 仅约 1.339 USDC，而且还没有计算指定规模价格影响、Gas、MEV、延迟和失败成本。两个快照也不是同一区块的同步链上数据，0.3% 池页面还显示 USDC 为 \$1.0042，偏差来源未知。

页面同时显示，0.05% 池流动性约 \$95.57M、24 小时成交量约 \$83.01M；0.3% 池分别约 \$16.25M 和 \$503.33K。这说明同一交易对的不同手续费池在深度和活跃度上可能相差很大，但总流动性仍不能替代当前附近的可执行深度。

今天的结论是：先统一报价方向，再扣双边手续费，页面价差就只剩很薄的验证余量。当前证据为 partial，没有确认任何可执行套利机会。下一步若继续，只针对 1,000 USDC 固定规模读取尽量同步的双池 Quoter 和 Gas 估算，仍不进行真实交易。

笔记 009: eth/0x4c5bad456b77f78bca98b3983737f5b5dd85f0ee02dc

作者 : James Wong | 时间 : 2026-08-08 10:51 UTC | 字数 : 1543 | 评分 : 67

来源 : <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

eth/0x4c5bad456b77f78bca98b3983737f5b5dd85f0ee02dc38670795414c976c8490

学习清算套利时，发现对如何计算uniswap v3 pool中一个货币的价格？缺乏了解。

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.19;

interface IUniswapV3Pool {
    function slot0() external view returns (
        uint160 sqrtPriceX96,
        int24 tick,
        uint16 observationIndex,
        uint16 observationCardinality,
        uint16 observationCardinalityNext,
        uint8 feeProtocol,
        bool unlocked
    );
    function token0() external view returns (address);
    function token1() external view returns (address);
}

interface IERC20Metadata {
    function decimals() external view returns (uint8);
}

contract UniswapV3PriceReader {
    // 示例 : WETH/USDC 0.05% 池子地址 (checksum 格式)
    address constant POOL = 0x88e6A0c2dDD26FEEb64F039a2c41296FcB3f5640;

    function getPrice() external view returns (uint256 priceToken0InToken1) {
        IUniswapV3Pool pool = IUniswapV3Pool(POOL);
        (uint160 sqrtPriceX96, , , , ) = pool.slot0();

        uint8 decimals0 = IERC20Metadata(pool.token0()).decimals();
        uint8 decimals1 = IERC20Metadata(pool.token1()).decimals();

        // priceRaw = (sqrtPriceX96 / 2^96)^2, 放大 1e18 避免精度损失
        uint256 priceRaw = FullMath.mulDiv(
            uint256(sqrtPriceX96) uint256(sqrtPriceX96),
            1e18,
            1 << 192
        );

        // 调整 decimals 差异 : price = priceRaw 10^(decimals0 - decimals1)
        if (decimals0 >= decimals1) {
            priceToken0InToken1 = priceRaw * (10 (decimals0 - decimals1));
        } else {
            priceToken0InToken1 = priceRaw / (10 (decimals1 - decimals0));
        }
    }
}

// 从 pool 合约读取 sqrtPriceX96 并计算价格
```

笔记 010: ## 做了什么

作者 : HOU RUI | 时间 : 2026-08-08 11:30 UTC | 字数 : 6183 | 评分 : 80

来源 : <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

做了什么

1. 搞清单边 LP 到底是什么 —— 用真实池子把成交均价、价格路径、与限价单的差别全部实算

2. 写了 `\build\_mint.py\` —— 输入意图输出未签名 calldata，核心是方向检查

3. 拆解一笔真实的大额套利 —— 90 跳、1015 万 gas、净利 13.3 万美元

4. 证伪了昨天的头号候选 USDC/SEI

验证 / 推翻了什么

1. 验证：单边 LP 的成交均价 = 几何均值，不是区间中点

实算 (USDC/WETH 池，现价 1901.66，在 1997~2187 投入 5 ETH)：

\

穿透区间后收到 10,449.21 USDC

成交均价 2,089.84 USDC/ETH

区间几何均价 $\sqrt{(Pa \cdot Pb)} = 2,089.84$ ← 精确相等

区间算术均价 $(Pa + Pb) / 2 = 2,092.00$ ← 比实际高

\

不是巧合 —— $\Delta y / \Delta x$ 相除就得到 $\sqrt{(Pa \cdot Pb)}$ 。

直觉：价格上涨时你在持续卖出，越涨手里 ETH 越少 → 高价段卖得少 → 均价被拉低。

区间越宽，这个偏差越大。

2. 验证：单边 LP 本质是“增强版限价单”

同样卖 5 ETH：

\ 做法 | 收到 USDC |

\---|---|

\ 限价单挂 1997 (最低价) | 9,985.00 |

| 单边 LP 1997~2187 | 10,449.21 |

\ 限价单挂 2187 (最高价) | 10,935.00 |

位于两者之间 (比挂最低价多 4.65%)，外加区间内手续费。

所以它不是“高年化理财”。正确用法是：先有交易意图 (“涨到 2000 以上我要减仓”)，

再用单边 LP 去执行它 —— 手续费是附加收益，不是目的。

建仓前必须自问：我是否愿意在这个价位区间成交？答案是“不愿意”就说明仓位是错的。

3. 验证：三种价格路径，三种结局

\

ETH 价 持有 ETH 持有 USDC 总值 vs 一直持有

1,997 5.0000 0 9,985 +0 ← 刚触及

2,092 2.4148 5,284 10,336 -124 ← 正在成交

2,187 0.0000 10,449 10,449 -486 ← 完全穿透

2,400 0.0000 10,449 10,449 -1,551 ← 卖早了

\

穿透后继续涨 → 少赚 1,551，和限价单一样的性质

穿透后跌回来 → 1901 → 2200 → 跌回 1901 锁定 10,449，持有只值 9,508，赚 941 (最理想)

朝反方向走 → 1901 → 1700 你还是 5 ETH，且一分手续费没赚到 (最常被忽略)

4. 推翻我自己： amountXMin 在单边场景下不是“滑点容忍”

昨天我把链上一笔真实交易的 $\text{amount0Min} / \text{amount0Desired} = 95.78\%$ 描述成

“滑点容忍 4.22%”，这个说法不准确。

$\text{mint}()$ 和 $\text{swap}()$ 的语义完全不同：

\ | 保护什么 |

\---|---|

\ swap | 我付固定的钱，保护收到的量 |

\ mint | 我给一个上限，保护被用掉的量 |

而单边场景下它进一步退化。实算价格从下界往上走：

\

当前价(SAND/WETH) 实际用掉SAND 通过4.22%检查?

46,141 28,723.09

46,095 0.00 REVERT

45,684 0.00 REVERT

\

没有中间状态。因为 $L = \min(L_from_amount0, L_from_amount1)$,

$amount1=0 \rightarrow L=0 \rightarrow$ 用掉 0 $\rightarrow$ 必然 revert。

所以单边场景下 $amountXMin$ 是个"价格越界即回滚"的二元开关,

任何 >0 的值效果相同。那 4.22% 非整数, 说明是前端按规则算的, 不是用户手填。

真正需要缓冲的是双边仓位 —— 价格小幅移动时比例变化但仍能建仓。

5. 写了 calldata 生成器, 用链上真实交易验证

\

\=== selftest: 复现链上真实交易的 calldata ===

基准 tx 0x856317...f66bc (SAND/WETH 1%, 单边 28723.09 SAND)

逐字节一致: 通过 (714 字符)

\=== 价格 <-> tick 往返验证 ===

tick -107400 -> 价格 46,141.27 -> tick -107,400.0000 偏差 0.000000

\

基准值直接 $\text{cast tx} \langle \text{hash} \rangle \text{input}$ 取, 不手抄 —— 第一次手抄进文档字符串漏了 28 个字符, 自检直接失败。

脚本的核心不是生成 calldata, 是方向检查。因为单边最危险的错误是方向设反, 而 $amountXMin$ 拦不住这类错误 (量确实用上了, 你只是被动完成了一笔反向交易)。

脚本查 $\text{factory.getPool}() \rightarrow \text{slot0().tick}$ 判断区间在不在正确一侧,

设反时明确报错。也会拦住"现价已在区间内"的情况。

过程中改掉一个错: 第一版输出说"逐步卖出 WETH", 但投入的是 USDC,

实际是买入 WETH —— 文案完全说反了。

代码逻辑对了、人话说错了, 一样会导致做错决策。

6. 证伪: USDC/SEI (昨天的头号候选)

昨天它是双条件榜首 (覆盖 2.40x、净 APR 43.56%)。深挖后发现建立在统计假象上:

\

逐日活跃 L: 2.76e16 $\sim$ 4.71e19 $\leftarrow$ 相差 1700 倍

我的份额: 0.510% $\sim$ 91.088%

均值 7.055% | 中位数 0.639% $\leftarrow$ 差 11 倍

\

某一天池子流动性塌到极低, 使 1 万美元的份额达 91% —— 那天手续费只有 \$184, 却被算成我几乎全拿。均值被这一天绑架。

\| 口径 | 份额 | 费率 APR | 覆盖率 | 判断 |

\|---|---|---|---|

| 均值 | 7.055% | 78.66% | 2.60x | 值得做 |

| 中位数 | 0.639% | 12.95% | 0.43x | 亏损 |

从"值得做"翻转成亏损。与我在资金费率里发现的 SKR 案例完全同构 ——

瞬时值是噪声, 中位数才是可持续水平。

我原本假设 74% 是靠成交量尖峰撑起来的, 查下来发现尖峰确实存在但不在窗口内

(前 3 高成交日占 41 天总量 43.6%, 全在 30 天窗口之外)。假设错了, 但找到了更严重的问题。

另外两个警告: 成交量从最早 14 天的日均 \$794,504 衰减到最近 14 天 \$225,353 (28%),

TVL 8.4%。

已修进扫描器：最终筛选改用中位数口径 + 要求 \均值/中位 < 3x\。

USDC/SEI 被淘汰，候选从 12 个降到 11 个。

7. 拆解一笔 90 跳的 MEV 套利

\0x6dc21ad762f5403369fa9475f455a3e6822ab1f4f4eaa8ae6bfc199d5bbd53ee\

\ 项 | 值 |

\---|---|

| 区块位置 | #1 (区块第二笔) |

| EOA nonce | 932,374 ← 发过 93 万笔交易 |

| swap 跳数 | 90 跳，横跨 V2+V3+V4 |

\ log 数 | 448 条 (288 个 Transfer) ， 36 种代币有净流动 |

| gasUsed | 10,151,599 ← 约占区块 gas 上限三分之一 |

| gasPrice | 895.25 gwei |

净额核算 (统计 448 条 log 里所有 Transfer 对套利合约的净流)：

\

收入 +2,621.75 WETH = \$4,551,933

支出 -1,753,467 USDT = \$1,753,467

支出 -1,524,270 USDC = \$1,524,270

支出 -17.12 WBTC (≈648 ETH) = \$1,125,009

毛利 +\$149,186

gas -\$15,779

净利 +\$133,407

\

WBTC 折算我做了交叉核对：交易内 \fwWBTC/fwWETH = 37.85\ 隐含 BTC = \$65,716，

与我之前从 WBTC/USDC 池查到的 \$64,600 只差 1.7% —— 合理，估值可信。

三个专业性标志：

1\ \value\ 字段编码了区块号 (\value = 25322434 = blockNumber\) —— 合约内部校验技巧

2. 利润留在合约里不转出 EOA (我查了向外的大额 WETH 转账，一笔都没有) ——

省掉转账 gas，正是 V4 ERC-6909 claim token 的思路

3. priorityFee 895.19 gwei vs baseFee 0.064 gwei —— 小费是基础费的 14,000 倍

最值得注意的一点：它主动把 10.6% 的毛利 (\$15,779 / \$149,186) 交给验证者，为的是抢到区块 #1。这不是可选成本，是入场费。

今天最重要的认知

这笔交易恰好是我结构性打不赢的那一类，而它的存在证明了 LVR 的资金流向。

\ 门槛 | 这笔交易 | 我的条件 |

\---|---|---|

\ 单笔资金 | ~\$440 万 | 几百美元 |

\ gas 预付 | 9 ETH (\$15,779) 失败也要付 | 全部资金的几十倍 |

\ 路径复杂度 | 90 跳跨三个版本 | — |

\ 反应速度 | 区块 #1 | — |

\ 基建 | 专用合约 + 93 万笔历史 | — |

那 14.9 万毛利，很大一部分正是从 LP 手里赚来的 ——

我前面算的 USDC/WETH 池 436% LVR，就是流向这种交易的。

能力归因：这笔钱赚到靠的是基建 (90 跳路径搜索 + V4 flash accounting)

\+ 资金 (440 万) + 出价能力 (10.6% 利润给验证者)，

而不是某个别人不知道的洞察。这三样我一样都没有。

认清这点比收藏案例有用。它同时给 LP 方向敲了警钟：

在这些机器人面前，被动 LP 是被收割的一方。

一个方法论收获

"份额接近 100%" 是单位错误或数据异常的信号，不是好消息。

昨天扫描器因这个断言拦下 157 个错误组合，今天 USDC/SEI 又因它被证伪。

任何分母敏感的量（份额、占比、杠杆倍数）都应该：

1\、看中位数而非均值

2\、加均值/中位比值检查，>3x 就说明被异常值绑架

笔记 011: 链上套利残酷共学 · Day 4 打卡目标 (2026-08-08)

作者：jinnzy | 时间：2026-08-08 12:27 UTC | 字数：547 | 评分：54

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

链上套利残酷共学 · Day 4 打卡目标 (2026-08-08)

今天研究 Robinhood Chain 上一笔 Uniswap v4/v3 三角套利：机器人通过 v4 unlock() 回调借出 ETH，完成 ETH → 222 → USDG → WETH，同一笔交易内偿还借款并转出约 0.000165725 ETH 毛利润。该案例说明，它不是传统 Ethereum L1 的公开 mempool 抢跑，而是依赖本地池状态、精确 AMM 模拟和低延迟 sequencer 执行的原子套利。

今天的学习目标是把案例抽象成可验证的扫描架构：索引 v3/v4 全量活跃池和 token，构建价格图搜索 3~5 跳闭环，用 tick、流动性和手续费做精确模拟，再扣除 Gas、滑点、价格冲击与失败成本，最后只输出告警或 Paper Trading 结果。验收标准是能够按历史状态重建 ETH → 222 → USDG → WETH 路径，并分别算出成交量、毛利润、Gas 和净利润。

证据边界：交易路径、金额、Gas 和状态可由链上复核；扫描策略、sequencer feed、bot 源码和真实基础设施成本仍待确认。本阶段不接私钥、不上链执行，先完成只读扫描与回测。

笔记 012: 假设同一个 ETH 在两个 DEX 的池子里价格不同：

作者：oetn | 时间：2026-08-08 12:48 UTC | 字数：1226 | 评分：66

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

假设同一个 ETH 在两个 DEX 的池子里价格不同：

DEX A : 1 ETH ≈ 2,000 USDC

DEX B : 1 ETH ≈ 2,100 USDC

最直观的套利流程是：

1. 在便宜的 DEX A 买入 ETH
2. 把 ETH 卖到贵的 DEX B
3. 收回更多 USDC
4. 扣除手续费、Gas 和价格冲击
5. 剩下的才是净利润

用一笔交易计算

假设 DEX A 有：

100 ETH

200,000 USDC

你想买 1 ETH。

根据：

$$x \times y = k$$

如果忽略手续费，卖出 1 ETH 后，池子剩下 99 ETH。为了保持乘积不变，池子需要有：

$$200,000 \times 100 \div 99$$

$$\approx 202,020 \text{ USDC}$$

所以你大约需要支付：

$$202,020 - 200,000$$

$$\approx 2,020 \text{ USDC}$$

也就是说，虽然池子显示价格是 2,000 USDC，但你实际买 1 ETH 需要约 2,020 USDC。

如果在 DEX B 卖出 1 ETH，实际收到约 2,073 USDC，扣除 Gas 10 USDC：

净利润 $\approx 2,073 - 2,020 - 10$

≈ 43 USDC

注意：这里还没有完整考虑两边手续费、交易延迟和第二个池子的价格变化，所以 43 USDC 只是简化估算。

这个 Demo 真正想教你的东西

套利不是：

便宜买，贵卖

而是：

便宜市场的实际买入成本

<

贵市场的实际卖出收入

所有执行成本

如果两笔交易在同一条链上，可以尝试放进一个智能合约交易中：

买入 → 卖出 → 检查最终余额

只要其中一步失败，整笔交易回滚。这叫接近“原子执行”。

如果是在两条不同的链上，就不能简单回滚：

链 A 买入成功

→ 桥接或转移

→ 链 B 卖出时价格已经变化

这时你承担的是跨链期间的单边价格风险。

今天可以直接提交的笔记

【Day 4 | DEX 间套利】

今天学习了

DEX

间套利的基本流程：在价格较低的市场买入资产，再到价格较高的市场卖出，最后扣除手续费、Gas、滑点和延迟成本。

我发现“页面价格差”不等于“套利利润”。实际分析时，必须分别计算：

1. 真实买入成本；
2. 真实卖出收入；
3. 两边交易手续费；
4. Gas；
5. 交易规模造成的价格冲击；
6. 执行期间的价格变化。

同一条链上的两步交易有机会放在一个原子交易中完成；跨链套利则通常无法原子回滚，因此会承担桥接和价格波动风险。

今天最重要的结论是：

套利机会不是价格差，而是扣除完整执行成本后仍然为正的可执行价差。

我还没有解决的问题是：如何自动同时获取两个 DEX 的报价，并判断这个价差在下一笔交易发生前是否仍然存在。

今天只需要记住一句话：

先算“实际买入多少钱、实际卖出多少钱”，再谈套利

笔记 013: 问题一：什么是 AMM、订单簿、流动性深度、价格冲击和区块确认？它们为什么是理解链上套利必须掌握的基

作者：A9xYau | 时间：2026-08-08 14:36 UTC | 字数：4824 | 评分：89

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

问题一：什么是 AMM、订单簿、流动性深度、价格冲击和区块确认？它们为什么是理解链上套利必须掌握的基础？

回答：

在学习链上套利（On-chain

Arbitrage）之前，必须理解市场价格是如何产生的，以及一笔交易为什么可能无法按照你看到的价格成交。

链上交易和传统交易所一个非常大的区别：

传统交易所主要使用：

订单簿 (Order Book)

而大部分 DEX (去中心化交易所) 使用：

AMM (Automated Market Maker, 自动做市商)

因此套利机器人必须同时理解：

AMM 定价机制；

订单簿撮合机制；

流动性深度；

价格冲击；

区块确认和交易执行时间。

这些因素共同决定：

“看到的套利机会”是否真的可以赚钱。

一、什么是 AMM (Automated Market Maker, 自动做市商) ？

回答：

AMM 是去中心化交易所 (DEX) 中最核心的价格形成机制。

传统交易所：

例如：

Binance、Coinbase

采用：

订单簿模式。

而：

Uniswap、PancakeSwap、Curve

主要采用：

AMM 模式。

1. 订单簿模式的问题

传统订单簿需要：

买方：

我要用 3000 USDT 买 ETH

卖方：

我要用 3010 USDT 卖 ETH

交易所负责：

撮合买卖双方。

如果没有卖家：

买家无法成交。

如果没有买家：

卖家无法成交。

因此需要：

大量市场参与者提供流动性。

2. AMM 如何交易？

AMM 不需要买卖双方同时存在。

它使用：

流动性池 (Liquidity Pool)

例如：

ETH/USDC 池：

ETH:

100

USDC:

300000

池子中的资产比例决定价格。

公式：

$$x \times y = k$$

其中：

x：

ETH 数量

y：

USDC 数量

k：

常数

例如：

池子：

100 ETH

300000 USDC

那么：

1 ETH 初始价格：

$$300000 / 100$$

$$= 3000 \text{ USDC}$$

如果有人买 ETH：

他投入 USDC。

池子：

USDC 增加。

ETH 减少。

因为：

$$x \times y = k$$

所以：

ETH 的价格会上升。

3. AMM 为什么会产生套利？

因为不同 AMM 池之间不会自动同步价格。

例如：

Uniswap：

$$\text{ETH} = 3000 \text{ USDC}$$

PancakeSwap：

$$\text{ETH} = 3010 \text{ USDC}$$

套利机器人：

低价池买入 ETH

↓

高价池卖出 ETH

↓

赚取价差

套利者实际上帮助不同市场恢复价格一致。

二、什么是订单簿（Order Book）？

回答：

订单簿是中心化交易所（CEX）主要采用的交易方式。

例如 Binance :

订单簿记录 :

所有买单 :

Bid

3000 买 10 ETH

2999 买 20 ETH

2998 买 50 ETH

所有卖单 :

Ask

3001 卖 5 ETH

3002 卖 20 ETH

3003 卖 50 ETH

最高买价 :

叫 :

Bid Price

最低卖价 :

叫 :

Ask Price

两者之间 :

叫 :

Spread (买卖价差)

订单簿如何成交 ?

例如 :

当前 :

卖一 :

3001 USDT

5 ETH

你提交 :

市价买入 :

10 ETH。

那么 :

第一部分 :

5 ETH

价格 :

3001

第二部分 :

继续吃 :

3002 的卖单。

最终成交价格 :

可能 :

3001.5

而不是 :

屏幕显示的 :

3001。

三、什么是流动性深度 (Liquidity Depth) ?

回答 :

流动性深度表示：

市场中有多少资金可以被交易，而不会导致价格大幅变化。

简单理解：

流动性越深：

大额交易越容易成交。

流动性越浅：

价格越容易被推动。

例如：

两个 ETH/USDC 池：

池 A：

10000 ETH

3000万 USDC

流动性很深。

买：

100 ETH

价格影响很小。

池 B：

10 ETH

30000 USDC

流动性很浅。

买：

5 ETH

可能直接推动价格上涨。

因此：

同样看到：

$\text{ETH} = 3000$

并不代表：

你可以买很多 ETH。

真正的问题是：

买多少以后，平均成交价格是多少？

四、什么是价格冲击（Price Impact）？

回答：

价格冲击表示：

由于你的交易规模相对于流动性太大，导致交易价格发生变化。

例如：

当前 ETH：

3000 USDC

你想买：

10000 USDC 的 ETH。

如果池子流动性足够：

成交：

接近：

3000。

但是如果池子很小：

你的买入会改变池子比例。

结果：

可能：

第一部分：

3000

第二部分：

3010

第三部分：

3025

最终平均：

3015。

这就是：

Price Impact。

为什么套利机器人必须计算价格冲击？

因为很多套利机会只是：

“理论价格差”。

例如：

看到：

Uniswap：

ETH：

3000

Binance：

ETH：

3010

感觉：

套利：

10 美元。

但是实际：

买入：

100 ETH。

可能：

因为滑点：

平均成本：

3008。

卖出：

3010。

扣除手续费：

可能亏钱。

所以：

真正计算的是：

实际卖出收入

-

实际买入成本

-

手续费

-

Gas

=

净利润

五、什么是区块确认 (Block Confirmation) ?

回答：

区块确认表示：

你的交易什么时候真正被区块链接受。

链上交易不是：

发送出去立即完成。

流程：

用户创建交易

↓

发送 RPC

↓

进入 Mempool (交易池)

↓

等待矿工/验证者打包

↓

进入区块

↓

确认

例如：

你发现：

ETH：

有 10 美元套利机会。

你提交交易。

但是：

等待 10 秒。

期间：

其他套利机器人：

已经完成交易。

价格：

恢复。

你的交易：

可能：

亏损。

因此链上套利非常依赖：

RPC 延迟；

Mempool 监听；

Gas 设置；

交易排序；

区块时间。

问题二：为什么必须区分“屏幕上的价差”和“可以执行的净收益”？

回答：

这是链上套利最重要的思维。

很多初学者看到：

DEX:

ETH = 3000

CEX:

ETH = 3010

认为：

存在：

10 美元套利。

实际上：

这只是：

屏幕价差（Displayed Spread）。

并不代表：

真实利润。

一、什么是屏幕价差？

回答：

屏幕价差：

就是你看到的数据：

例如：

DEX：

ETH

3000

CEX：

ETH

3010

价差：

10 USDT

但是这个价格通常只是：

最新成交价；

当前报价；

小额交易价格。

不代表：

你的大额交易可以按照这个价格成交。

二、什么是真实可执行收益？

回答：

真实套利收益：

必须考虑：

1. 买入价格变化；

2. 卖出价格变化；

3. 流动性；

4. 手续费；

5. Gas；

6. 延迟风险。

公式：

净收益

=

卖出实际收入

-

买入实际成本

-

DEX手续费

-

CEX手续费

-

Gas费用

-

滑点损失

-

失败成本

三、一个实际例子

假设：

看到：

DEX ETH:

3000

CEX ETH:

3010

理论利润：

10 USDT/ETH

准备套利：

100 ETH。

第一步：

DEX 买入。

理论：

100×3000

\=

300000 USDT

但是由于流动性：

实际：

平均：

3006

成本：

300600 USDT

第二步：

转到 CEX 卖出。

价格变化：

3010

变成：

3007

卖出：

300700 USDT

第三步：

扣除成本：

买入：

300600

卖出：

300700

毛利润：

100 USDT

扣除：

Gas :

30

手续费 :

50

最终 :

20 USDT。

原本看到 :

1000 USDT 利润。

实际 :

只有 :

20 USDT。

四、为什么专业套利机器人必须模拟成交？

因为 :

不能只看 :

Ticker。

必须计算 :

真实成交路径。

例如 :

DEX :

需要模拟 :

AMM swap ;

当前池子储备 ;

滑点 ;

Gas。

CEX :

需要模拟 :

Order Book 深度 ;

吃多少档价格 ;

手续费。

专业系统通常包含 :

价格发现模块

↓

流动性分析模块

↓

成交模拟模块

↓

利润计算模块

↓

风险过滤模块

↓

执行模块

五、链上套利者真正寻找的是什么？

不是 :

“价格差”。

而是 :

可执行价格差

也就是 :

扣除所有成本之后：

仍然赚钱的机会。

因此：

普通用户看到：

ETH:

3000 vs 3010

看到的是：

10 美元机会。

专业套利机器人看到的是：

买入100 ETH实际成本：

3006

卖出100 ETH实际收入：

3007

Gas：

20

手续费：

30

最终：

亏损43美元

总结：

理解链上套利，必须牢记：

1. AMM 决定 DEX 的价格形成方式；
2. 订单簿决定 CEX 的价格形成方式；
3. 流动性深度决定大资金是否能按照当前价格成交；
4. 价格冲击决定真实成交价格；
5. 区块确认决定套利机会是否还能存在；
6. 屏幕价差不等于利润；
7. 只有扣除所有成本后的净收益，才是真正可以执行的套利机会。

真正优秀的套利系统，不是在寻找：

“哪里价格不同”。

而是在寻找：

“哪里存在经过交易模拟后仍然具有正收益的执行机会”。

笔记 014: 今天看了Uniswap的白皮书，从V2的 $x \setminus y = k$ 的模型，看到V3的tick类的CLMM模型

作者：Jako | 时间：2026-08-08 14:37 UTC | 字数：317 | 评分：45

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天看了Uniswap的白皮书，从V2的 $x \setminus y = k$ 的模型，看到V3的tick类的CLMM模型。结合昨天看的分享，理解了这类DEX的运行机制，现在基本上理解了原理，但是对询价、交易的接口还没有详细看，后续如果要自己实现路由、三角套利计算，还是要看下具体的接口。

还了解了Uniswap还有一个V4版本，支持hook，后续再研究一下这里面有没有什么特别的玩法。

目前对pumpfun这类发射平台只是了解了大概，对内部的具体机制实现还是没有详细了解，昨天听到LP大佬的分享，感觉这块的机会还是需要了解一下。

现在把套利地图越阔越大了，决定先广撒网，了解清楚各种链上玩法的内部逻辑，从底部开始了解，才能更好地看懂各种套利机制。

笔记 015: Day 4 | 集中流动性、可执行深度与股票代币机会观察

作者：Prousty Wu | 时间：2026-08-08 14:39 UTC | 字数：761 | 评分：55

Day 4 | 集中流动性、可执行深度与股票代币机会观察

今天从“看池子页面”进一步转向“验证可执行机会”。

1. 理解了 V3 集中流动性：LP 把资金放进特定价格区间，提高现价附近的资本效率；离开区间后头寸停止赚取手续费，并可能变成单边资产。V4 沿用集中流动性思想，但通过 PoolManager、Flash Accounting 与 Hooks 重构池子架构，并不等于自动更智能。
 2. 区分了 TVL、24h Volume、活跃深度和交易规模。TVL/历史成交量不能替代指定 amountIn 的真实报价；价格冲击取决于方向、规模以及交易路径会穿过哪些 Tick。
 3. 梳理了股票代币的锚定条件：要检查法律权利、托管与储备、申购赎回机制、KYC/白名单、交易时间、转账限制、代币精度、股票拆分和公司行动。同名代币不代表同一资产，核心身份是 chainId + contractAddress，再叠加发行条款和可兑换关系。
 4. 建立了从 DEX Screener 到真实机会的验证流程：官方地址 → 候选池 → fee/Tick/Hook/活跃流动性 → 同一 blockTag 的规模化报价 → 完整交易模拟 → Gas、融资、排序与失败成本。相同 blockTag 只保证读取可比，不保证未来成交。
- 今日观察 Robinhood Chain 的 NVDA/USDG：主 V4 与主 V3 页面价格约差 0.66 bps；低流动性尾部 V4 与主 V3 约差 53.40 bps，但这只能作为候选信号，不能直接视为利润。
- 下一步：分别以 100 / 1,000 / 5,000 / 10,000 USDG 做双向闭环报价，记录 amountOut、价格冲击、Gas、成功概率与每次尝试的期望净收益。

笔记 016: 链上套利 (On-Chain Arbitrage) 是指利用区块链网络中不同去中心化交易平台 (DEX)、

作者 : wodeche | 时间 : 2026-08-08 14:47 UTC | 字数 : 2050 | 评分 : 73

来源 : <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

链上套利 (On-Chain Arbitrage) 是指利用区块链网络中不同去中心化交易平台 (DEX)、流动性池 (Liquidity Pool) 或跨链桥之间的代币价格差异，通过触发智能合约执行买卖交易，从而无风险或低风险获取差价收益的自动化交易策略。

要理解链上套利，需要掌握以下 5 个核心基础概念：

1. 自动做市商与定价机制 (AMM & Pricing)

链上 DEX (如 Uniswap、Sushiswap、PancakeSwap) 与传统中心化交易所 (CEX) 的订单簿模式不同，大多数采用自动做市商 (AMM) 机制。

恒定乘积公式 ($\$x \times y = k$)：流动性池中两种代币的数量乘积维持常数。当有人在一个池子里用 $\$x$ 换取 $\$y$ 时， $\$x$ 的数量增加， $\$y$ 的数量减少，导致 $\$y$ 的相对价格升高。

套利机会的产生：当大额交易发生在池 A 时，池 A 内的价格会剧烈波动；而池 B 尚未更新价格。此时，池 A 与池 B 之间就出现了短时间的价差。

2. 核心套利类型 (Arbitrage Strategies)

根据交易路径和市场结构，常见的链上套利分为三类：

1. 空间套利 (Spatial)：DEX A (低价买入) → DEX B (高价卖出)
2. 三角套利 (Triangular): Token A → Token B → Token C → Token A
3. 跨链套利 (Cross-Chain): Chain A (如 Ethereum) → 跨链 Bridge → Chain B (如 Arbitrum)

空间套利 (同链 DEX 间套利)：相同的代币对 (如 ETH/USDT) 在 Uniswap 和 Sushiswap 上存在价差。在价格低的平台买入，在价格高的平台卖出。

原子三角套利 (Triangular Arbitrage)：在同一个 DEX 或不同 DEX 上，通过 3 个或更多代币池完成环形交易 (例如： $\$ \text{USDC} \rightarrow \text{ETH} \rightarrow \text{WBTC} \rightarrow \text{USDC}$)，最终换回更多数量的原代币。

跨链套利 (Cross-Chain Arbitrage)：利用不同区块链网络 (如 Ethereum 与 Arbitrum/Optimism) 之间的同种代币价差套利，通常需考虑跨链 bridge 的延时与手续费。

3. 闪电贷 (Flash Loans)

闪电贷是链上套利最重要的独特工具，允许交易者无需任何抵押品即可借出巨额资金 (如数百万美元)，前提是必须在同一个区块 (同一笔交易) 内完成借款、套利和还款 (附带利息)。

原子性 (Atomicity)：

如果套利交易计算失误、利润不足以支付利息或遭遇滑点，整笔交易会直接回滚 (Revert)，就像从未发生过一样。

风险边界：借款人不会损失借贷本金，仅会损失触发交易失败时消耗的微量 Gas 费。

4. MEV 与区块空间竞争 (MEV & Gas Racing)

最大可提取价值 (MEV / Max Exchange Value) : 矿工/验证者 (以及套利 Bot) 通过在区块中重新排列、插入或排除交易所能获得的额外收益。

抢跑 (Front-Running) 与 夹心攻击 (Sandwich Attack) : 搜索者 (Searcher) 发现链上未确认的公开套利机会或大额单后, 提高 Gas 费将自己的交易排在前面执行。

优先 Gas 拍卖 (PGA) 与 Flashbots : 为了避免在公开内存池 (Mempool) 中竞价打 Gas 战并导致大量失败交易, 现在的链上套利 Bots 大多使用 Flashbots 等私密 RPC 通道, 将交易直接打包发送给验证者。

5. 成本与关键风险 (Costs & Risks)

链上套利并非绝对无风险, 实际操作中需精算以下成本与风险:

关键要素

说明

Gas 费成本

复杂智能合约交互的 Gas 费较高, 若价差过小可能导致净收益为负。

交易滑点 (Slippage)

交易执行过程中, 池内流动性变化导致实际成交价劣于预期价。

交易失败 (Revert Rate)

在竞争极其激烈的环境中, 如果被其他 Bot 抢先执行, 你的交易可能会失败并白白浪费 Gas 费。

智能合约风险

调用的新协议或自定义套利合约可能存在代码漏洞或闪电贷攻击风险。

笔记 017: Day 4 : Pool 基础状态

作者: edwar-glowing | 时间: 2026-08-08 15:00 UTC | 字数: 334 | 评分: 46

来源: <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 4 : Pool 基础状态

学习

理解:

Pool 保存什么

token0 / token1

fee

liquidity

slot0

tick

sqrtPriceX96

不推导复杂数学, 只要求知道各字段作用。

Action

读取一个 Uniswap V3 Pool :

token0

token1

fee

liquidity

slot0

把原始数据保存到 SQLite。

当天产物

04_pool_state_reader.py

arbitrage.db

[图片]

笔记 018: 链上套利残酷共学 Day 1 : 建立统一套利判断框架

作者 : blimnwind-bot | 时间 : 2026-08-07 19:36 UTC | 字数 : 3464 | 评分 : 78

来源 : <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

链上套利残酷共学 Day 1 : 建立统一套利判断框架

第一天我先不急着找具体的套利机会，也不急着写 Bot。

今天先解决一个更基础的问题：

到底什么样的机会，才能称为一个值得研究的套利机会？

过去看到 DEX 套利、跨链套利、三角套利、清算、MEV、资金费率、Carry Trade、波动率曲面等策略时，很容易直接关注收益率。

但真正开始研究以后，我觉得应该先建立一套统一的判断框架。

以后无论看到什么策略，都用同一套标准拆解。

一、先区分毛利润、净利润和期望利润

最简单的套利可以写成：

毛利润 = 最终资产价值 - 初始资产价值

但毛利润基本没有实际意义。

真正需要计算的是：

净利润 = 毛利润 - 全部执行成本

执行成本至少包括：

Gas\

交易手续费\

DEX LP Fee\

跨链费用\

滑点\

价格冲击\

借贷成本\

资金费率\

对冲成本\

资金占用成本

再进一步，还需要考虑失败。

所以真正值得看的其实是：

期望利润 = 成功概率 × 成功利润 - 失败概率 × 失败损失 - 风险准备金

例如一个机会显示可以赚 100 USDC。

如果完成交易需要：

Gas 15\

手续费 10\

滑点 20\

价格冲击 15\

其他成本 10

实际只剩 30 USDC。

如果这个机会还有较高概率因为报价变化、抢跑、跨链延迟或者交易失败而亏掉 Gas，那么它的期望收益可能已经接近 0。

所以以后看到“价差 1%”，第一反应不应该是 1% 的利润。

第一反应应该是：

这个 1% 最后能够留下多少？

二、第二个问题：交易路径能不能闭环

任何套利都必须画出完整资金路径。

例如：

USDC → ETH → USDT → USDC

或者：

Arbitrum USDC → ETH → Bridge → Base ETH → USDC

重点不是中间某一步赚钱多少。

重点是：

我投入什么资产，最后能不能重新得到同一种计价资产，而且数量增加。

因此我准备先固定使用 USDC 作为统一计价单位。

以后所有策略最终都转换成：

投入多少 USDC

最后可以兑现多少 USDC

这样不同套利策略之间才能直接比较。

三、第三个问题：原子执行还是非原子执行

这个区别非常重要。

原子套利

如果所有操作能够在同一笔交易中完成：

$A \rightarrow B \rightarrow C \rightarrow A$

任何一步失败，整个交易一起回滚。

这种结构最干净。

主要风险集中在：

Gas\

区块状态变化\

MEV 竞争\

交易能不能进入目标区块

非原子套利

例如跨链：

Arbitrum 买入\

→ Bridge\

→ 等待\

→ Base 到账\

→ 卖出

第一步成功以后，后面的交易可能失败。

桥接过程中价格也可能已经发生变化。

所以非原子套利必须额外考虑：

价格风险\

时间风险\

库存风险\

跨链风险\

对冲成本

这也是为什么看到跨链价差，并不代表这个价差真的能够吃到。

四、第四个问题：成功成本和失败成本必须分开计算

以前比较容易只算成功以后赚多少钱。

但套利系统还必须回答：

失败一次会亏多少钱？

成功成本包括：

Gas\

交易手续费\

桥费\

滑点\

价格冲击\

借贷费用\

对冲费用

失败成本可能包括：

已经支付的 Gas\

第一腿成交以后无法完成第二腿\

跨链以后资产卡在目标链\

需要额外交易恢复仓位\

对冲损失\

机会消失后的强制平仓损失

所以以后每一个策略，我都会同时记录：

成功赚多少

以及

失败亏多少

五、我的七道套利判断闸门

以后看到任何套利机会，先过这七关。

第一关：路径能不能闭环

最终能不能重新变成统一计价资产。

第二关：利润为什么存在

为什么两个市场价格不同？

是谁在支付这笔利润？

为什么其他套利者还没有把它消掉？

如果连利润来源都解释不了，就先不碰。

第三关：交易规模能不能放大

100 USDC 能赚钱，并不意味着 10,000 USDC 也能赚钱。

我要分别测试：

100\

500\

1,000\

5,000\

10,000 USDC

然后观察：

价格冲击\

滑点\

流动性\

净利润

最终寻找的是：

利润最大的交易规模，而不是价差最大的交易。

第四关：扣掉全部成本以后还剩多少

以后统一计算：

表面价差

减 Gas

减手续费

减滑点

减价格冲击

减桥费

减借贷和资金成本

减对冲成本

最终得到真实净利润。

第五关：机会能活多久

一个机会即使有 2% 利润，如果只能存在 50 毫秒，而我的系统需要 2 秒才能完成交易，对我来说它就是不存在的。

所以还需要测：

发现机会时间

获取报价时间

计算时间

模拟时间

发送交易时间

区块确认时间

跨链到账时间

整个套利真正研究的是：

利润空间和时间窗口之间的关系。

第六关：失败以后会发生什么

我要提前模拟：

第一腿成功，第二腿失败

Gas 突然上涨

报价失效

交易被抢先

桥接变慢

目标链流动性下降

RPC 出问题

资产到了另一条链但已经无法盈利

如果这些情况发生，最大损失是多少。

第七关：这个机会能不能重复

一次赚到钱只能叫事件。

能够重复出现，才能开始讨论策略。

最终需要统计：

每天出现多少次

平均持续多久

平均利润多少

成功率多少

最大容量多少

失败一次亏多少

竞争增加以后还有没有利润

六、套利类型地图

目前我先把准备研究的方向整理成几个大类。

价格差套利

CEX 与 CEX\

CEX 与 DEX\

DEX 与 DEX\

同一 DEX 不同池\

三角套利

核心变量：

价格差 × 流动性 × 执行速度

跨链套利

不同链之间的价格差\

桥接流动性差异\

跨链稳定币价差\
聚合器路由差异\
预置库存套利

核心变量：

价差 $\times$ 跨链时间 $\times$ 库存配置

状态型套利

清算\
稳定币脱锚\
协议再平衡\
预言机更新\
强制交易

核心思想是寻找：

被迫交易者。

MEV

Backrun\
套利\
清算\
交易排序\
Bundle

这里的核心已经从单纯“有没有价差”，进一步变成：

谁能够先捕获这个状态变化。

Carry Trade

现货与永续\
不同交易所资金费率\
期货 Basis\
借贷利率差\
不同链上的资金成本差异

核心不是赚瞬间价格差，而是：

持有一个市场中性的结构，持续获得 Carry。

波动率曲面

期限结构\
Skew\
Smile\
不同执行价 IV\
不同到期日 IV\
跨平台期权定价差

这里交易的不再只是价格，而是：

波动率本身的相对定价。

七、今天建立统一机会评分卡

以后看到任何机会，我准备统一记录：

策略名称：

套利类型：

涉及链：

涉及协议：

初始资产：

最终资产：

完整路径：

是否原子执行：

理论价差：

毛利润：

Gas：

手续费：

滑点：

价格冲击：

桥接费用：

资金成本：

其他成本：

预计净利润：

失败最大损失：

机会持续时间：

可执行规模：

预期成功率：

利润为什么存在：

利润由谁支付：

什么力量会让机会消失：

竞争者是谁：

需要什么数据验证：

什么条件出现以后直接否定这个策略：

今天最重要的产出不是找到一个套利机会。

而是建立一套以后可以反复使用的判断方式。

以后无论看到：

DEX 套利\

跨链套利\

三角套利\

清算\

MEV\

Carry Trade\

波动率曲面

都先放进同一个框架：

机会来源 → 完整路径 → 可执行价格 → 交易规模 → 全部成本 → 执行时间 → 失败损失 → 期望收益 → 可重复性
先判断这是不是一个真实存在的机会，再讨论怎么写代码把它抓出来。

笔记 019: 今天主要做了一个初始的成本计算器，很多参数都没有扩展，目的是只做验证。

作者：Montaire | 时间：2026-08-08 00:32 UTC | 字数：1102 | 评分：62

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天主要做了一个初始的成本计算器，很多参数都没有扩展，目的是只做验证。

- 1\ 主流币在主流链之间的跨链套利是结构性亏损，不是利润薄。真实可执行价差连过桥损耗的三分之一都不到。市场早被机器人磨到比桥费还低，这条路不用再看。
- 2\ 那点价差还是噪声，不是价格错位。相隔一分钟重扫，各链的贵贱排名整个翻过来。一个不可重复的价差不是机会
- 3\ 成本项排序被彻底推翻：Gas 是最小的一项，小到可以忽略，差了两三个数量级。排在前面的是过桥损耗和「选错聚合器」，而后者我原本的模型里根本没有这一项。
- 4\ 「滑点」这个词把两种成本混在一起了，必须拆开：纯滑点（市场深度不够）和路由损耗（选错聚合器）。拆开后果很干净——主流币上纯滑点近似为零，路由损耗是它的几倍到几十倍；长尾币上反过来，纯滑点绝对主导。所以主流币纠结选对聚合器，长尾币纠结压小规模。
- 5\ 多一层聚合不等于更优。聚合器的聚合器在小额时能挑到最优源，大额时反而明显吃亏。
- 6\ 链下年化排行榜几乎和可执行性排序相反。榜首年化高得离谱的那个，盘口薄到连最小规模都吃不下。高费率是流动性差的症状，不是机会的证明。

- 7\、一个链下机会要过五道关：年化 → 有没有现货腿能对冲 → 两条腿价格锚不锚 → 盘口吃不吃得下 → 历史上回本天数兑现过没有。每加一道关排名就大变一次，三十个候选层层筛完只剩一个，而它还有未查清的风险。
- 8\、资金费率一律用中位数，不能用均值。它的分布是尖峰厚尾的，极端值就是常态特征。实测有合约的均值和中位数符号相反。
- 9\、结算周期因币而异，波动大的周期短、波动小的周期长，绝不能假设统一值。这个数进回本天数的分母，用错直接差几倍。必须每次现查。
- 10\、负费率币的反向做法整条路已排除。
借币利息吃掉全部收益，而且相当一部分标的根本不在杠杆列表里、压根不可借。这是「研究过了不行」，不是「没研究」。
- 11\、期现基差套利跑不过市场资金成本，钱借出去吃利息都比做它强。判断标准不该是「是不是正数」，而是「跑不跑得过头资金成本」。
- 12\、资金成本基准要用市场上真实的借币利率，不要自己拍一个「无风险利率」。
交易所的稳定币借币利率是公开的，拿它当门槛，有据可依。

markdown

笔记 020: 核心架構與運作機制

作者：Yolandatsai | 时间：2026-08-08 00:38 UTC | 字数：689 | 评分：65
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

核心架構與運作機制

入口層 (LI.FI API / MCP Server)：\ 提供跨 75+ 條區塊鏈 (含 EVM、Solana、Bitcoin 等) 的聚合路由。透過 LI.FI MCP Server (<https://docs.li.fi/mcp-server/overview>) 或 LI.FI Agent Skills, AI 能夠零配置發現並調用跨鏈、換幣與合約調用工具。

大腦與執行層 (Hermes Agent)：\ 透過長期記憶與閉環學習機制 (Closed Learning Loop)，將與使用者的互動與複雜的鏈上操作提煉成專屬 Skill (如 lifi-swap-skill)，在終端機 (TUI) 或雲端節點中持續自主優化。

結合與實現路徑

工具集成：在 Hermes Agent 環境中載入 LI.FI 的 MCP 服務或標準 Agent Skills，使代理具備查詢鏈、代幣、最優報價 (/quote 或 /advanced/routes) 的能力。

意圖與路徑規劃：當使用者下達跨鏈或資產配置指令時，Hermes 調用 LI.FI 獲取不安全交易數據 (transactionRequest)，經使用者授權或歷史審核記錄自動簽署執行。

閉環記憶迴圈：成功執行跨鏈交易後，Hermes 會自動記錄該次 Gas 消耗、滑點設定與路由偏好，存入本地記憶 (~/.hermes/)，使下一次同類任務能以極高效率「one-shot」完成。

笔记 021: 今天建立统一净收益表 \research/opportunity-log.csv\，把 D3 的 1

作者：Wang Hua | 时间：2026-08-08 02:44 UTC | 字数：287 | 评分：49
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天建立统一净收益表 \research/opportunity-log.csv\，把 D3 的 18 条 LI.FI 路线从原始 JSON 直接重建导入，分列预计输出、最低输出、Gas、协议费、延迟和未知成本。关键数字：乐观口径 (扣 Gas+协议费) 最佳净变化为 0.0558 USD，最低输出口径最佳为 0.3952 USD，18 条路线全部为负，没有一条可称为利润；D3 显示的正差额 (+0.24 USD) 只是 USDC/DAI 价格源估值差。新增 4 项测试，全仓 11 项通过。产物包括生成脚本、CSV 和分层说明文档。明天用 3 个金额重复报价测试容量。

笔记 022: 2026-08-08

作者：Sansho2 | 时间：2026-08-08 04:00 UTC | 字数：842 | 评分：66
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

2026-08-08

1. 今天学/做了什么

Day4：把「毛价差 vs 净收益」做成可复跑成本计算器 scripts/cost_model.py (跨链默认计入 LI.FI/服务费地板 25bps；滑点随名义超线性放大；含 Gas/失败期望/延迟项)。跑多档名义塌陷表，对照 lab 里两类线索：跨链聚合价差、同链多路由路径差。继续强调双腿同时成立，单边=方向风险。

2. 得到什么结果（数字）

- 假设毛价差 40bps + 跨链：名义 \$100-\$10k 净利全负（成本总在 ~43-236bps，25bps 地板占大头）→ 噪声/否决。
- 假设毛价差 75bps + 跨链：\$500-\$1k 纸面净利可转正，但更大名义被冲击击穿；且输入若仍是 DefiLlama 聚合价，必须先过 LI.FI 可成交验证，不能直接当候选。
- 同链路径差 38bps（无桥）：中等名义纸面可正，但语义是「执行路由质量」，不是无库存双边已锁 38bps；双边仍要第二腿+冲击。
- 产物：data/quotes/day4_.json；Lab 侧 Funding 已具备分项成本/连续过线只读信号（阈值细节仅本地）。

3. 失败或不确定点

冲击用简化平方缩放，非真实池深度；同链表易被误读成双边套利；跨链 75bps 线索未在本次打卡窗口重跑 LI.FI Quote；Day3 样本归档仍需补密度。

4. 对净收益模型的更新

成本按名义扫描才能看到「中间最优、两头塌陷」；跨链硬门槛 25bps 写进计算器默认项；路径差与可锁双边毛利必须分列记账。

5. 明天要验证的最小问题

Day5：把失败/延迟（Quote TTL、非原子单腿、回滚仍耗 Gas）写成显式成本项与否决条件，并补一组 LI.FI 实测对照纸面表。

笔记 023: https://github.com/ZeroNullNone/onchain_arbitrage_

作者：Nothing | 时间：2026-08-08 07:45 UTC | 字数：1463 | 评分：76

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

https://github.com/ZeroNullNone/onchain_arbitrage_study/blob/main/docs/daily/day_04.md

Day 4 — LI.FI API Probe

状态与结论

状态：完成。

2026-08-08 UTC 完成 3 routes × 3 sizes，共 9/9 个成功 Quote。

当前 Slice 只证明 Quote 可被完整保存与离线重建，不证明存在套利机会。

LI.FI fixed fee 已包含在输出；gas 是否包含没有 Source 字段，因此保留为未知。

完成项

新增同步、单次、只读 GET /v1/quote Probe。

Probe 记录完整 request、response/error、request ID、UTC 与 latency。

Raw 文件使用 exclusive create，已有证据不会被覆盖。

映射 chain/token/from/to/min/gas/fee/duration/tools/approval/transaction。

Token amount 使用 integer raw units + explicit decimals；USD 使用 Decimal。

Route fingerprint 只哈希稳定语义，不含 amount、quote ID 或交易动态字段。

保存 Base swap、Arbitrum swap、Base→Arbitrum bridge 的 100/500/1,000 USDC fixtures。

新增完全脱网的 fixture tests 与字段语义文档。

关键证据

9 个 fixture 都可重建原始 query、route steps、output、minimum output、fee 与 gas。

Base→Arbitrum 三个 size 均选择 feeCollection > eco，fingerprint 一致。

Arbitrum 三个 size 均选择 feeCollection > kyberswap，fingerprint 一致。

Base route 在 size 间出现 fly / kyberswap 切换；fingerprint 按实际路径变化。

Cross-chain fixture 的 LI.FI fee 为输入 USDC 的 0.25%，且 included=true。

所有 gas cost 均保留原生 ETH raw units，included=None，未猜测或置零。

返回的 transaction request 仅作为 raw evidence 保存，从未签名或广播。

验证

uv run pytest

Offline tests 覆盖 9 fixtures、3×3 universe、fingerprint、cost units 与失败响应。

下一步

Day 5 在此 raw envelope 与 parser 上增加低并发 append-only collector。

Timeout、retry/backoff、rate limit 与 durable normalized storage 留给 Day 5。

笔记 024: 已跑完24小时数据，先说结论：

作者：cfanboy | 时间：2026-08-08 08:01 UTC | 字数：871 | 评分：69

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

已跑完24小时数据，先说结论：

全线为负！跑满一个完整昼夜、6个资产、3条链、44,028条报价，净收益 > +5 bps 的可信事件：零。而 gas 还没算(同链双腿约 0.6 bps @\$1,000)。

摊销口径(搬运成本按该链对长期中位计,因为库存再平衡是事后批量做的):

[图片]

唯一有 $\geq +5\text{bps}$ 事件的是 DAI, 44 次。查了这 44 次: 其中 44 次靠“桥费为负”、只有 2 次毛价差为正。头部事件的毛价差全是负的(7.4 到 21.2), 正值完全来自 LI.FI 的桥报价与它自己的兑换报价之间约 20~28 bps 的分歧。

这不是市场错位,是同一家服务两个报价通道的内部不一致。

与前几轮抓到的

fly、1inch

同链倒挂、桥永动机是同一类问题——只是这次它躲在最后一层。

这轮数据里三道闸的工作量:

缺腿作废 3,653 次(只剩一条腿的链无法做配对自检)

同链倒挂作废 1,213 次(买入立刻卖出为正,物理上不可能)

桥报价矛盾剔除 1,026/7,806 条(来回搬运净赚)

如果没有这三道闸,这份报告会显示几百个“机会”。而且它们高度集中在 Base: 缺腿作废里 DAI@base 712、WBTC@base 657、weETH@base 596……因为 Base 的报价被 fly 主导,隔离后只剩单腿。

weETH 尤其失败:只配两条链,Base 一废就没得比,1,058 轮里只有 44 轮有效样本。这个标的选择是我的失误——两条链没有冗余。

在 L2 主流资产 + \$1,000 规模上,这条路线不成立。而且不是差一点——中位数在 1 到 23 bps,差的是数量级。这本身是个干净的结论:市场有效到我们测不出可捕获的偏离。

笔记 025: 本次从 LI.FI 拉取了三天的跨链报价数据(每次间隔30 分钟), 定时向 LI.FI (ht

作者: starkun | 时间: 2026-08-08 08:03 UTC | 字数: 1548 | 评分: 80

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

本次从 LI.FI 拉取了三天的跨链报价数据(每次间隔30 分钟), 定时向 LI.FI (<http://li.fi/>) 报价接口要报价, 然后把每次报价算成一条“成本观测记录”。

为什么要做这件事呢? 因为单次报价没有太大意义. 比如现在问一次 LI.FI (<http://li.fi/>), 从 Arbitrum 把 1000 或 10000 USDC 换/跨到 Base, 发现成本是 22 bps, 这可能代表很多情况:

可能是真实长期水平

可能是短暂好机会

可能只是报价噪声

可能是路由临时切换

可能是上游数据异常

只看一次,分不出来。跑完三天数据后,每条路径大约有:

24 小时 / 0.5 小时 = 48 次

48 次 3 天 = 144 次观测

有了这些样本,才能回答:

这条路径平时成本是多少?

最低能到多少?

最高会飘到多少?

路由会不会经常变?

1000 USDC 和 10000 USDC 成本差别大不大?

它是不是偶尔才好,还是经常都好?

本次成本观测结果

路径	规模	观测数	真实成本bps	成本极差	路由特点	上游异常
:-----	:-----	:-----	:-----	:-----	:-----	:-----
USDC→USDC	1000 USDC	142	25.23	[25.23~25.34]	0.11	eco 为主,少量 polymerStandard
USDC→USDC	10000 USDC	142	25.02	[25.02~25.03]	0.01	eco 为主,少量 polymerStandard
USDC→WETH 往返	1000 USDC	142	60.64	[49.24~73.31]	24.07	多路由频繁切换
USDC→WETH 往返	10000 USDC	142	60.65	[43.92~71.71]	27.79	多路由频繁切换

其中,“真实成本bps”那一栏的数据 25.23[25.23 ~ 25.34], 表示: 中位成本 = 25.02 bps,最低成本 = 25.02 bps 最高成本 = 25.03 bps. 成本极差是:最高成本 - 最低成本.

从成本数据中可以看到:稳定币路径成本几乎不动. 这说明它主要是固定费率结构,不是市场撮合导致的波动.

跨资产往返路径成本中位数约 60 bps,而且区间宽. 这说明它受实时路由、DEX 成交、桥选择影响更大.

本次拉取数据和处理数据,用 142 次成本观测 + 72 小时历史价差,证明 ARB→BAS 的 USDC 稳定币跨链和 USDC→WETH 跨资产往返路径,在本窗口内都没有可复现正净收益.

笔记 026: Day 4 : 两链预置库存与事后调仓

作者 : Flxxx | 时间 : 2026-08-08 08:27 UTC | 字数 : 1028 | 评分 : 66
来源 : <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 4 : 两链预置库存与事后调仓

今天完成了两链预置库存模式的资金流和库存账建模,保持只读与 Paper Trading,未连接钱包、未 Approve、未签名、未广播.

初始库存: Arbitrum 为 10,000 USDC、0 WETH; Base 为 0 USDC、5 WETH. 模拟成交: Arbitrum 买入 2,000 USDC -> 1.05 WETH; Base 卖出 1 WETH -> 1,930 USDC; 两端 Gas 合计 2 USDC. 成交后库存: Arbitrum 为 8,000 USDC、1.05 WETH; Base 为 1,930 USDC、4 WETH.

买入单价为 2,000 / 1.05 = 1,904.7619 USDC/WETH. 匹配 1 WETH 的毛价差为 25.2381 USDC,扣除 Gas 后约 23.2381 USDC. Arbitrum 额外留下的 0.05 WETH 是未完全匹配的库存敞口,不能忽略.

调仓比较: 立即跨链桥可以减少对两端预置库存的依赖,但有桥费、等待、失败和价格变化风险; 等待反向套利可能节省专门桥费,但机会不确定,库存风险会持续; 集中批量调仓可以摊薄固定 Gas 或桥费,但会延长不平衡时间并增加批量价格冲击. 跨链桥通常是事后调仓,而不是套利成交的原子路径.

失败分支: Arbitrum 买入成功、Base 卖出失败时,系统成为 WETH 净多头, WETH 下跌会受损; Base 卖出成功、Arbitrum 买入失败时,相对预期配对仓位成为 WETH 净空头,之后若需要买回 WETH,价格下跌会降低恢复成本.

今日报警规则: 只有当毛价差扣除 DEX fee、两端 Gas、滑点缓冲、预期失败成本和事后调仓成本后仍超过最低收益门槛,并且两端库存未超过上限、调仓后能回到目标区间时才报警,否则标记为观察或拒绝. 滑点缓冲优先用 amountOutMin / toAmountMin 与理想 Quote 的差额; 没有保护字段时,用同链、同路由、同金额档位的历史 Quote 偏差 P95. 失败成本按“失败概率 × 单腿失败后的处理成本”估计.

未完成: 尚未接入真实桥接报价,调仓成本和等待时间仍是参数假设; 失败率和滑点缓冲还需要继续用 Paper Trading 累积样本. 下一步进入 Day 5,学习区块状态、报价过期、Gas 竞价、失败交易和 MEV 执行风险.

笔记 027: 今日完成

作者 : Louis lue | 时间 : 2026-08-08 09:51 UTC | 字数 : 1364 | 评分 : 82
来源 : <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今日完成

初始化 arbitrage-monitor TypeScript 项目,建立 collector、API、dashboard、adapters、strategy、database、shared、docs 与 tests 等目录.

配置 Node.js、TypeScript、tsx 和 Vitest,启用严格类型检查.

按测试优先流程完成只读 collector 最小入口: 先验证缺少实现时测试失败,再补充实现并使测试通过.

编写 README、系统架构、v1 验收标准和套利研究问题清单,明确 v1

只做监控与模拟,不连接主钱包、不保存私钥、不自动交易.

新增只读 Web 状态 Dashboard 与 /api/health 健康检查接口. Dashboard 展示服务、只读模式、collector 骨架和执行关闭状态. 将服务绑定到容器内 0.0.0.0:3000; Docker 端口重新映射后重新启动应用,并在容器内部验证健康检查正常.

关键结论

Docker 中仅让程序监听端口还不够：应用必须监听 0.0.0.0，并由宿主机通过 3000:3000 发布端口；公网开放还应增加 HTTPS 和认证。

“容器内运行”“宿主机已发布端口”和“用户设备可访问”是不同验证层级。目前已确认容器内服务运行及健康检查成功；外部客户端可达性仍需从实际设备访问确认。

监控和交易执行必须保持隔离，当前接口明确返回 mode=read-only、executionEnabled=false。

证据

项目路径：/opt/data/arbitrage-monitor

初始化提交：acad77f chore: initialize arbitrage monitor project

Dashboard 提交：0613849 feat: add read-only status dashboard

自动化测试：2 个测试文件、3 个测试全部通过。

类型检查：npm run typecheck 通过。

运行验证：GET http://127.0.0.1:3000/api/health 返回 HTTP 200，服务状态为 ok，模式为 read-only，自动执行为 false。

项目文档：README.md、docs/architecture.md、docs/acceptance-criteria.md、docs/research-questions.md。

未完成 / 下一步

尚未完成学习者侧的套利知识检索和个人笔记；下一步优先比较同链跨 DEX、三角套利和跨链套利，并写出自己的净利润公式。

尚未接入 LI.FI、DEX Quoter、RPC、数据库和 Telegram 告警，因此当前 Dashboard 没有真实报价或套利机会。

当前 Web 服务仍是容器内临时后台进程，容器重启后不会自动恢复；下一步应配置独立 Compose 或 s6 服务。

宿主机端口已由用户重新绑定，但尚未从用户设备实际验证最终访问地址。

笔记 028: 链上套利残酷共学打卡 - 2026-08-08

作者：KuTear | 时间：2026-08-08 10:10 UTC | 字数：1299 | 评分：72

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

链上套利残酷共学打卡 - 2026-08-08

今日主题：跨所零暴露对冲实战巡检、全网知识图谱升级与高频风控沉淀

一、跨所对冲持仓与收益风控巡检

1. 主流标的跨所对冲策略运行状况：

- 运行模式：严格无裸露 Delta (Delta-Neutral) 对冲，双端多空持仓数量与名义价值完全对齐。

- 对冲表现：一端做空锁定正向基差与资金费率，另一端做多完全对冲单边波动风险。

- 净收益状态：双端合并呈现稳健正浮盈锁定状态，当前净收益稳步累积中。

- 风控与执行机制：Delta 敞口严格归零。自动化交易系统持续执行步进加仓与收敛止盈逻辑 (stepOpen / stepClose)，待基差回归收敛后自动分批平仓兑现利润。

2. 多节点与资金池全景风控：

- 多端分布：接入的主流 CEX 与链上衍生品 DEX 节点运行正常。

- 资金健康度：各节点保证金充足，维持健康的维持保证金比例 (Maintenance Margin Rate)，严格控制资金杠杆，杜绝爆仓清算风险。

二、全网套利机会扫描与风控策略

1. 合约乘数 (Multiplier / Contract Size) 自动归一化：

- 在扫描全网 DEX 与 CEX 时，重点防御因合约计价单位不同 (如部分 DEX/CEX 上 1 张合约代表 100/1000 币) 产生的虚假表面价差。

- 底层交易引擎必须强制接入交易所元数据 precision / contract_size 进行自动换算，避免假信号触发异常交易。

2. 流动性与充提状态防护：

- 对订单簿深度过浅或处于充提维护状态的标的，策略引擎保持 disableOpen: true 防御模式，严防陷入流动性陷阱。

- 对重点监控标的持续处于自动化监听序列，严格满足保本价差门槛后再行建仓。

三、理论与系统演进思考

1. 跨链套利保本价差 (Break-even Spread) 原则：

- 充分吸收知识库中关于跨链全链路摩擦成本的计算模型：

$$Net\ Profit = Gross\ Edge - Trading\ Fees - Gas - Bridge\ Fees - Slippage - Rebalance\ Costs$$

- 单笔交易必须严格设定资金下限 (如 $\geq 1,000\ USD$)，避免固定桥费 (如 LI.FI 25bps + Bridge 固定费) 吞噬收益。

2. Searcher 架构解耦与原子化保护：

- 在链上执行层面，借鉴“链下搜索器 + 提交 EOA + 链上执行合约”的三层解耦架构，通过私有 RPC / Bundle 发送交易，并设置硬性利润断言，未达收益自动回滚，确保本金安全。

笔记 029: 笔记

作者：mookixiao | 时间：2026-08-08 10:11 UTC | 字数：2281 | 评分：62

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

笔记

1. 基本套利类型

1. DEX间套利：ETH在Uniswap上3000USDC、在Sushiswap上3010USDC，可以在Uniswap上买入、在Sushiswap上卖出
2. 跨链套利：同一个代币，在Ethereum上100USDC，在Solana上105USDC，可以将资产从Ethereum搬到Solana上卖出
3. 三角套利：比如USDT --> ETH --> BTC --> USDT，转一圈之后USDT变多
4. 清算（liquidation）：在链上借贷协议中，借款人的抵押品贬值接近爆仓线时，套利者帮系统偿还债务，并以低于市场价的折扣价拿走抵押品

1. 这里的系统指的是什么？

5. MEV：套利者通过给更多gas费的方式调整链上交易的打包顺序，抢先低价买入再高价倒手卖出，赚取差价，即抢跑/夹心攻击问题

1. Uniswap和Hyperliquid有什么区别？Uniswap和链的关系是什么？

1. Uniswap和Hyperliquid

1. Uniswap：经典现货AMM DEX，专注于代币间的点到点兑换，没有订单簿，代币价格由资金池比例决定
2. Hyperliquid：专门做链上永续合约/衍生品交易的订单簿DEX，采用传统交易所常用的挂单/吃单模式，支持高杠杆
2. Uniswap和链的关系：Uniswap本质上是一套智能合约程序，早期部署在以太坊主网上，后续也在Arbitrum、Base、Polygon等数十条主流公链上都部署了副本，并依靠这些公链提供计算资源并记录交易日志
2. AMM听起来好简单，感觉是不是监控大额买卖单就能一定程度上预测价格变化，就能套利了？

1. 理论上成立，但是实际很难操作，因为你看到大额买卖单，还有成百上千个专业交易机器人也看到了，它们会给节点支付更高gas费以抢到你前面成交（抢跑、夹心攻击）

3. Uniswap和Sushiswap是同类型的DEX？排名前几的DEX都有哪些？

1. 是的，Sushiswap最初复刻的就是Uniswap v2的开源代码

2. 现货（spot dex）：Uniswap（全网龙头）、PancakeSwap（BNB链龙头）、Raydium/Oraca（Solana生态龙头）、Aerodrome（Base链龙头）

1. 合约/衍生品（perp dex）：Hyperliquid、Aster、dYdX、Jupiter Perp

4. 这些Ethereum链、Solana链有什么区别？L1和L2有什么区别？作为套利者这些许许多多的链都要关注吗？还是说重点关注其中几个就可以？

1. 不同的公链特点不同

1. Ethereum：追求极度的安全性，但是速度较慢、费用高昂
2. Solana：追求高吞吐量和低延迟，速度快、费用低

2. L1：比如

1. L2：

3. 不需要都关注，前期重点关注2~3个核心生态即可，比如以太坊上的优质L2（Arbitrum、Base等）、Solana生态

5. gas费是不是就是CEX的手续费？gas费是DeFi范围的概念还是DEX范围的概念？它按什么计算，单次还是按交易金额比例？DeFi世界的交易除了gas费还有别的费用吗？

1. gas费是支付给区块链节点的服务费，补偿其计算和存储资源；cex手续费是给cex交易平台的佣金
2. 范围：gas费是区块链底层通用概念，无论是线上转账、部署合约还是在dex交易，只要修改了链上状态，就必须支付gas费
3. 按单次计算，也就是说转账100刀和10000刀消耗的gas费完全相同
4. 其他费用：dex手续费（dex平台收取的比例费用）、跨链桥手续费（使用跨链协议时支付给中继节点的服务费）

6. 跨链桥是什么东西？所有区块链两两之间都有跨链桥吗？

1. 连接两条独立区块链的管道，比如以太坊和Solana底层数据无法直接互通，跨链桥通过在A链锁定资产、在B链生成/释放资产的方式实现资产跨链

2. 不是两两区块链之间都有跨链桥，比如如果小众链A和小众链B之间没有直连桥，但是要在两者之间完成资产转移，需要主流链作为中继：小众链A --> Ethereum/Arbitrum等 --> 小众链B

7. LI.FI到底是什么东西？对于套利者它有什么使用价值？

1. 本质：一个跨链流转和兑换聚合器，接入了数十个跨链桥和上百个DEX
2. 对套利者的价值

1.

最佳路径智能路由：当需要把A链的某种代币换成B链的另一种代币时，LI.FI可以在毫秒内算出一套损耗最小、速度最快的跨链方案

2. 数据监控：通过调用LI.FI的API，可以获知不同链、不同资产在不同时间段的预估报价、gas损耗、跨链路径变化等信息，以辅助策略脚本做出判断

8. 目前，DeFi的世界里是不是大部分是金融投资和投机活动，实际生活经济行为很少？

1. 是的，目前DeFi生态中90%的资金和活动集中在金融投资、杠杆交易和做市套利等资本运作上

2. 实际经济行为举例：现在跨境贸易中大量企业或个人通过USDT/USDC进行跨国结算，绕过SWIFT系统，省去高额手续费和数天等待时间

笔记 030: 【8月8日 学习打卡】链上套利 Day 4：监控系统升级与自动交易

作者：feiyee2165-blip | 时间：2026-08-08 10:13 UTC | 字数：605 | 评分：65

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

【8月8日 学习打卡】链上套利 Day 4：监控系统升级与自动交易

一、跨链桥监控系统 v2

- 数据源扩展为 4 个：LI.FI + Relay + Rango + Synapse，覆盖 25+ 跨链路径
- 逆向 Rango 前端 API（提取内置 key + 边缘端点 + 绕过 Cloudflare）
- 多源容错：一个源限流其他源顶上，报价缓存避免触发限流
- 阈值：差价 >10% 才报警（异常报价机会）

二、自动交易系统

- 监控到 >10% 差价 → 立即自动交易
- 盈利静默记录，亏损/失败才通知
- 每笔限额风控

三、执行钱包体系

- 主钱包 + 7 链资金分散（ARB/BASE/OP/BSC/AVAX/ETH/SOL）
- 低手续费优先跨链，干跑→试水→放量验证链路

四、群聊精华：MEV 盲套利

- 链上实时读状态算利润，无利润立即 revert（探测 gas 仅 7.4 万）
- V2 内存可算，V3/V4 需链上查 tick
- "MEV 拼的是路径发现和 gas 优化，不是速度"
- 资金费率套利最简单（头部日 7-8 万笔）

五、其他

- QQ 双向对话打通（与 Telegram 平级）
- OKX MCP 调研：公开 key 只能查行情
- Rujira/Liquidy 收入分析：做市 DAO 与基建层关系

明日继续：Gate cross 跨所套利研究

笔记 031: LI.FI 对套利者（尤其是跨链套利）非常有用，它是跨链资金调度和执行的核心基础设施工具。对套利者的

作者：liiiiimh | 时间：2026-08-08 11:18 UTC | 字数：1705 | 评分：90

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

LI.FI 对套利者（尤其是跨链套利）非常有用，它是跨链资金调度和执行的核心基础设施工具。对套利者的核心价值单纯发现两个市场/链之间的价差只是第一步。真正能不能赚钱，取决于：

资金从哪来、走哪条路由、多久到账

实际到手多少（扣除所有费用后）

Gas、滑点、桥接/DEX 费用、价格冲击

LI.FI 解决的就是这些“执行层面”的问题：

聚合大量流动性源：支持 50–60+ 条链（EVM、Solana、Bitcoin、Sui 等），聚合 20+ 桥（Stargate、Across、Mayan 等）和 30+ DEX/聚合器（Uniswap、1inch、Odos 等）+ Solver 网络。

给出真实可执行报价：一次 API 调用就能拿到完整路由，包括：

预计到手金额（toAmount）和最低到手金额（toAmountMin）

LI.FI 服务费（默认 0.25%）

桥/DEX 费用、Gas 估算、滑点、预计耗时

完整的 transactionRequest（可直接签名广播）

多步骤路由：支持 swap → bridge → swap、甚至更复杂的组合，适合三角套利或跨链价差。

适合机器人/AI Agent：API + SDK 完全可编程，很多套利机器人和 AI Agent 都在用它做“发现价差 → 计算真实利润 → 自动执行”。

状态跟踪：跨链交易可实时查询进度，失败有 fallback 机制。

简单说：没有 LI.FI 这类工具，跨链套利的执行成本和失败率会高很多；有了它，可以把“纸面价差”快速转化成“扣除所有磨损后的真实利润”。怎么使用（套利实操流程）1. 基础准备

注册 LI.FI Partner Portal (<https://portal.li.fi>) 获取 API Key（免费，提升请求频率）。

安装 SDK（推荐）：npm i @lifi/sdk

或直接用 REST API：<https://li.quest/v1/>

2\ 典型套利流程（伪代码逻辑）

1. 监控价差：用 CEX API、链上预言机、或自己扫池子，发现例如 ETH 上 USDC 和 Arbitrum 上 USDC 有价差。

2. 请求真实 Quote（关键）：

bash

GET <https://li.quest/v1/quote>

?fromChain=1 # Ethereum

&toChain=42161 # Arbitrum

&fromToken=0xA0b8... # USDC

&toToken=0xaf88... # USDC

&fromAmount=1000000000 # 1000 USDC（注意小数位）

&fromAddress=你的地址

&slippage=0.005 # 0.5%

3. 计算真实利润：

看 estimate.toAmount / toAmountMin

减去 LI.FI 0.25% 服务费 + Gas + 其他费用

对比目标市场卖出价格，确认净利 > 0（还要考虑资金占用时间、失败风险）

4. 执行：

拿到 transactionRequest

用自己的钱包签名并广播

用 /status 接口轮询直到完成

5. 循环：机器人持续监控 → 计算 → 执行

3\ 进阶用法

使用 /advanced/routes 获取多条路由，自己挑最优的（最快/最便宜/最可靠）。

过滤特定桥/DEX (bridges=across,stargate)。

多步骤 Composer：把“跨链 + 目标链 swap + 存入 yield”打包成一笔交易。

AI Agent 集成：官方已有 MCP Server、CLI、Agent 文档，适合做自动化套利 Agent。

笔记 032: 链上套利共学 Day 4 | LI.FI Agent 集成学习：5 端点最小集 + /tools 桥链对

作者：Cz07cring | 时间：2026-08-08 11:51 UTC | 字数：1863 | 评分：73

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

链上套利共学 Day 4 | LI.FI Agent 集成学习：5 端点最小集 + /tools 桥链对结构实测

今天学什么

针对一个点深度学习：LI.FI 官方给 AI agent 的集成指南 (docs.li.fi/agents/overview)。

核心是「5 端点最小集」——官方说这 5 个 REST 端点覆盖 agent 大多数场景。

我不仅读了文档，还把其中两个端点接入到我们的 Go 套利框架 (/tools、/status)，并实测发现了一个文档没写的结构细节。

今天完成了什么

一，通读 LI.FI Agents 文档 (overview + concepts)

- 四种 agent 接入方式：REST API（通用，推荐）/ MCP Server（Claude/Cursor 零配置）/ CLI（省 token）/ SDK（前端钱包）

- 我们的 Go 项目走 REST API，与官方推荐一致

二，5 端点最小集（官方定义）

1. /quote — 返回可立即执行的交易（含 transactionRequest 待签名）

2. /status — 跨链进度轮询（NOT_FOUND/PENDING/DONE/FAILED）

3. /chains — 支持的链列表（69 链）

4. /tokens — 各链代币与价格

5. /tools — 可用的桥（27 个）和 DEX（31 个）

三，接入到 Go 框架（TDD 全绿）

- /tools 端点：internal/lifi/tools.go + tools-scan 命令（按链/类型列出桥）
- /status 端点：internal/lifi/status.go（TransferStatus + IsDone/IsPartial 语义）
- transactionRequest 解析：Quote 结构扩展（To/Data/Value/GasLimit，执行层签名直接消费）

踩过的坑（重点）

文档没写的结构细节：/tools 返回的桥和 DEX 结构不一样！

DEX 的 supportedChains 是「单链数组」：[1, 56, 8453]

桥的 supportedChains 是「链对数组」：[{"fromChainId":1,"toChainId":8453}, ...]

第一版代码按文档想当然写成单链数组，实测直接 JSON 解析失败：

cannot unmarshal object into Go struct field Tool.bridges.supportedChains of type int

修复：自定义 UnmarshalJSON 容忍两种格式（先试链对，再试单链），并加了 SupportsPair(from,to) 判断桥的方向路由。这呼应了之前 tokens.go 的教训——LI.FI API 字段结构要用真实响应验证，不能只信文档。

学习收获

一，agent 实盘的最小闭环（文档给了全景）

发现机会 → /quote 拿可执行交易 → 签名广播 → /status 轮询 → 落库 outcome。

我们项目现在处于第一步（监控发现），后面三步是实盘阶段（需授权）。

二，substatus 语义对套利很重要

DONE 的 substatus 有三种：COMPLETED（成功）/ PARTIAL（收到不同代币，仍算成功）/ REFUNDED（退回）。

PARTIAL 对套利是风险——收到非预期代币，净收益核算要重算。这印证了我们 execution 包 12 状态迁移表的设计方向。

三，限流分级（实测印证文档）

文档：无 key 200 req/2h，带 key 200 req/min。

实测：我们的 key 对单链请求 200，多链批量 401（key 权限分级）。所以 spread-scan 用无 key 客户端 (/tokens 不限流)，监控完全健康。

下一步

/quote 限流恢复后：实测 USDC Mantle→Base 等链对真实桥费 + 容量（toAmountMin 5 万刀门槛），完成桥费实测表实盘阶段（需授权）：接入 /status 轮询闭环 + transactionRequest 签名执行（小狐狸 Agent Wallet）

学习记录，非投资建议。

笔记 033: 模块 3 第 2 天：Hermes 信号 workflow 骨架

作者：Milli | 时间：2026-08-08 12:06 UTC | 字数：786 | 评分：78

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

模块 3 第 2 天：Hermes 信号 workflow 骨架

共学第 4 天 | 任务：设计「价差 → 可执行信号」的流水线骨架，确定数据源

覆盖：五环节流水线 / 数据源对比 / LI.FI 报价参数 / 净收益判断闸门 / 告警设计

一、流水线架构（五环节）

① 价格源采集 → ② 价差计算 → ③ LI.FI 报价 → ④ 净收益判断 → ⑤ 告警/信号

CoinGecko (askA - bidB) /quote toAmount - 成本 净收益 > 阈值

链上 RPC - 桥费 - gas /routes - MEV 折扣 → 推送提醒

每环节的输入/输出：

| 环节 | 输入 | 输出 |

|-----|-----|-----|

| ① 采集 | 链/合约地址列表 | 各市场实时价格 |

| ② 价差 | 两个市场价格 | spread (毛价差) |

| ③ LI.FI 报价 | 起链/目的链/token/金额 | toAmount + 费用明细 + 时长 |

| ④ 净收益 | 报价结果 | 通过/拒绝 二元信号 |

| ⑤ 告警 | 通过的信号 | 群聊推送 |

核心认知：② 的毛价差不能直接决策，必须经过 ③ 把成本算进去，才能得到 ④ 的真净收益。

二、数据源选择（今天的关键决策）

| 数据源 | 拿到的数据 | 延迟 | 用途 |

|-----|-----|-----|-----|

笔记 034: Day 4 | 从交易复盘走向支付路由

作者: Zacteh | 时间: 2026-08-08 12:10 UTC | 字数: 1537 | 评分: 90

来源: <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 4 | 从交易复盘走向支付路由

日期: 2026-08-08

今日命题

把已有的“链上交易发生了什么”推进为“发起交易前, Agent 应该如何解释一条路由”。

发起人的近期推文反复出现两条线索:

1. AI 等待与多任务切换会稀释注意力, 因此今天只保留一个可验收目标;
 2. Agent 的实际价值来自端到端完成任务, 例如从钱包 ETH 经 LI.FI 变成 USDC, 再衔接法币支付。
- 今天只实现其中可安全验证的链上决策层: 读取实时 LI.FI 报价, 不签名、不广播、不接触法币出金。

90 分钟安排

| 时间 | 内容 | 验收标准 |

| --- | --- | --- |

| 0-10 分钟 | 复跑现有交易检查器 | 能解释交易费、日志数量和资产流 |

| 10-25 分钟 | 阅读 LI.FI Quote/Token 接口 | 写清输入、输出和安全边界 |

| 25-70 分钟 | 实现 route-inspector | 能实时展示 ETH → USDC 路由 |

| 70-85 分钟 | 类型检查与真实请求 | pnpm typecheck 和 pnpm route 通过 |

| 85-90 分钟 | 复盘 | 留下结论与下一步 |

上次进度复盘

样例交易 0x9ac0...0328 并不是 LI.FI 套利交易: 它调用 0x26ededb8, 成功产生 1,000 条同一代币的 Transfer 日志, 把名为 Notcoin、符号为 NOT 的代币批量分发到大量地址。实际手续费约为 0.01165928076594797 ETH。

关键教训: 代币名称和符号不构成真实性证明; 先看合约地址、调用方法和完整资产流, 再判断交易语义。

今日实现

新增 pnpm route:

先通过 /v1/token 获取来源代币 decimals;

把人类可读数量转成最小单位;

通过 /v1/quote 获取实时路由;

展示预计到账、最低保证到账、gas/协议费用、预计时长和每一步工具;

只展示未签名交易预览, 代码没有钱包、私钥、签名或广播逻辑。

理解检查

1. toAmount 是当前预计到账, toAmountMin 是计入滑点后的最低保证值, 两者不能混用。
2. 跨链路线可能先在源链换币, 再通过桥转移; “ETH → USDC”不等于一步交易。
3. 报价是瞬时快照, 不代表稍后仍可成交, 也不代表用户余额或授权充足。
4. LI.FI 解决的是链上流动性与执行编排; USDC → Fiat → 商家 仍需合规出金和支付服务商。

下一步

下一次共学应加入“相同输入下 CHEAPEST 与 FASTEST 的结构化对比”, 并把报价快照保存为可回放的研究数据; 仍保持只读, 等安全模型和失败处理完整后再讨论签名执行。

资料

发起人 X: <https://x.com/brucexu_eth>

LI.FI Quote API: <<https://docs.li.fi/api-reference/get-a-quote-for-a-token-transfer>>

LI.FI Token API: <<https://docs.li.fi/api-reference/fetch-information-about-a-token>>

LI.FI Agent endpoint 说明: <<https://docs.li.fi/agents/reference/endpoint-specs>>

笔记 035: 链上套利残酷共学 · Day 4 打卡 (2026-08-08)

链上套利残酷共学 · Day 4 打卡 (2026-08-08)

在 GitHub [w3player/arbitrage-playbook](#) 学了 5 个套利打法 issue，重点是 #1「LP 收过路费」被 starkxun 用数据证伪。沉淀为初学者能懂的笔记。

一、这个 repo 是什么

hstarorg 建的「套利实战手册」issue 收集簿：把 X (推特) 上看到的套利打法转成 issue，自己加点评，邀请 starkxun 等人来证伪/补充。目前 5 个 open issue (8/5–8/6，与共学同期)：

- #1 LP 套利探讨 (从机器人路线上"收过路费") —— 最硬核
- #2 资金费套利探讨
- #3 Gate CrossEx 跨所工具
- #4 Backpack 套利展示
- #5 li.fi 跨链

二、重点 #1：LP「收过路费」为什么是幻觉 (starkxun 证伪)

原贴美好设想

卷不过套利机器人

就在机器人必经之路放个高费率池，收它的手续费 ("开收费站")。入场：爆量、缺池、剧烈波动、反复穿越。

机制先搞懂

- Uniswap V3 集中流动性：钱只铺在指定价格区间，区间内深度厚、手续费多；价格跑出区间就一分不赚。
- 单边存 USDT (区间在现价下方) = 等价于"在下方挂一串限价买单 + 额外收手续费"。这一点原贴是对的。

但你是"外汇窗口"，不是收费站

机器人只在「跟你交易对它有利」时才来——它外面买便宜币，卖给你换走 USDT，赚的就是你的报价 vs 真实市场价的差。

它赚的钱 = 你的报价 - 真实市价 ← 从你口袋出

你赚的钱 = 手续费 ← 从它口袋出

净值可正可负。你的隐形亏损叫 LVR (Loss Versus Rebalancing, 相对再平衡损失) =

逆向选择：排队换币的全是比你快、比你懂的人。

真实回测 (两个案例，触目惊心)

- 案例 A (小涨 +2.6%，最好情况)：手续费 +4.29 U，gas 5 U → 净 0.71 U (0.7 bps)，白忙。
- 案例 B (下跌 20%，"生意最好")：88/135h 在区间内 (65%)，按原贴算"开张成功"。
- 手续费 +180.60 U (看着香)
- 撤池拿回 0 USDT + 17,219 个已跌 20% 的币
- 存货盈亏 1,157.40 U
- 净 981.80 U (981.8 bps，亏本金 9.8%)

最反直觉：手续费收入和存货亏损是同一原因驱动的——成交越频繁=价格穿越越剧烈=你被填的单越多=存货亏越狠。"生意好"和"亏钱"是同一件事。

区间扫描暴露根本矛盾

[-5%,-1%] 在区42h 净 +1.4 bps ← 唯一微赚

[-10%,-2%] 19h 0.6 bps

[-20%,-2%] 19h... 0.7 bps

[-50%,-10%] 0h 5.0 bps ← 没开张，资金白占

矛盾：要有流量→区间必须贴近现价 (窄)；不被砸穿→区间必须远离现价 (宽)。两个要求直接冲突，无中间解。

窄区间有流量但最易被大阴线扫穿成全仓接盘；宽区间安全但没流量。

收获

验证"躺赚"说法 → 先写成能算的式子 (净收益=手续费-存货盈亏-gas) → 用历史数据跑，别听故事。这正是共学"假设验证到底" + starkxun 铁律的范本。

三、#2 资金费套利 (最推荐新手入门)

- 正费率：现货买 + 合约空，收多头付的资金费。
- 负费率：现货做空要借币，基本借不到 → 负费率套利基本不可达。
- 跨所资金费差：A 正 B 负 → A 空 B 多吃差额 (注意流动性)。
- 对应我之前写的资金费监控脚本 ("发现"环节)。机制简单适合练手，但正费率少见、小资金不够手续费。

四、#3/#4/#5 简要

- #3 Gate CrossEx：Gate 在各所垫资给大家玩跨所资金费套利，降低"两边都要库存"门槛，但额度是 Gate 给的、有上限有退出风险。

- #4 Backpack : "固定奖励活动看起来稳赚"——任何"稳赚+固定奖励"往往早期红利、后期卷没，别看到就冲。
- #5 li.fi : 只读查 Route 成本，对应我落地的 lifi_spread_probe.py (跨链 edge 全负、Base 源虚高告警)。

五、今日结论

全 repo 最有价值的是 #1 : 示范了「看到诱人打法 → 数据证伪 → 再决定做不做」的完整范式。和 Lizzie、cfanboy 方法论 100% 一致。初学者最该练的不是"找躺赚"，是"快速证伪一个躺赚说法"。

六、明日最小实验

照 starkxun 的思路，写一个自己的"LP 收租回测"最小脚本 (The Graph 拉数据)，验证"挂单收租"在我关心的链上是否成立。

笔记 036: 2026-08-08 — Day 4: 真实案例净收益计算 (跨链 DEX 价差 × 成本模型)

作者 : bamboo | 时间 : 2026-08-08 12:29 UTC | 字数 : 3397 | 评分 : 89

来源 : <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

2026-08-08 — Day 4: 真实案例净收益计算 (跨链 DEX 价差 × 成本模型)

主题: 找一个链上价差，按 Day 3 成本模型算出可执行净收益

方法: LI.FI Quote API (真实聚合报价) + Base 链 RPC 链上验证

结论先行: 屏幕价差 13.6 bps << 往返成本 ~50 bps → 这笔没肉，成本是价差的 3.7 倍

一、今日任务回顾 (PLAN-week1)

找一个真实链上价差

用成本模型 (Gas/滑点/协议费/桥费/失败交易/资金占用/机会成本) 算可执行净收益

用 Etherscan/链上数据查交易验证

产出: 本笔记 (证据记录)

二、案例设计 : 同一笔钱，两条链买 ETH，价差在哪？

操作 : 5000 USDC 分别在 Base 和 Arbitrum 上换成 WETH (同币种、同金额、同一时点)，对比买入均价。

数据源 : LI.FI Quote API (li.quest/v1/quote，聚合 30+ 链真实 DEX 流动性报价)，Base 链公共 RPC 验证合约存在性。

原始数据 (2026-08-08 实测，LI.FI 实时报价)

| 链 | 投入 | 产出 | 买入均价 | 服务费 | 油费 |

|---|---|---|---|---|

| Base | 5000 USDC | 2.6024 WETH | 1,921.29 USDC/ETH | \$12.55 | \$0.006 |

| Arbitrum | 5000 USDC | 2.5989 WETH | 1,923.91 USDC/ETH | \$12.50 | \$0.047 |

价差 = $(1923.91 / 1921.29 - 1) \times 10000 = 13.6 \text{ bps}$ ，Base 便宜，ARB 贵。

链上验证 (不是纸上数据)

- eth_getCode 验证 LI.FI approval router (0x1231DEB6...) 在 Base 链真实存在 (5176 bytes 合约代码)

- eth_getCode 验证 USDC 合约 (0x833589fC...) 已部署

- 读取时区块: 49701404 (Base)

三、成本模型套用 (Day 3 六件套)

| 成本项 | 金额/比例 | 说明 |

|---|---|---|

| Gas | \$0.006-0.05 | Base/ARB L2 极低，可忽略 |

| 服务费 (协议费) | $\$12.5 \approx 25 \text{ bps}$ | LI.FI Fixed Fee，不随规模摊薄 |

| 滑点 | 已含在报价 | toAmountMin 保障 |

| 桥费 (跨链) | $\sim 25 \text{ bps}$ 单程 | 学员实测 (doctorzero666/wmroar) : 桥费线性不摊薄 |

| 失败交易 | 风险项 | 跨链路径 7-1080s 不等，超时=机会死亡 |

| 资金占用/机会成本 | 低 | 5000U 量级影响小 |

关键结论 (来自 Day 3 + 学员实测) :

- 跨链往返净收益 $\approx 2 \times$ 单程费率 (wmroar 实证) → break-even 价差 $\approx 50 \text{ bps}$

- 本案例价差 $13.6 \text{ bps} < 50 \text{ bps}$ → 净收益为负，不做

成本是价差的 3.7 倍 ($50/13.6$) ——再次验证课程主线"屏幕价差 ≠ 可执行净收益"。

四、动手过程中踩的坑 (可复用经验)

1. DEX Screener API 本机不通 (000/拦截) → 换 LI.FI Quote API (可直连，且聚合报价更权威)

2. Uniswap V3 factory 地址踩坑 : 我一开始用 Base 上猜的 factory 地址 (0x33128a8fC...) 读池子，返回价格荒谬 (1 WETH =

1.9e-9 USDC) ——用 pool.factory() 交叉验证发现 execution reverted, 判定地址错误。教训：链上合约地址必须以官方文档/已部署确认为准，读出的价格要过合理性检验（WETH 不可能 1e-9 USDC）。Base 链 Uniswap V3 官方部署地址待查官方文档（本次改用 LI.FI 不依赖该地址）

3. 网络抖动：mainnet.base.org / base-rpc.publicnode.com 双 RPC 交叉验证一致才采信；urllib 遇 SSL EOF 重试 3-4 次

4. LI.FI API 返回结构变了：新版顶层直接是 route 对象（includedSteps），不是老的 {"route": [...]}——解析脚本按新结构写

五、概念消化：Beta / Delta / Alpha（今日答疑，并入延伸知识）

一句话：Beta 和 Delta 都是“敏感度”——Beta 对大盘，Delta 对标的；对冲清零后剩下的纯价差收益就是 Alpha。

|| 对什么敏感 | 记忆锚 |

|---|---|---|

| Beta (β) | 大盘（标普/纳指） | 船随海浪晃多少 |

| Delta (Δ) | 标的自己（价格动1块→盈亏多少） | 方向敞口 |

| Alpha (α) | 都不敏感 | 纯手艺钱 |

顺口溜：Beta 跟着大盘晃，Delta 盯着标的涨，两样清光剩 Alpha，对冲赚的是手艺粮。

实战验证（XAU-CL 金油组合，多黄金/空原油）：

- 原油暴涨 10% → 组合基本不受影响（Delta 中性）
- 美股大盘暴跌 5% → 组合基本不受影响（商品对大盘 Beta≈0）
- 套利者目标：beta+delta 清零 → 只赚 alpha（价差回归）

今日案例关联：跨链价差套利同样是 Delta 中性结构（两腿都是“买 ETH”的镜像动作），但成本模型判死刑——因为 Alpha 太小（13.6bps）盖不住摩擦。

六、卡点 & 待办

[] Dune 上手未完成：本机 Dune 需要 API key 注册，本次未跑（The Graph QoS 宕机也影响 Dune 数据滞后，skill 已记录该风险）；待有 key 后补 Dune SQL 基础查询（查某池子成交量/资金流）

[] Uniswap V3 官方 factory 地址：查官方文档确认 Base 链部署地址，补全“同 DEX 内多费率池价差”案例（本次被地址坑挡路，先记录不阻塞）

[] 脚本收编：scripts/uni_v3_pool_price.py（本次写的池价读取脚本，地址修正后可用）+ 跨链比价逻辑考虑收进 scripts/lifi_quote_check.py 扩展

七、明天计划（8/9 周日，机动日）

补漏：查 Uniswap V3 官方部署地址，补做“同 DEX 内多费率池价差”案例（用正确地址重跑）

可选：DEX Screener 四步扫 BSC 新池练手（只观察不建池）

参考链接

- LI.FI Quote API: <https://li.quest/v1/quote>（未认证 2h/75 次限流）
- Base RPC: <https://mainnet.base.org> / <https://base-rpc.publicnode.com>
- Day 3 成本模型: notes/2026-08-07.md + research/icl_notes_summary.md（桥费 0.25% 线性、往返 -2x 单程）
- 脚本: scripts/lifi_quote_check.py（现成）、scripts/uni_v3_pool_price.py（本次新建）

笔记 037: | `/v1/quote` (GET) | `/v1/routes` (POST) |

作者：Freetrip | 时间：2026-08-08 13:25 UTC | 字数：693 | 评分：58

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

| `/v1/quote` (GET) | `/v1/routes` (POST) |

| :---- | :----- | :----- |

| 返回 | 单条最优路由，直接可执行 | 多条候选路由，排好序让你挑 |

| 场景 | 脚本自动成交、只要最优解 | 前端给用户「快/便宜/少跳」几个选项 |

quote 返回单个最优 Step 对象，LI.FI 已经替你排序选好。\\

quote 的返回里直接带 transactionRequest（那段 data calldata）拿到就能签名广播，一步到位。

routes 返回 routes[] 数组，多条候选按你指定的排序（最便宜/最快/推荐）排列，让你自己挑。\\

routes 的返回里每条 route 的 steps 只有 estimate，没有 transactionRequest。你选中一条后，得再对每个 step 调 `/v1/stepTransaction`（或用 SDK）去换取真正的上链交易数据。

流程不同：\\

quote: GET `/v1/quote` ——> 单条 Step(含 transactionRequest) ——> 直接签名发送\\

routes: POST `/v1/routes` ——> 多条 Route(只有 estimate) ——> 挑一条 ——> `/v1/stepTransaction` 逐步取 tx ——> 发送

quote 是「快餐，端上来就吃」；routes 是「菜单，点了还要现做」。

笔记 038: 链上发现一个价差，只是套利流程的开始。最后能不能执行，要看资金从哪里来、走哪条路、多久到达、实际收到

作者：wayhome | 时间：2026-08-08 13:29 UTC | 字数：1002 | 评分：77

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

链上发现一个价差，只是套利流程的开始。最后能不能执行，要看资金从哪里来、走哪条路、多久到达、实际收到多少，以及中间会损耗多少。

执行之前也不能只看两个市场的价格。至少要先拿到真实 Quote，检查服务费、Bridge 或 Swap 费用、Gas、滑点、Price Impact、最低到手数量和预计耗时。最好再做一次链上模拟。只有这些数字都放进去之后，才能判断这个机会是不是还有利润

LI.FI (<https://li.fi/>) 做的第一件事，是把 Chain、Token 和工具标准化。你集成一家，就瞬间拥有了大部分区块链、token price 等数据，也可以任意 swap 和 bridge。同时也会给你做好实际价格的计算、手续费等处理。

接下来，LI.FI (<https://li.fi/>) 会在它接入的 Bridge、DEX 和 Solver 中寻找可以执行的路径。

Quote 最重要的是下面的一些数据：

toAmount：预计收到多少

toAmountMin：考虑滑点后最低保证多少

feeCosts：

LI.FI (<https://li.fi/>)

、Bridge、DEX 或其他工具收取的费用

gasCosts：预计需要的链上 Gas

executionDuration：预计完成时间

tool 和 includedSteps：实际走了哪些工具和步骤

transactionRequest：等待钱包签名的交易数据

跨链套利

同一个 Token 在不同 Chain 或不同市场之间出现价格差时，LI.FI (<https://li.fi/>) 可以计算资产跨链和兑换的实际 Route。这里必须把跨链耗时考虑进去。很多套利窗口只有几秒，普通跨链流程却未必具有原子性。Quote 看起来盈利，不代表资产到达目标链时机会还存在。

更可行的做法往往是在多条链预先准备资金，先在两端完成交易，再用 LI.FI (<https://li.fi/>) 做库存再平衡。这样可以把“抓机会”和“跨链调仓”拆开。

##

笔记 039: 今天按照昨天确定的方向，完成了第一个 LI.FI 只读报价观察器。我没有连接钱包、没有签名，也没有发

作者：rakisa | 时间：2026-08-08 13:29 UTC | 字数：1990 | 评分：90

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天按照昨天确定的方向，完成了第一个 LI.FI 只读报价观察器。我没有连接钱包、没有签名，也没有发送交易，而是用公开占位地址调用 Quote API，研究真实路由返回的到账量、费用、Gas 和预计时间。

今天做了什么

我选择 Base USDC → Arbitrum USDC，分别测试 100、1,000 和 10,000 USDC 三种输入金额。请求使用 stablecoin preset、CHEAPEST 排序，并显式设置 0.5% 滑点参数。

为了避免只看单程报价，我把每笔去程的 toAmount 继续作为回程输入，再请求 Arbitrum USDC → Base USDC，形成一个只存在于报价层面的 Paper Trading 闭环。

整个过程一共保存了六份原始报价、一个 JSON 汇总和一个 CSV 汇总。

实际结果

```
| 初始金额 | 去程预计到账 | 回程预计到账 | 往返数量减少 | 减少率 |
|-----:|-----:|-----:|-----:|-----:|
| 100 USDC | 99.75 | 99.500625 | 0.499375 | 0.499375% |
| 1,000 USDC | 997.5 | 995.00625 | 4.99375 | 0.499375% |
| 10,000 USDC | 9,975 | 9,950.0625 | 49.9375 | 0.499375% |
```

三档去程都使用了 Fee Collection → Eco。回程中，100 和 10,000 USDC 仍然走 Eco；1,000 USDC 的回程却选择了 Polymer Standard。

对应的两腿预计总时间分别是：

100 USDC：约 14 秒；
1,000 USDC：约 1,087 秒，也就是约 18.1 分钟；
10,000 USDC：约 14 秒。

两腿 Gas 估算分别约为 0.0305、0.0363 和 0.0305 美元。这些都是当时 API 的估值，不是已经发生的链上结果。

我对费用的理解

这六条报价都包含 0.25% 的 LI.FI fee collection。去程后剩余 99.75%，回程再从剩余数量中扣一次，所以：

text

$0.9975 \times 0.9975 = 0.99500625$

$1 - 0.99500625 = 0.499375\%$

这正好对应三档报价的往返数量减少率。

如果按 1 USDC 约等于 1 美元，把 Gas 也近似计入：100、1,000、10,000 USDC 三档的总成本率约为 0.5299%、0.5030% 和 0.4997%。金额越小时，几美分 Gas 的相对占比越明显；金额变大后，按比例收取的费用成为主要成本。

不过这个估算还没有包含首次 ERC-20 授权可能产生的额外 Gas、报价变化、失败交易和跨链等待期间的资金成本。

今天发现的两个容易犯错的地方

第一个错误是认为相同资产、相同链组合一定会走同一条桥。本次只有 1,000 USDC 的回程被切换到了 Polymer Standard，而且预计时间从 7 秒增加到了约 18 分钟。单次实验不能解释原因，但足以说明观察器必须同时保存金额、时间、桥和预计时长，不能只保存到账数量。

第二个错误是直接用 fromAmountUSD - toAmountUSD 计算路线成本。API 当时给 Base USDC 和 Arbitrum USDC 的美元价格略有差异，因此直接相减会把价格源差异也算成成本。对于同名 USDC，我应该先比较 token 单位，再单独统计 fee 和 Gas。

今天的结论

本次三个闭环最终得到的 USDC 都少于初始数量，再加上 Gas 后更不可能构成正收益，所以今天没有发现套利机会。

更重要的是，即使某一时刻的报价闭环出现正数，也不能直接当作可执行套利。Base → Arbitrum → Base 不是原子交易，完成第一腿时第二腿报价可能已经改变。本次实验的价值是建立了真实数据结构和成本基线，而不是证明某条路径能赚钱。

下一步

下一步准备把一次性脚本改成定时观察器，重复采集同一组路线，并比较：

stablecoin preset 默认 0.1% 滑点与 0.5% 的区别；
CHEAPEST 与 FASTEST 的路线差异；
一段时间内桥、Gas、费用和预计时长如何变化；
接口失败、无路线和报价过期应该如何记录。

笔记 040: Day 4 | 从 Route 监控到可实现盈亏的套利全流程

作者：dyhbrewer | 时间：2026-08-08 13:41 UTC | 字数：927 | 评分：68

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 4 | 从 Route 监控到可实现盈亏的套利全流程

今天把研究目标从“观察 Ethereum USDC → Arbitrum USDC 的 LI.FI Route”升级成端到端套利系统设计。

最重要的结论：跨链桥接是非原子的，看到价差后再桥接、再去另一端成交，往往会在到账前失去机会。更可执行的结构是两端预置库存，在便宜端买入、贵端同步卖出，LI.FI 主要负责交易后的库存再平衡，而不是承担每次套利的实时执行腿。

因此，单纯把 1,000 USDC 从 Ethereum 桥到 Arbitrum 并不产生 Alpha，收到的仍然是 USDC，只会扣除 LI.FI Service Fee、Bridge / DEX / Solver Fee 和 Gas。真正的机会必须来自不同交易场所对同一资产的可成交价差，例如 Kraken ETH/USD 与 Arbitrum

WETH/USDC。

我重新定义了机会计算：

压力测试净利润

\= 卖出端真实 VWAP

买入端真实 VWAP

CEX 手续费

DEX / Solver / Bridge Fee

两端 Gas

库存再平衡摊销

Slippage 与 Price Impact

延迟、失败和资金占用准备金

只有行情与

Route

均未过期、两边深度足够、连续多个样本的压力测试净利润为正、库存充足且链上仿真成功，才把信号定义为“可执行机会”。

系统将拆成六层：行情采集与标准化 → 机会引擎 → 风控与成交前仿真 → 双边执行状态机 → 成交/到账核对 → 实际 PnL 与库存再平衡。

开发顺序也明确为：

1. 只读行情与 Paper Trading；
2. Shadow Execution，只生成拟执行订单；
3. 小额人工确认；
4. 带单笔、日亏损、库存偏差、Gas 和连续失败熔断的限额自动执行。

下一步将在现有 LI.FI Route Monitor 基础上加入 Kraken WebSocket 深度、Arbitrum DEX 报价、机会引擎、Paper fills、压力测试净利润和库存模拟器。真实交易权限、钱包签名和提现权限暂不接入

笔记 041: Day 4 · integrator 陷阱 (实测 2026-08-07, 8/8 打卡)

作者：99hansling | 时间：2026-08-08 13:41 UTC | 字数：2495 | 评分：88

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 4 · integrator 陷阱 (实测 2026-08-07, 8/8 打卡)

Day 3 跑闭环时撞见一个不一致：扫描器（默认 integrator=jumper.exchange）的 quote feeCosts 全空，而 Day 2 手跑（不传 integrator）有一笔 25 bps 的 LIFI 固定费。

Day 4 的任务正是验证这个参数如何改变结论。今天提前跑了对照。

对照实验：同路径，带 / 不带 integrator

路径 ARB USDC→WETH→USDC, slippage=0.005, 两档规模。两组都在 2026-08-07 跑出。

| 规模 | integrator | strict net bps | gas (双腿) | feeCosts | 结论 |

|---|---|---|---|---|---|

| \$1,000 | jumper.exchange | 91.84 | \$0.1026 | [] (无费) | negative |

| \$1,000 | (不传) | 153.80 | \$0.1500 | LIFI Fixed Fee 25 bps/腿 | negative |

| \$10,000 | jumper.exchange | 97.66 | \$0.0363 | [] (无费) | negative |

| \$10,000 | (不传) | 153.72 | \$0.1867 | LIFI Fixed Fee 25 bps/腿 | negative |

integrator 吃掉的边际：\$1k 档 61.96 bps, \$10k 档 56.06 bps。

计划预计 ≈ 50 bps (双腿各 25)。实测略高，差额来自费率复利 (included=true, 已从 toAmount 扣除，跨腿复配) 加两次报价间的路由漂移。量级吻合，方向确定。

feeCosts 的真身 (无 integrator, N=10000 前腿)

json

```
{ "name": "LIFI Fixed Fee", "amountUSD": "24.9823",  
  "percentage": "0.0025", "included": true,  
  "feeSplit": { "lifiFee": "25000000", "integratorFee": "0" } }
```

25 bps (0.0025), included=true → 已从 toAmount 扣掉，不会重复计费

传 integrator=jumper.exchange 时这条整个消失 (jumper 作为 integrator 谈下了 0 LIFI 费)

这就是 Day 2 那笔 25 bps 费的来源，也是今天扫描器看不到它的原因

Day 4 的教学点 (已坐实)

一个你不知道的参数，可以让结论差 62 bps。同一条路径、同一个时刻，带不带 integrator，strict 净边际从 92 滑到 154。两边都是负数、verdict 不变，但如果你在边界附近（比如某条线 strict net 恰好在 0 附近），这个参数会直接翻号。

从今天起每条证据记录必须写明 integrator——扫描器默认 jumper.exchange，手跑若忘了传，等于多算 50 bps 成本。

与仪器结构性偏置的关系（不是 Day 4 作业，但顺带记下）

这组对照也印证了 reflection-ledger 那条批评的一个侧面：同一台仪器、同一个市场状态，仅因一个参数就摆动 62 bps——说明「strict net = 92」这个数字本身携带的信息，比它看起来的少。真正能翻号的边际（>100 bps）要么在 screen 层被当 suspect-feed 剔除（DEFAULT_SCREEN_BAND_BPS=[5,150]），要么在 classify() 被 >20 bps 判 implausible（src/cost.mjs:186）。接受窗口仍是经验空集；今天的对照只是量出了其中一个偏置源的大小。

产出物

证据记录：data/evidence/2026-08-07.jsonl（含两组共 4 条）

原始响应：data/raw/2026-08-07T14-33--quote.json（无 integrator 那 6 个）

配额：本日累计 12 次 /quote，窗口剩 ~63

额外：把 LI.FI 接进了 edgeprobe agent 流水线

原始意图是「以 LI.FI 跨链路由/报价为主信号源」让 agent 自动发掘机会，但 edgeprobe 之前只跑 perp funding，全仓库 grep lifi 零命中。今天补上：

- 新增 edgeprobe/connectors/lifi-crosschain.ts：LI.FI /quote connector（keyless 只读）+ 闭环留存函数（3 quote/loop，opt-strict spread $\approx$ 2xslippage 诊断）+ 75/7200s 限流 pacer
 - 新增 src/cli/crosschain-research.ts：跨链研究 runner，live 跑通 ARB USDCWETH
→ strict 100 bps negative，证据存 data/evidence/
 - 10 个测试（只读性、密钥扫描、限流、闭环留存、诊断），edgeprobe 95→106 全绿
- 这把「打卡用的 LI.FI 扫描器」和「agent 流水线」之间那道缝补上了。

笔记 042: 链上套利共学 · Day4 打卡笔记（08/08）

作者：haibian-slowly | 时间：2026-08-08 13:43 UTC | 字数：2704 | 评分：88

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

链上套利共学 · Day4 打卡笔记（08/08）

今日信息

日期：2026-08-08（周六）

主题：第一次调用 LI.FI Quote

核心问题：怎么拿到一份真实报价？

打卡正文

【第 4 天】

1. 今天要验证的问题

怎么拿到一份真实报价？报价长什么样？哪些字段决定“能不能执行”？

2. 今天实际做了什么

1. 用 LI.FI Quote API（GET <https://li.quest/v1/quote>）请求了一笔真实报价：Arbitrum USDC（42161）→ Optimism USDC（10），金额 1000 USDC；同一参数也通过 MCP get-quote 走了一遍，两路返回一致（MCP 就是包的同一个端点）；
2. 把完整原始 JSON 保存到了 onchain-arbitrage-lab/data/raw_quote_2026-08-08_arb42161_opt10_1000usdc.json（不是只打印价格）；
3. 逐字段拆解返回结构：action / estimate / feeCosts / gasCosts / includedSteps / transactionRequest / approvalAddress；
4. 把真实报价里的成本项映射到 Day 3 的 cost_model 七项成本，验证模型字段在真实数据里的落点。

3. 证据（本次真实运行结果）

路由概览：tool = eco（Eco 桥），共 2 步（includedSteps）：

feeCollection（源链扣 0.25% 协议费，42161→42161）→ cross（Eco 桥，42161→10）

| 关键字段 | 本次实测值 | 含义 |

|---|---|---|

| action.fromAmount | 1,000,000,000（1000 USDC，\$998.96）| 输入，最小单位（decimals=6）|

estimate.toAmount	997,500,000 (997.5 USDC, \$996.26)	最终到手
estimate.toAmountMin	997,500,000 (= toAmount)	滑点下限; 稳定币路径实测滑点=0
estimate.feeCosts	LIFI Fixed Fee 2.5 USDC (0.25%, included=true)	源链直接扣的协议费
estimate.gasCosts	ETH ~\$0.024 (gasLimit 3,434,500)	链上 Gas, SEND 类型
estimate.executionDuration	7 秒	跨链预计耗时
estimate.approvalAddress	0x1231DEB6f5749EF6cE6943a275A1D3E7486F4EaE	ERC20 授权对象 (随路由变化)
transactionRequest	to/from/chainId/gasLimit/gasPrice/data (1925 字节) 齐全	可直接签名的交易体
integrator	lifi-api (默认)	集成方标识

成本合计 (真实数据): 输入输出 = 1000 997.5 = 2.5 USDC (全部来自 LIFI Fixed Fee) + Gas \$0.0239 ≈ 总成本 ~\$2.52, 占本金 ~0.25%。

4. 得到了什么结论 (来自数据)

1. 稳定币跨链的成本大头是协议固定费 (0.25%), Gas 几乎可忽略 (\$0.024); 该路径滑点为 0 (toAmount == toAmountMin, Eco 稳定币 1:1 路径无价格冲击) ——与 Day 3 模型里“桥费是最贵部分”的判断一致, 但真实数据告诉我固定费按输入扣、不是按输出;
2. 成本在报价里分两层: feeCosts (按代币扣的协议费) 和 gasCosts (按原生币 ETH 扣的 Gas) ——正好落进 Day 3 模型的 protocol_fee 和 gas 两个字段;
3. 看跨链收益要看 toAmountUSD 而不是只看币数量: \$998.96 → \$996.26 差了 \$2.70, 比费用 \$2.50 还多——因为 USDC 两链价格微差 (Arb 0.998964 vs OP 0.998761)。算净收益时必须用 USD 口径;
4. 报价是“可执行数据”不是“参考价格”: transactionRequest 字段齐全 = 签名即执行, approvalAddress 是授权对象——第一次真正看懂「报价 → 授权 → 执行」流水线;
5. 报价时效很短: 两次请求 (MCP/REST) 的 gasPrice 已经不同 (0x131b1d0 → 0x131b9a0), 执行前必须重新拉取, 不能拿旧报价签名。

5. 今天失败了什么

没有真实发送交易——只读阶段, 符合铁律: 花钱/签名/转账永不自动;

传参踩坑: MCP get-quote 的 fromToken/toToken 只认合约地址不认符号——USDC 在每条链是不同合约 (Arb 0xaf88...e5831, OP 0x0b2C...Ff85), 用符号会报错或拿错标的。

6. 明天只解决什么

批量采集不同规模报价: 交易规模如何影响最终到手金额? (大额是否触发更深的流动性/更高费率的桥? 滑点什么时候开始咬人?)

一句话总结

报价 = 一条真实可执行的路由: 1000 USDC 跨链到手 997.5 (0.25% 固定费 + 0 滑点 + \$0.024 Gas); 报价不是参考价, transactionRequest 齐了就等签名——但 60 秒内不执行就得重新拉。

明日预告

Day5 (08/09): 批量采集不同规模报价

笔记 043: Day 4 | 2026.08.08 | Solana 三角套利: 从发现到实盘

作者: xzone911 | 时间: 2026-08-08 14:37 UTC | 字数: 2545 | 评分: 90

来源: <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 4 | 2026.08.08 | Solana 三角套利: 从发现到实盘

今日目标

在 Solana 上实现原子三角套利闭环: WSOL → SPCXx → URANUS → WSOL。从 StonkFun 发现代币、到链上 Rust executor、到 Jito 竞价广播, 完整走通。

路径设计

WSOL → SPCXx → URANUS → WSOL(原子, 任一跳失败则回滚)

| 跳 | 协议 | 候选池 |

|-----|-----|-----|

| 第一跳 | Orca Whirlpool / Raydium CLMM | 2-4 个候选池, 按输入金额择优 |

| 第二跳 | Raydium CLMM(StonkFun 创建) | 固定桥接腿 |

| 第三跳 | Meteora DAMM v2 / DLMM | 2-3 个候选池, 择优 |

执行器设计

链上 Rust executor:

1. 验证被允许的 program、pool、mint、vault, 拒绝任意 CPI 注入
2. 记录用户 WSOL / SPCXx / URANUS 初始余额
3. CPI 第一跳, 用 after - before 得到真实输出
4. 用真实输出构造并 CPI 第二跳
5. 用真实输出构造并 CPI 第三跳
6. 断言最终 WSOL $\geq$ 初始 WSOL + min_gross_profit
7. 断言当前 slot $\leq$ valid_until_slot

关键设计: 不设中间跳 minimum_amount_out, 由外层程序做余额差断言。这避免了链下预估的舍入偏差。

主网验证结果

模拟对账(slot 437891857): Orca $\rightarrow$ Raydium $\rightarrow$ Meteora, 三次 CPI 全部成功, 链上真实 transfer delta 与本地数学逐 lamport 一致, 中间币 delta = 0, WSOL 漂移 = 0, 利润不足 ProfitTooLow 正确回滚。

短暂盈利窗口(slot 437892404): 最优池切换至 Meteora D7, 三段 CPI 全部成功, 毛利 +12,897 lamports, 仍不覆盖 tip + 交易费 + 安全垫 + 最低目标净利, 未广播。

关键发现

1. Token-2022 兼容性

SPCXx 是 Token-2022 代币(transfer-hook、pausable)。必须使用支持 Token-2022 的 swap 指令版本, 传入完整 mint/token program/memo/hook 账户。检查发现 hook programId=null、paused=false、无 transfer-fee——但每次快照必须重新解析 mint。

2. Jito 竞价策略

单交易 sendBundle, tip 内嵌在套利同一笔交易内。Tip = max(实时 50 分位 floor, 毛利 $\times$ 20%), 上限 100k lamports。利润不足覆盖 tip + 费 + 安全垫 + 目标净利时, 不构造、不签名、不广播。

3. 输入金额自适应搜索

0.01~1 SOL 粗搜 100 档, 最佳附近两轮 10 等分细化。最终每个方向覆盖 118 个金额, 全部复用同一份链上快照, 不增加 RPC 请求。

4. URMOM 第二条闭环

从 StonkFun 高量候选中发现 URMOM(SPL Token, 9 decimals, mint/freeze authority 为空):

- SPCXx/URMOM Raydium CLMM: 固定桥接
- URMOM/WSOL Meteora DLMM: 第三跳
- 24h volume ~\$0.8-0.9m, 直接 WSOL 池流动性 ~\$4-5k
- 已接入官方 DLMM 数学并通过 Jupiter 逐 raw unit 交叉验证

踩坑记录

1. DAMM CPI 参数顺序

动态 DAMM 把 output 与 optional referral 槽位写反。正向因为 output/referral 都是 WSOL 被掩盖, 反向才发现。

2. Orca SwapV2 参数

尾部两个 bool 应为 amount_specified_is_input, a_to_b, 旧 helper 写成相反。正向因为两个都是 true 被掩盖。

3. DEX Screener 索引不完整

新池、未收录池和低美元流动性都会漏掉。改为直接查询 Meteora 程序账户(getProgramAccountsV2 + memcmp 过滤), 找回了 DEX Screener 没有的链上池。

4. Helius processed backend skew

WSS 通知偶尔领先 HTTP 负载均衡。对 -32016 做 6 次线性退避(15-90ms), 不降低通知 slot, 不引入 confirmed/finalized 延迟。

5. X86 交叉编译

本机 aarch64, 目标服务器 x86_64/glibc 2.41。显式 target + vendored OpenSSL 解决交叉编译。私钥/RPC 通过 tmpfs credential 注入, 不写磁盘不写日志。

执行器安全边界

- 只允许 4 个固定 program(Orca/Raydium/Meteora DAMM/DLMM)
- 每跳验证 pool owner、mint、vault
- 最低毛利 = tip + 签名费 + 安全余量 + 目标净利润
- 资金不足、利润不够、slot 过期 $\rightarrow$ 不签名不广播
- 私有 RPC/Jito 专用, 不同时 fanout 到普通 RPC
- 同一时间最多一个未决 bundle

笔记 044: Day 4 学习笔记 (2026-08-08)

Day 4 学习笔记 (2026-08-08)

今天不新开方向，把前三天收拢成两个验证习惯。

一、跨链报价要用「时间分布」看，不用单次看。连续三天每 30 分钟拉一次 LI.FI 报价：USDC→USDC 成本极稳 (25.02~~25.34bps)，USDC→WETH 往返却会抖 24~~28bps、路由频繁切换。单次 22bps 可能是真实水平，也可能是噪声或路由临时切换——只有长期样本才能区分「真机会」和「报价幻觉」。另外验证了 integrator 参数陷阱：扫描器默认带 jumper.exchange 时 feeCosts 全空，25bps 被吃掉，报价前必须核对 feeCosts。

二、链上可见 ≠ 真实利润。Aave 清算和 Robinhood 三角共用同一套骨架：链下搜索→提交 EOA→执行合约→利润归集/refund。毛机会 0.53 ETH 全付给 builder，真利润靠链下退款；执行合约余额为 0。所以判断机会必须算「净收益 = 毛机会 协议费 兑换 Gas 失败成本 ± 退款」，而不是看屏幕价差。

03 MEV与抢跑 (5 篇)

笔记 045: MEV-Flashbot-Sandwich-RS : 从 Mempool 到 Bundle 的 MEV

MEV-Flashbot-Sandwich-RS : 从 Mempool 到 Bundle 的 MEV Searcher 架构

【项目定位】

MEV-Flashbot-Sandwich-RS 是一个以 Rust 为核心的 Ethereum MEV 学习与实验项目。核心方向是监听 mempool 中的 pending swap，分析高滑点交易，进行本地模拟和机会判断，再构造交易组合并通过私密通道提交。仓库还增加了 LI.FI 跨链套利磨损评估模块 (Go 后端 + Next.js 看板)。

【图 11-1 : MEV Searcher 核心流水线】

Ethereum / Mempool



核心认识：真正通用的能力不是某一种 Sandwich 策略，而是“监听 → 解码 → 状态模拟 → 机会计算 → 风控 → Bundle 提交”的完整基础设施。

1 Sandwich 的收益来自哪里

假设用户 Alice 准备在 AMM 中用大额 USDC 买 ETH。大额买单本身会推动池内 ETH 价格上涨。如果用户同时设置了较大的 Slippage Tolerance (滑点容忍度)，就给交易执行留下了一定价格空间。

【图 11-2 : Sandwich 的交易排序】

正常：

Alice BUY ETH → ETH 池内价格上涨

Sandwich：

Searcher BUY ETH (Front-run)

↓

Alice BUY ETH (Victim Tx，继续推高价格)

↓

Searcher SELL ETH (Back-run)

Searcher 不是在预测 ETH 的长期涨跌，而是在预测“一笔已经进入交易传播网络的订单将如何改变 AMM 状态”，并尝试利用交易排序提取价值。

策略并不是简单寻找“大额交易”，而是寻找：

大额交易 + 足够价格冲击 + 足够滑点空间 + 足够流动性条件 + 扣除全部成本后仍有正收益。

优化问题可以理解为：寻找最佳 Front-run 规模 x ，使 $\text{Net Profit}(x)$ 最大，同时必须满足 Victim 最终收到的资产数量仍不低于其 minOut ，否则 Victim 交易会失败。

2 mempool.rs：机器人的“眼睛”

mempool.rs 负责监听 pending transactions，并过滤值得分析的交易：

普通转账 / Approve / 无关合约 → 忽略

Uniswap Swap → 解码

大额 Swap / 高 Price Impact → 进入深度模拟

低延迟非常重要。如果节点只推送 tx hash，程序还需要再次调用 getTransaction 获取完整交易，就会增加网络往返。因此生产级 Searcher 通常会追求更直接的 orderflow、更快的节点连接和本地状态访问。

【图 11-3：公共 RPC 与本地节点的差别】

公共 RPC：

Bot → Internet → RPC → 查询交易/eth_call → Internet → Bot

本地基础设施：

Reth Node → IPC / Local State → revm → MEV Engine

后者减少网络往返，并允许策略引擎高速重复模拟不同交易规模。

3 mev.rs：策略与模拟“大脑”

mev.rs 可以理解为整个系统最核心的策略层：

识别目标交易 → 读取 Pool State → 模拟 Victim → 模拟候选 Front-run → 再模拟 Victim → 模拟 Back-run → 计算 PnL → 判断是否值得提交。

净利润不能只看 AMM 价差，应至少计算：

$\text{Net Profit} = \text{Trading} / \text{MEV Gross Profit} - \text{DEX Fees} - \text{Gas} - \text{Builder Payment} - \text{Priority Fee}$ 资金与失败机会成本。

所以“看到价差”距离“可执行盈利”仍有很大距离。

4 Reth + revm：为什么本地模拟重要

Reth 提供 Ethereum 节点和本地链状态；revm 可以作为嵌入式 EVM 执行与模拟引擎。组合后，Searcher 可以在本机测试多个候选参数，例如 1 ETH、2 ETH、5 ETH、10 ETH 的 Front-run 规模，再通过优化算法寻找更优 x 。

这也是 Rust 的价值所在：重点并非单纯“Rust 比 Python 快”，而是整个 network → node → state access → EVM simulation → strategy → signing 链路都需要尽可能降低延迟。

5 Flashbots / Builder：解决交易提交和排序问题

Searcher 找到机会后还必须解决“如何进入目标区块、并尽量按照需要的顺序执行”。因此系统会构造 Bundle。

【图 11-4：Bundle 概念】

Bundle

├ Tx A：Searcher Front-run

├ Tx B：目标交易 / 相关交易

└ Tx C：Searcher Back-run

↓

Builder / Relay

↓

Block

私密提交的重要意义之一，是减少策略交易直接暴露在公共 mempool 后被其他 Searcher 复制、抢跑或反向利用的风险。

生产环境通常还会考虑 Multi-Builder Submission，即同时向多个 Builder 渠道竞争 inclusion，而不是只依赖单一路径。

6 真实 MEV 竞争远比“写一个机器人”复杂

【图 11-5：真实竞争结构】

Searcher A

↗

Orderflow / Tx → Searcher B → Builder A/B/C → Proposer → Block

↘

Searcher C

竞争维度包括：

\- Orderflow access：谁更早看到可执行交易；

\- Node / network latency：节点和网络延迟；

\- Simulation speed：模拟速度；

\- Strategy quality：机会识别与参数优化；

\- Builder connectivity：Builder 覆盖；

\- Bid optimization：愿意让渡多少利润换取区块 inclusion；

\- Capital & risk control：资金效率和失败控制。

因此，“GitHub 项目能 cargo run”不等于“Ethereum 主网能够稳定盈利”。这个仓库更适合作为 MEV 基础设施与策略研究教材。

7 LI.FI 模块：与 Sandwich 完全不同的套利逻辑

仓库中的 li.fi 模块属于跨链价差与执行成本评估，而不是交易排序攻击。

【图 11-6：LI.FI Break-even Spread】

Chain A：ETH = \$3,000

Chain B：ETH = \$3,025

↓

Gross Spread $\approx$ 0.83%

↓

减：Swap Fee

减：Bridge / Service Fee

减：Gas A + Gas B

减：Slippage / Price Impact

减：跨链时间与执行风险

↓

Net Executable Edge

例如全部摩擦成本为 0.61%，市场价差为 0.83%，理论剩余 Edge 只有约 0.22%。所以跨链套利的关键不是“有没有价差”，而是“真实 Quote 和全部摩擦成本之后还有没有可执行利润”。

Go 后端的 breakeven 逻辑负责评估 Break-even Spread；Next.js 前端负责把报价、费用和磨损可视化。

8 把几种收益来源分开

【图 11-7：不同策略的收益来源】

Phoenix / Perp 激励套利 → Exchange Incentive

LI.FI 跨链套利 → Cross-chain Market Price Difference

DEX-DEX Arbitrage → AMM Price Difference

Back-run MEV → 大额交易造成的新价格失衡

Sandwich MEV → Victim Slippage / Ordering

Liquidation MEV → Liquidation Reward

LP 做市 → Trading Fees + Inventory Rebalancing Exposure

最大的学习重点是：Price Arbitrage 是在寻找市场价格不一致；MEV 更强调利用区块内交易排序和状态变化。两者可以交叉，例如大单造成 DEX 间新价差后，Searcher 通过 Back-run 吃掉价差。

9 Perp DEX 是否可以传统 Sandwich

不能简单认为“所有链上永续合约都可以夹”。如果杠杆开仓包含 AMM Spot Swap，相关 Swap 部分可能具有传统价格冲击；早期 vAMM 模型也可能存在排序型机会。但现代许多 Perp DEX 使用预言机定价、异步执行或专门撮合机制，传统 AMM Sandwich 并不是主要 MEV 来源。

更常见的其他方向包括：Liquidation MEV、Oracle / Latency Arbitrage、交易排序和清算竞争等。分析某个 Perp 协议时必须先理解其实际定价和执行模型，而不能只因为“它是链上合约”就认为可被传统 Sandwich。

10 对这个项目的学习价值判断

最值得学习的通用模块：

1\、Mempool / Orderflow Listener；

2\、Transaction Decoder；

3\、Local State + EVM Simulation；

4\、Opportunity & PnL Engine；

5\、Risk Control；

6\、Bundle Builder；

7\、Multi-Builder Submission；

8\、LI.FI Quote / Break-even Evaluation。

这套基础设施的价值在于策略可以替换：Sandwich、DEX-DEX Arbitrage、Back-run、Liquidation、Oracle Latency、跨链价差等都可以复用部分底层能力。

11 风险与长期趋势

\、Sandwich 属于高度竞争的 MEV 领域，收益会被其他 Searcher 和 Builder 竞价快速压缩。

\、私钥 AES-GCM 加密只解决“静态私钥存储”的一部分问题，并不能保证整个自动交易系统安全；合约授权、路由、策略逻辑、依赖库和服务器本身都可能成为攻击面。

\、Private Orderflow、Intent、Solver、Encrypted Mempool 等方向会继续减少部分公开 mempool 的抢跑和 Sandwich 空间。

\、因此，从长期学习角度，应把重点放在链上状态分析、模拟、执行成本、Back-run / Arbitrage、Builder 基础设施和风险控制，而不是只学习如何 Sandwich 用户。

12 本节结论

MEV-Flashbot-Sandwich-RS 最值得学习的不是“一个可以夹单的机器人”，而是一套 MEV Searcher 的工程骨架：

Mempool / Orderflow → Decode → State → Simulation → Opportunity → PnL / Risk → Bundle → Builder → Block。

对于套利研究，尤其应该形成一个固定习惯：任何机会都必须从“理论价差”继续计算到“真实可执行净利润”。LI.FI 模块的 Break-even Spread 思路与这一原则完全一致。

后续源码阅读建议：Cargo.toml → src/main.rs → src/mempool.rs → src/mev.rs → src/tx_sender.rs → ifi/backend/strategy/breakeven.go。逐层确认 README 中描述的能力究竟已经实现到什么程度。

笔记 046: markdown

作者：shuwenwy | 时间：2026-08-08 13:39 UTC | 字数：4931 | 评分：88

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

markdown

1. 总纲：三件事环环相扣

状态 → 排序 → 利润

状态：池子价格、流动性、slot0 / reserves 的真实链上状态

排序：交易在区块中的位置（谁先执行、是否被抢跑）

利润：扣掉 Gas、滑点、失败成本后仍为正的净利

三者缺一不可：状态算准但排不到位会被抢；排到了但状态过时 revert；利润模型漏计成本会“看似赚钱”。

配套铁律：

链上计算（或执行时读链）+ 私有提交通道，缺一不可

——前者解决「算得准 / 执行时仍成立」

——后者解决「不被抢」

2. 数据层：别指望聚合 API，自己算

2.1 公共 API 的天花板

| 痛点 | 典型表现 |

|---|---|

| 限流 | 常见约 1 req/s，撑不住高频扫描 |

| 延迟 | 响应约 ~100ms，机会窗口已过 |

| 本质 | 聚合方自己也是「读链 + 订日志」再加工给你 |

结论：最终都是自己算。公开 API 适合学习与低频监控，不适合竞争级套利。

2.2 成熟架构：RPC + 订阅 + 本地计算

自建 / 专用 RPC 节点

↓

订阅多池深度、价格区间、Swap 等日志

↓

本地维护状态 (reserves / tick / liquidity / hot pools)

↓

本地计算下单量、到手价、净利

要点：

用节点拿原始状态，不拿别人算好的「现价」

eth_call 读合约 view 函数：不花 Gas（只读模拟，不上链）

WebSocket / 日志订阅负责「及时知道谁动了」，不保证数据完整正确——最终仍要以执行时读链为准

2.3 Hot Pool 流水线（可落地的搜索骨架）

日志订阅 → 筛「最近活跃池」→ 维护 hot pool 名单

↓

新块到来 → 对 hot pools 批量 eth_call 探测

↓

有利润 → 立刻构造 tx 上链

前提：批量 eth_call / 模拟需要自建或高配额 RPC，公共端扛不住。

3. 两条执行哲学：链下模拟 vs 盲套利

3.1 标准路径（链下模拟再上链）

链下模拟有利润 → 发交易 → 上链成交

适用：机会稀疏、竞争弱、出块不极快。

失效条件：高速链 + 多人抢同一路径——模拟算完，机会已被别人吃掉。

3.2 盲套利（执行时再算 / 再验）

监测目标池流动性变动 → 立刻上链

↓

合约内读状态、算利润（或校验链下参数）

↓

有利润 → 执行 swap；无利润 → revert

特点：

不依赖「先完美模拟再动手」的时间窗口

失败交易会变多（无利润就取消）——这是设计结果，不是 bug

在极快链（讨论中如 robinhoodchain）上，常被描述为「没时间链下算完」

3.3 价格实时性的真正来源

价格实时性来自执行时读链，不是来自订阅推送。

订阅只回答「该不该去看」；成交瞬间的 slot0 / reserves 才决定这笔能不能赚。

4. 链上算 vs 链下算：社群争议与可统一模型

4.1 正方：合约内算利润 + 早 revert

观点（bbvi 侧）：

链上取流动性 → 算利润 → 无利润立刻 revert

早失败只烧少量 Gas；硬跑完整笔路径再失败，Gas 更贵且仍无利润

V3/V4 流动性区间「也能在合约里算最优金额」，但 Gas 很重

证据侧：有 call stack 的 slot0 / PoolManager 外部查询，说明执行时在读池状态

4.2 反方：利润链上不可控，staticcall 只做校验

观点（Mr.Lai 侧）：

「合约内算」往往吃不到利润，Gas 一高利润归零

看 gas used：重逻辑若在链上，Gas 形态会明显不同

更常见模式：

链下算好路径、数量、sqrtPriceX96 / tick

↓

上链 staticcall 读 slot0

↓

与链下参数对不上 → 立刻回滚

即：链上不做「完整利润求解」，只做状态一致性守卫。

4.3 可统一的工程模型

| 层级 | 做什么 | 成本特征 |

|---|---|---|

| 链下 | 路径发现、最优数量、净利估算 | CPU 便宜，状态可能过时 |

| 链上轻校验 | slot0 / tick / 最小利润断言 | Gas 低，防过时 |

| 链上重计算 | V3/V4 区间最优求解等 | Gas 高，仅当延迟收益 > Gas |

| 失败策略 | 尽早 revert | 用小失败成本换试错频率 |

调用栈辨析（bbvi）：

合约读自己内部变量 → 通常看不到「对外查池」的 call stack

出现对池子 / PoolManager 的 STATICCALL（如 slot0、getSlot0）→ 说明在外部查询流动性/价格

Uniswap V4：PoolManager.unlock → unlockCallback 是 flash accounting 标准形态

两种路线可以并存：链下算重、链上验轻；或高速场景下链上多算一点换「执行瞬间正确」。

5. 提交通道：私有通道 vs 「满地失败交易」

5.1 诊断法则

若某地址链上大量失败交易，很大概率没有用私有提交通道。

私有通道（如 Flashbots Bundle）常见形态是：会失败的交易根本不进公开区块。

5.2 经典 MEV workflow

扫 Mempool（看公共 pending）

→ 发现机会

→ 打包 Bundle（有序交易）

→ 提交 Builder

→ 进区块执行

→ 提取 MEV

5.3 路径发现的时代划分

| 时代 | 路径怎么找 | 本质 |

|---|---|---|

| Searcher 时代 | 扫 mempool | 抢别人暴露的机会 |

| Intent 时代 | Solver 竞争 | 抢「谁给用户意图更好的成交」 |

| 盲套利 | 链上 staticcall 探测 | 找别人还没盯到的路径 |

共性：谁先发现别人没发现的路径，谁才有利润。

主网 MEV 军备竞赛已数年；讨论共识是——优先找活跃但未饱和的链；主网纯套利常被形容为「大炮打鸟」。

6. 资金与商业逻辑

6.1 高频重复，不是单笔暴利

案例讨论中的量级（社群口述，需自行核验）：

~26 天约 68 万 笔交易

商业逻辑：高频小利重复，不是单笔吃肉

有人提到约「一周两个点」量级；另有地址「一周约 12U」——说明同赛道收益分化极大
盈亏同源：meme LP / 高波动路径，赚的逻辑与亏的逻辑是同一套风险源

6.2 闪电贷与「无本金」

Uniswap V2/V3/V4 路径上，池子天然支持闪电贷式拆借/回调，常无额外「闪电贷协议费」（与「ETH 活跃不活跃」无直接关系）
「无本金」指不囤大库存，不指零成本：仍有 Gas、失败率、竞争、机会密度

6.3 e 本位 vs 要对冲吗？

| 立场 | 论点 |

|---|---|

| e 本位 | Gas 付 ETH、利润也是 ETH → 账本按 ETH 数量看，不必对冲 |

| 对冲派 | ETH 代币价波动会吃掉「赚到的那个 e」的购买力；活跃度低 ≈ 利润薄，更经不起单边行情 |

实操含义：先定记账本位（ETH 数量 vs USD），再决定要不要套保。

7. 原理卡片（速查）

卡片 A · 读链不花 Gas

eth_call / staticcall / view

→ 本地或节点内模拟执行

→ 不广播、不改状态、不付 Gas

付 Gas 的是「真正发上链的那笔 tx」，包括失败的那笔。

卡片 B · 原子套利 + 最小利润守卫

多步 swap / 清算 / 还款 装进同一笔交易

末尾：balance 增量 >= minProfit，否则 revert

回滚防单腿；不防已消耗的 Gas。

卡片 C · 高速链上的触发器

流动性变动（日志）→ 触发

不要等「完美链下模拟」

上链后验利润 / 验 slot0

卡片 D · 基础设施最低配

自建或专用 RPC

+ 日志订阅（活跃池）

+ 批量 eth_call

+ （可选）私有提交 / Bundle

+ 链上 minProfit 或 slot0 校验

8. 争议对照表（便于继续深挖）

| 议题 | 观点 A | 观点 B | 工程取舍 |

|---|---|---|

| 利润在哪算 | 链上算，无利早 revert | 链下算，链上只验 slot0 | 延迟收益 vs Gas；高速链偏 A，成熟主网偏 B |

| 订阅有没有用 | 可推实时价 | 只负责「热池名单」，价以执行时为准 | 订阅做索引，读链做真相 |

| 外部查池是否多余 | 多此一举（应用内部状态） | 必须查（执行瞬间状态） | 看 bot 是「自持状态机」还是「无状态探测」 |

| 失败交易多 | 说明策略差 | 说明在公开 mempool 试错 / 无私有通道 | 结合失败率、Gas、利润账户一起看 |

| 要不要对冲 | e 本位不必 | 法币回撤必须 | 先定 KPI 本位 |

9. 分析工具与取证习惯（讨论中反复出现）

Phalcon / BlockSec：拆 call stack、看 slot0、PoolManager.unlock、WETH withdraw 等

DeBank：看利润归集与资产变化，不把「提交 EOA」当成赚钱地址

看 gas used + 调用栈深度：粗判链上逻辑轻重

看失败交易密度：粗判是否私有通道、是否「上链再验」

10. 一句话收束

别买别人的「现价」；订日志找热池；新块 eth_call 探测；

链下算路径，链上验状态或算净利；能私有提交就别裸奔 mempool；

商业上拼的是高频可重复的净利，不是故事里的单笔暴利。

附录

利润/行为观察：debank.com/profile/0xd71212080d90ffa50a80bff2af2ade6f99f29b9...

交易拆解示例：app.blocksec.com/phalcon/explorer/tx/robinhood/0x951e8796...

相关内部文档：链上套利实现原理与操作手册.md（含 0xD712... Robinhood 案例）、链上套利知识总结.md

笔记 047: 学习笔记：以太坊区块构建系列（Greenfield Capital 译文）第 1 课

作者：journeyanhk | 时间：2026-08-08 14:09 UTC | 字数：1241 | 评分：82

来源：https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205

学习笔记：以太坊区块构建系列（Greenfield Capital 译文）第 1 课

基于 Bruce 老师翻译的 Greenfield Capital「以太坊区块构建」系列文章（文章2入门篇 + 文章1需求侧研究），完成第 1 课学习：理解区块空间市场与优先费竞争。

学习内容

1. 区块空间市场

- 以太坊每 12 秒产生一个区块，区块空间是稀缺资源
- 关键认知：交易顺序改变结果，同一笔交易在区块不同位置价值完全不同
- 例：套利交易放区块前面执行=赚钱，放后面=机会消失

2. 两个市场维度

- 纳入市场（inclusion market）：交易能否上链，由基础费定价（EIP-1559 销毁）
- 位置市场（position market）：交易排在区块哪里，由优先费定价
- 简单转账只需纳入，套利交易必须抢位置

3. 交易供应链角色

- 验证者（Validator）：最终确认，保持轻量
- 区块构建者（Block Builder）：决定交易顺序的包工头
- 中继（Relay）：防作弊的公证人
- MEV 搜索者（MEV-Searcher）：扫描机会提交 Bundle 的人
- 订单流提供方：钱包、dApp 前端
- 流程：用户 → 订单流 → 路由层(RPC/OFA) → 构建者 → 中继 → 验证者 → 上链

4. 优先费竞争（核心认知）

- 两个套利者抢同一机会 = 在构建者拍卖中比出价（优先费）+ 比速度（延迟）
- 构建者选择出价更高的 Bundle，落选者一分钱赚不到
- 套利者愿意付几乎全部利润的原因：竞争逼的，不付=抢不到=赚0，付90%还能赚10%
- 唯一底线：出价不能超过利润

课后思考题（已完成）

Q1: 为什么转账不付优先费、套利必须付？

→ 转账只需纳入（基础费），套利要抢位置（优先费）

Q2: 两个套利者怎么竞争？

→ 构建者拍卖层面比优先费出价 + 比交易送达速度

Q3: 为什么愿意付几乎全部利润？对盈亏平衡 bps 的影响？

→ 竞争均衡下平均利润率约 10%（90% 用于优先费）；盈亏平衡要从 31-57bps 加上竞争税

关键收获

1. 优先费是竞争产物，不是主动付的：竞争越激烈，实际盈利越接近 0
2. 盈亏平衡模型升级：价差利润 - 所有成本 - 优先费 > 0 才算盈利
3. 散户套利难的根源：对手是 Wintermute 这类每笔付 \$3 优先费的专业玩家
4. 套利（价格趋同）≠ 三明治攻击（伤害用户）：前者是市场行为，后者是掠夺

下一步

第 2 课：三类 MEV 市场分析（统计套利 vs 三明治 vs 原子套利）

理解各市场的主导者、优先费强度和机会频率差异

笔记 048: @brucexu_eth 用 Hermes 制作的交互式教学网站\

作者：XCHA000 | 时间：2026-08-08 14:17 UTC | 字数：2800 | 评分：73

来源：https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205

@brucexu_eth 用 Hermes 制作的交互式教学网站\

网站地址：<<https://arb-demo.brucexu.xyz/>>\

发布时间：2026年8月7日（会持续更新）

一、核心第一性原理（最重要，必须先记住）

价格偏差只是表象，状态转换才是套利。

把链上套利统一看成一个闭环循环：

1. 观察状态（读取资产余额、池子储备、协议状态）
2. 执行一组有约束的转换（swap、借贷、清算、跨链等）
3. 回到同一计价资产

统一判据（核心公式）：\

终值 初值 全部成本 > 0

全部成本包括：

滑点（价格冲击）

交易手续费（swap fee）

Gas

竞价支付（priority fee / bid）

库存风险

时间风险

失败成本

“看起来有价差” ≠ “可执行利润”。\

区块状态、排序权、库存和时间都可能成为路径的一部分。

二、五大机制家族（机制边界）

Bruce 把链上套利相关机制分成五大家族，帮助建立清晰分类：

1. Atomic Arbitrage（原子套利）\

单笔交易内完成所有状态变更。失败则整笔 revert。\\

Bundle 则按 provider 的允许-revert 规则决定候选是否有效（不是简单的“整包回滚”）。

2. Inventory Arbitrage（库存套利）\

依赖预先持仓或跨域库存。不一定原子。\\

暴露结算风险、桥接风险、价格风险。

3. Convergence / Carry（收敛 / Carry 套利）\

持有价差直到它收敛。\\

收益来自时间和资金占用，而不是一次链上闭环。

4. Protocol Incentives（协议激励）\

奖励、返佣、清算激励会改变转换成本。\\

这种收入不是天然无风险套利。

5. MEV / Order Flow（MEV / 订单流）\

价值来自交易排序、包含权和订单流信息。\\

与价格套利重叠，但概念不完全等同。

三、交互实验室详细拆解

Lab 01：三角套利执行模拟器（Atomic）

路径：在三个常数乘积池中执行 $A \rightarrow B \rightarrow C \rightarrow A$

模型特点：连续 v2-style 教学模型（把上一腿的实际输出传给下一腿），不使用静态报价乘积。

为什么不能只乘三个报价？\\

报价乘积只描述无限小交易的边际价格。有限输入会沿着 $x \cdot y = k$ 曲线移动储备，下一腿会受到实际滑点和手续费影响。

公式（v2）：\

$\text{amountOut} = y \cdot [\text{amountIn} \cdot (1 - \text{fee})] / [x + \text{amountIn} \cdot (1 - \text{fee})]$

关键观察点：每一腿的滑点、手续费、执行价 vs 初始边际价、最终毛 PnL 与净 PnL（扣 gas/竞价后）。

Lab 02：MEV 价值流与归因

拆解谁真正拿到钱：

Searcher / Solver：发现机会并构造交易（通常拿最大份额）

Builder：组装区块并出价

Proposer / Validator：选择最高 bid 并提议区块

Protocol / LP：收取 swap fee / 协议费

L1 Burn : Base fee 被销毁 (价值汇, 不是任何人的利润)

关键原则 :

不重复计数 (内部转付不能算两次)

Bid / payout 数据 $\neq$ 净利润

Proposer 单次高额收入 $\neq$ 长期最高净利润

用户执行损失是独立反事实指标

Lab 03 : 私有交易与 Bundle 生命周期

完整流程 :

1. 本地构造并签名 (私钥不离开本地)
2. 提交到私有端点 (不进公共 mempool)
3. 针对目标状态模拟
4. 过滤无效候选
5. 参与区块构建竞争
6. Builder 向 Proposer 出价
7. Proposer 签署 blinded block
8. 发布并以 canonical block + receipts 确认真正包含

核心提醒 :

模拟成功 $\neq$ Builder 接受 $\neq$ 最终包含

默认 Bundle 通常要求未允许的交易不能 revert, 否则整个候选无效

可用 revertingTxHashes 或 canRevert 允许部分 revert

私有通道只改变传播与竞争方式, 不改变链上 gas 规则, 也不保证零成本或必然包含

Lab 04 : Mempool Backrun 模拟

定义 : 排在目标交易之后执行。

本实验只建模其中一种「反向交易 + 价格收敛」场景。

与 Sandwich 的区别 : Sandwich 是在用户前后两边夹击 ; Backrun 只在后面。

学习重点 : 用户交易造成价格位移后, 如何用后续交易收敛并获利 (模型内)。

Lab 05 : Uniswap 机制与 LP 实验室

版本对比 (v1 $\rightarrow$ v4) :

v1 : 只能 ETH-ERC20, 全范围 $x \cdot y = k$

v2 : 任意 pair, 全范围, 经典 0.3% fee, 有 TWAP

v3 : 集中流动性 (按 tick 区间), 多费率档

v4 : Singleton + Hooks + Flash Accounting, 可高度自定义

无常损失 (IL) 公式 (全范围 50/50) : \

$$IL(r) = 2 \cdot \sqrt{r} / (1+r) - 1$$

第一性原理解 IL : \

外部价格上涨时, 套利者把升值资产从池子里买走, LP 被迫持有更多贬值资产。这是持续再平衡带来的相对表现落后, 不是只有退出时才发生。HODL

Lab 06 : MEV-Boost 生命周期

追踪从订单流到最终确认的完整路径 (公开 / 私有), 包括 Validator 注册、Builder 竞价、Relay、mev-boost、Fulu V2 / legacy V1 发布分支等。

一句话总结今天的核心收获 : \

链上套利的本质不是“发现价差”, 而是在正确的约束下完成状态转换闭环, 并扣除所有真实成本后仍有正收益。

理解概念我都头晕了, 和AI

对话了很久, 感觉效率不高, 依旧没有实践, 但是更进一步了, 感觉没有理解的特别深刻, 看明天还记得多少

笔记 049:

作者 : corn15 | 时间 : 2026-08-08 14:53 UTC | 字数 : 8265 | 评分 : 90

来源 : <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

1. 一筆交易從送出到上鏈

簡化流程：

text

使用者 / Bot

|

| signed transaction

▼

RPC / Ethereum Node

|

▼

Mempool / Private Orderflow

|

▼

Builder

|

▼

Relay

|

▼

MEV-Boost

|

▼

Proposer

|

▼

Ethereum Network

|

▼

Validators attest

|

▼

Canonical Chain / Finality

要注意：

送出 transaction $\neq$ 已上鏈

上鏈 $\neq$ finalized

mempool 不是全球唯一且完全同步的 queue

不同 node 看到 pending transaction 的時間可能不同

transaction ordering 會直接影響 MEV 與套利結果

2. Slot 與 Proposer

Ethereum PoS 將時間切成 slot：

text

1 slot = 12 秒

32 slots = 1 epoch

每個 slot 會由 protocol 選出一個 validator 擔任 proposer。

概念上：

text

Active Validators

|

| RANDAO randomness

| + validator state

▼

pseudo-random proposer selection

|

▼

Slot N 的 Proposer

Proposer selection 不是：

誰 gas 出得高誰當 proposer

誰先搶到誰當 proposer

builder 自己決定 proposer

而是 consensus protocol 根據 validator state 與 randomness 決定。

3. Builder 是什麼？

Builder 的工作不是決定「誰有出塊權」，而是：

建立一個對 proposer 最有價值的 execution payload。

Builder 可以聚合：

text

Public transactions

Private transactions

Searcher bundles

Arbitrage bundles

Liquidation bundles

Backruns

...

再透過 transaction ordering 最大化整個 block 的價值。

例如：

text

TX A

TX B

Searcher Arbitrage Bundle

TX C

Liquidation Bundle

Backrun

...

Builder 最後會對 proposer 的 blockspace 出價：

text

Builder A: bid = 0.80 ETH

Builder B: bid = 0.92 ETH

Builder C: bid = 0.85 ETH

4. Relay 是什麼？

Relay 是 Builder 與 Proposer 中間的可信中介。

主要用途：

1. 聚合 builder bids
2. 驗證 builder 提交的 block / payload
3. 確認 proposer payment 是否成立
4. 暫時保管完整 execution payload
5. 在 proposer 簽名之前，不讓 proposer 看到完整 transaction contents
6. proposer 選中並簽名後，再揭露完整 payload

Relay 可以理解成：

text

Builder

|

| full block + bid

▼

Relay

|

|— validate block

└─ validate bid
└─ simulate
└─ temporarily hide tx contents
|

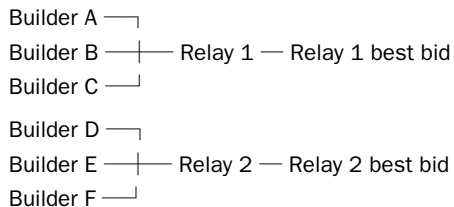


header / blinded payload + bid

5. Relay、MEV-Boost、Proposer 的關係

一個 relay 可以收到多個 builders 的 bids :

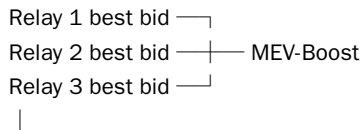
text



Proposer 通常會連多個 relays。

MEV-Boost 可以看成 relay aggregator :

text



highest valid bid



Proposer

所以可以理解為兩層：

text

Builders competition



Relay 選自己的 best bid



Relays competition



MEV-Boost 選整體 best bid



Proposer

6. Proposer 一開始看得到完整 block 嗎？

通常不是。

Relay 已經持有完整 execution payload，但一開始只會提供：

text

ExecutionPayloadHeader

+

Bid

而不是：

text

TX A

TX B

Arbitrage Bundle

TX C

...

流程：

text

Builder

|
| full execution payload + bid



Relay

|
| blinded payload / header + bid



MEV-Boost

|



Proposer

|
| 選中 bid
| 簽名 blinded block



Relay

|
| 驗證 proposer signature



Reveal full execution payload

|



Proposer 廣播完整 block

用詞修正

不是：

Relay 把 transactions 「解密」

比較準確的是：

Relay 原本就持有完整 payload，只是先 withholding；proposer 簽名後再 reveal / unblind。

7. Builder、Proposer、Searcher 各自賺什麼？

Searcher

Searcher 尋找：

Arbitrage

Liquidation

Backrun

其他 MEV 機會

例如：

text

套利總價值 = 1.0 ETH

Searcher 願意付給 Builder = 0.8 ETH

Searcher 自己留 = 0.2 ETH

Builder

Builder 收集：

text

Priority fees

Searcher payments

Private orderflow

其他 MEV

假設整個 block：

text

Gross block value = 1.0 ETH

Builder 為了贏得 proposer auction :

text

Bid to proposer = 0.9 ETH

則粗略：

text

Builder margin

= Gross block value - proposer bid

= 1.0 - 0.9

= 0.1 ETH

還要再扣：

Server

Network

Engineering

Orderflow rebates

其他營運成本

Proposer

Proposer 擁有：

這個 slot 的 block proposal 權。

Builder 彼此競爭，會用 bid 買這個 blockspace。

因此 proposer 可以獲得：

text

Builder bid

+

Consensus rewards

8. 為什麼 Builder 願意把大部分收益給 Proposer ?

因為 builders 彼此競爭。

例如：

text

Block gross value = 1.0 ETH

Builder A bid = 0.50

Builder B bid = 0.70

Builder C bid = 0.80

Builder D bid = 0.90

Proposer 會偏好：

text

Builder D

Builder D：

text

收入 1.00 ETH

支付 proposer 0.90 ETH

毛利 0.10 ETH

因此 builder competition 會把大量 MEV value 競爭回 proposer。

9. Searcher Auction 與 Builder Auction 是兩層不同競爭

可以這樣理解：

text

原始鏈上 MEV 機會

|

▼

Searcher

「我要交多少價值給 Builder，

才能換到 inclusion ? 」

|



Builder

「這整個 block 值多少？

我要出多少 bid 才能贏？」

|



Proposer

「誰的 bid 最高？」

所以：

text

看到 1 ETH 套利

不代表：

text

你能賺 1 ETH

真正收益可能被：

Searcher competition

Builder competition

Proposer auction

逐層壓縮。

10. 為什麼 slot 有 12 秒，套利仍然在意毫秒？

核心觀念：

12 秒是 consensus 的節拍，不是套利 bot 的 reaction-time budget。

假設：

text

t = 0 ms 新 state / pending tx 出現

t = 50 ms Searcher A 看到

t = 80 ms A 算完套利

t = 100 ms A 把 bundle 送到 builder

t = 150 ms Searcher B 才看到

t = 220 ms B 算完套利

如果 A 的交易先改變 pool state：

text

A 執行前：

DEX 1 = 3000

DEX 2 = 3020

A 執行後：

DEX 1 $\approx$ 3010

DEX 2 $\approx$ 3010

B 雖然距離 slot 結束還有很多秒，但套利已經不存在。

因此真正競爭的是：

text

誰先看到 state change

↓

誰先 decode / simulate

↓

誰先找到 profitable route

↓

誰先送到 builder

↓

誰得到 inclusion

11. 為什麼自架 Node 可能比較快？

Public RPC

text

Ethereum P2P Network



Provider Node



Provider Backend



Load Balancer



WebSocket / HTTPS



Internet



Your Bot

Self-hosted Node

text

Ethereum P2P Network



Your Node



IPC / localhost / LAN



Your Bot

自架 node 主要改善：

1. Data latency

更早收到：

Block

Pending transactions

Logs / state changes

2. State access latency

Bot 可以直接透過 local IPC 查：

text

pool reserves

balance

allowance

contract state

simulation data

比反覆走 public RPC：

text

bot



internet



RPC provider



node

更快且沒有一般 public endpoint 的 rate limit 問題。

12. 但不是所有套利都需要自架 Node

可以粗略分類：

策略	Public RPC	自架 Node
學習 / Transaction Analysis	很夠	不必要
低頻鏈上套利	通常夠	可選
Block-to-block Arbitrage	可做	有幫助
Liquidation	視競爭程度	通常有幫助
Pending TX Backrun	比較吃虧	很有幫助
高競爭 Latency MEV	很難競爭	幾乎必要
Builder	不適合	必要

因此：

不應該因為要做套利，就直接假設一定需要自架昂貴節點。

比較合理的做法是先量測 bottleneck。

13. 套利速度可以拆成四類

text

1. Data latency

多快看到 block / pending tx

2. Compute latency

多快 decode、simulation、計算套利

3. Submission latency

多快把 transaction / bundle 送到 builder

4. Inclusion competitiveness

builder 是否願意放你

最後 winning builder 是否包含你的 bundle

自架 node 主要改善：

text

1. Data latency

2. State access / simulation latency

但不直接解決：

text

3. Builder connectivity

4. Bundle competitiveness

14. 本次最重要的心智模型

Ethereum PBS / MEV Supply Chain

text

Users / Pending Transactions

|



Searchers

找套利、清算、backrun

|



Builders

聚合 orderflow

排序 transactions

最大化 block value

|



Relays

驗證 builder block

暫時隱藏 tx contents

聚合 bids

|



MEV-Boost
聚合多個 relay bids

|



Proposer
選擇最高有效 bid

|



Ethereum Network

|



Validators Attest

|



Canonical Chain / Finality

15. 目前應記住的重點

1. 一個 slot 是 12 秒，但套利真正競爭可能發生在幾十到幾百毫秒內。
2. Proposer 是 protocol 選出的 validator，不是 builder。
3. Builder 負責建立高價值 block，並對 proposer 出價。
4. Relay 是 Builder 與 Proposer 中間的可信中介。
5. Relay 一開始不把完整 transactions 給 proposer。
6. Proposer 簽名選中的 blinded block 後，relay 才 reveal full payload。
7. MEV-Boost 會聚合多個 relay 的 bids。
8. Builder 被接受你的 bundle，不代表你的 bundle 最終一定上鏈。
9. Searcher、Builder、Proposer 是三個不同的經濟角色。
10. 自架 node 的價值主要是更低 data latency 與更快 local state access。
11. 對低競爭套利而言，public RPC 可能已經足夠。
12. 先證明 latency 是瓶頸，再投入昂貴基礎設施。

04 做市与LP（3 篇）

笔记 050: Day 4 : 全自動套利系統架構 v0.2

作者：Web3Rason | 时间：2026-08-07 17:11 UTC | 字数：5188 | 评分：87

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 4：全自動套利系統架構 v0.2

上一版 v0.1 的三條結論被推翻了兩條半。這篇主要記推翻的過程，因為錯誤的修正方式比結論本身更值得留下來。

一、三條決策級修正

1 「主流打不贏，那就換長尾」——這個推論是錯的

v0.1 寫「主流資產不進即時路徑；長尾/RWA 是可探索的例外」。方向對，但範圍錯得危險。

錯在把「長尾」當成一個整體，沒有區分同鏈原子與需要搬運兩種場景。

逐個實測長尾候選的成本地板（不是價差，是地板）：

| 資產 | 成本地板 |

| :----- | :----- |

| UNI @ Optimism | 562.9 bps |

| COMP @ Base | 427.8 bps |

| AAVE @ Optimism | 198.1 bps |

| BAL / ENA / MORPHO | \$1,000 規模無路由 |

長尾的成本地板比主流深一個數量級（主流跨鏈 25 bps，長尾 200–560 bps）。

「無人競爭」和「無流動性」是同一件事的兩面。

| 場景 | 長尾是什麼 |

| :----- | :----- |

| 同鏈原子（一筆 tx 內成立） | 縫隙——機器人沒鋪到 |

| 跨鏈／跨所搬運（付兩次成本地板 + 兩邊深度） | 墳墓 |

→ 選標的第一準則是「成本地板」，不是「波動率」或「有沒有人競爭」。

仍成立的例外不是「換冷門幣種」，而是「同一資產、事前已驗證有雙向路徑與目標鏈賣出深度」的淺池。

關鍵動作是把判準從「只比買入價」改成「同時驗買入價 + 跨鏈路徑 + 目標鏈同數量賣出深度」——訊號量大減，真實度上升。

2 LP 做市不是「已證偽」，是有量化分界線的條件命題

v0.1 整條標 已證偽，這是過度概括。

同一套方法修掉 2 個單位 bug + 1 個篩選偏差後，結論整個翻轉：

| 日波動 | 手續費對 LVR 的覆蓋率 |

| :----- | :----- |

| < 0.1% | 5.86x |

| 0.5 – 1.5% | 0.93x |

| > 3% | 0.33x |

分界線是日波動 1.5%。負的那組全是主流 ETH 池；正的那組是低波動小幣 1% 費率池（淨 APR +43.56%）。

不是誰對誰錯，是「你在哪個波動率區間做」。進場前算 手續費率 $> \sigma^2/8 \times$ 區間槓桿。

但規模天花板是硬的：因份額 >50% 被剔除的組合佔多數，實際可投遠小於 1 萬美元。定位是小資金實驗，不是主業。

3 幣股 RWA 要拆成三個假設分開判

| 假設 | 狀態 | 依據 |

| :----- | :----- | :----- |

| ① 休市／盤前盤後喂價落後 | 已證偽 | 該市場的 oracle 由 deployer 自己喂，用的就是跟 Yahoo/IB 同一份盤前盤後 last-trade。實測最大偏離僅 ~40 bp < 跨鏈成本 25–30 bp |

| ② 中介報價資產被抽乾 | 有實測 | 事件中淺池掛在公允價 +19%~+47%，而聚合價只動 +0.4% |

| ③ 事件驅動窗口 | 有實測 | 窗口 2 分鐘 / 10 分鐘 / 1 小時 |

通用教訓：整個方向裡，只有一個人去查了「oracle 是誰喂、喂什麼源」，一查就推翻了最多人在賭的前提。

「這個價格是誰產生的」必須排在所有價差分析之前。

二、執行層的兩條補正

「bundle 整包回滾」是錯的

v0.1 只寫「Atomicity：全成或全退」，套到 bundle 上會錯：

預設 bundle 要求未被明確允許 revert 的成員成功；意外 revert 讓候選無效／不可包含，不是「整包鏈上回滾」

revertingTxHashes / canRevert 可允許指定成員 revert

獲准 revert 的交易若仍被包含，已使用的 gas 仍會消耗

未包含的候選通常沒有鏈上 gas，但仍有發送、基礎設施與機會成本

→ 失敗成本要拆兩格：revert_included_gas（真燒錢）vs not_included_cost（只有機會成本）。

執行三段論：simulation $\neq$ builder acceptance $\neq$ canonical inclusion

模擬只是特定 state / slot 的預檢；RPC 回應或 bundle hash 不是 inclusion proof。

只有 canonical block、receipts 與實際 ordering 能證明包含。

失敗原因要能在日誌裡分辨：陳舊狀態 | 競爭 bundle | bid 不足 | 到達太晚 | 目標 slot 路徑不通。

三、研究紀律新增兩條（這兩條比上面任何一條結論都重要）

證據去重：轉述不是獨立樣本

「有 N 個人這樣說」會系統性虛胖：

看起來 4 人證實某方向可行 → 實際是同一筆交易，價格與 priority fee 完全一致，n = 1，且四人都不是當事人
兩份證據的數字完全一致到小數位 → 預設同源，除非能證明是各自量的

取樣是被「工具供給」決定的，不是被「機會分布」決定的

實測率最高的方向（47%）之所以最高，是因為那個方向剛好有免費唯讀 API。

而且其中 20+ 條測的是同一個常數——同一個數字被量了 20 遍，不是 20 個獨立樣本。

→ 讀任何測量分布前，先問「這批人手上有什麼免費量具」。

→ 實測率高 $\neq$ 機會多。它衡量的是量具供給。

四、參數驗證進度 (接續 Day 2)

#	參數	值	狀態
1	聚合器平台費	25 bps	官方 FAQ + 實打 API
2	聚合器 rate limit	匿名 75 次/2h ; Key 100 req/min	實測 header
3	直連橋費 (USDC/L2)	1.0–1.6 bps	實打官方 API
4	V3 深度計算法	liquidityNet 前綴和	原始碼
5	MEV tip 比例	策略相依 67–99%	學術數據
6	CEX 費率階梯	一般 10/10 bps ~ VIP 1.1/2.3 bps	官方費率頁
7	規模曲線	倒 U 型, 收斂點 ~\$10,000	五檔實測
8	長尾同鏈成本地板	198 ~ 563 bps	新增
9	同鏈 DEX 間雙邊費	60 bps 硬底線 (vs 實測價差 3.52 bps)	新增
10	LP 覆蓋率分水嶺	日波動 1.5%	新增
11	MEV 佔優先費支出	66–80%, 前 15 名後斷崖	新增 (鏈上資料可複查)
12	兩端滑點	一	必須即時算

五、一個對自己的檢討

我自己的彙整程式，同一天犯了兩次同樣的錯：在彙整層寫 `JSON.stringify(x).slice(0, N)`。

第一次砍掉 50% 的發現

修完之後，在新寫的腳本裡原封不動又寫一次，砍掉 32% (219,382 字元砍掉 69,382)

第二次的代價很具體：被砍掉的那段剛好包含樣本量最大的那份實測 (14,219 個樣本)，而且系統性壓低了資料最支持的那個方向 (某方向「正在做」人數被低估一倍以上)。

修法不是把 N 調大——資料變多又會爆。修法有兩條：

1. 結構上不讓單一步驟吞全部：按類別分檔，各自處理，合併層只讀摘要
2. 計數交給程式，不交給模型——模型讀漏會數錯，而且不會告訴你它讀漏了

這條放進紀律：任何「靜默丟資料」的寫法，比錯誤的結論更危險，因為錯誤結論會被下一份證據推翻，靜默截斷不會——它看起來永遠正常。

附：目前的優先序

優先	動作
P0	成本工程：把成本地板從 25 bps 降到 1–2 bps (直連橋)。但這是必要不充分——把 break-even 降下來不會憑空生出機會
P0	驗證層：toAmountMin 取代 toAmount、token 口徑、報價源污染測試、雙邊建模
P1	事件驅動 + 事前備妥 (優勢在事件發生前資金與監控已就位，不在當下看到消息)

笔记 051:

在網上看到了Polymarket的套利文章，不過因為台灣網域被封的原因，我使用predict的資料進

作者: youchen-design | 时间: 2026-08-08 12:28 UTC | 字数: 1581 | 评分: 86

来源: <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

在網上看到了Polymarket的套利文章，不過因為台灣網域被封的原因，我使用predict的資料進行驗證與計算

文章:<https://x.com/hunterweb303/status/2085552085618340330?s=20>

結果:1.. 先把數學化簡乾淨 (推文這裡有誤導)

我從 SDK 文件驗證了轉換規則：投入 n 張 NO $\rightarrow$ 拿回 $(n1)$ 抵押品 + 每個剩餘結果各一張 YES。代入後整條式子塌縮成：

利潤 = (所有 K 個結果「實際脫手 YES 的價格」總和) $- 1$

與你選哪個子集無關。推文花大篇幅講的「尋找最賺錢的子集」不創造利潤，子集只影響本金佔用和腿數。你真正在收的是多結果盤口的 overround (貼水)。這會直接影響你要不要照他那份開發需求去寫「子集搜尋器」——那是在優化一個不影響損益的東西。

2. 手續費公式 (我用 210,843 筆真實成交腿反推，不是看文件)

$fee = 1.8\% \times \min(p, 1p) \times shares$ 只有 taker 付，maker 恆為 0

擬合變異係數 0.044，很乾淨。maker 那 110,050 筆腿費用合計 \$0。

(中途我算錯過一次：fee_type=SHARES 的欄位單位是「股」不是美元，沒換算會把低價位費率灌到 66%，修正後是平的 1.8%。)

3. Taker 版：實測是負的

453 個 NegRisk 事件裡只有 10 個 $\Sigma \text{bid}(\text{YES}) > 1$ ，而這 10 個的手續費全部大於毛利：

事件	毛利	手續費	淨利
afc-west (K=4)	+0.0160	0.0183	0.0023
lol-worlds (K=6)	+0.0090	0.0127	0.0037
fed-sept (K=5)	+0.0070	0.0133	0.0063
fed-cuts (K=13)	+0.0050	0.0059	0.0009

盤口剛好被套利到手續費邊界——這不是巧合，這就是均衡。論文研究期間 Polymarket 收 0 手續費，predict.fun 收 200bps，這個差異就是走不通的全部原因。

4. Maker 版還活著

$\Sigma \text{ask}(\text{YES})$ 1 約 0.055~0.081，即每輪 0.5%~1.6%、免手續費。而且流量是真的（fed-cuts 名目 \$933k、polymarket-migration \$535k，成交都在數小時內）——「沒人交易」這個否認我試過了，不成立。

代價就是推文說對的那件事：你要在 K 條腿同時被動成交，缺腿就是裸方向部位。這是做市，不是套利。

5. 動手前必須先測的一個數字

SDK 原文：轉換的 fees 「deducted from both outputs」。轉換本身可能有費，若存在會吃掉整個 0.5%~1.6%。這是我無法在不實際下單的情況下驗證的唯一變數，必須用小額實單測——在寫任何監控工具之前先測，因為它可能直接歸零整個結論。

補充：makerRebateBps 只有 Up/Down tag 有 2500bps，NegRisk 市場沒有，幫不上忙。

笔记 052: (本打卡由 Hermes Agent 辅助整理并提交。内容全部来自我的研究和实践，Agent 仅负责)

作者：geyu210 | 时间：2026-08-08 13:08 UTC | 字数：1045 | 评分：71

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

(本打卡由 Hermes Agent 辅助整理并提交。内容全部来自我的研究和实践，Agent 仅负责格式排版与提交流程。)

【今天做了什么】

今天继续研究 1inch Aqua，从开发者文档和合约源码入手，进一步理解了 Aqua 的核心机制：资金仍然在 maker 钱包中，Aqua 只记录 virtual balance、allowance 和 position 状态；position 由 app、strategyHash 和 token 共同确定。

在本地完成了一个只读检查程序，可以在同一个区块读取：

- maker 钱包余额；
- 对 Aqua 的 ERC-20 allowance；
- Aqua 的 rawBalances；
- token slot 的 active、disabled 或 inactive 状态；
- 当前 allocation 是否得到钱包余额和 allowance 的账面支持。

同时修正了两个容易误解的地方：

- tokensCount = 0 不能证明 position 从未创建，只能说明当前 token slot 没有 active 证据；
- fully_backed 只代表账面余额覆盖，不代表实际 swap 一定可以成交。

找到了一笔真实的 Ethereum 主网 Aqua position。它包含 1INCH 和 UNI 两个 token slot，当前仍然是 active，并且两边都显示为 fully_backed。其中 1INCH 的 virtual balance 约为 1336.745，UNI 的 virtual balance 约为 49.77。

整个过程只使用公开 RPC 和只读合约调用，没有连接钱包、读取私钥、签名或发送交易。

【收获与反思】

Aqua 和普通 pool 最大的不同，是它把“资金托管”和“流动性授权”拆开了。资金不需要存进池子，但 position 之间会共享同一个钱包余额，因此一个 position 的成交可能影响其他 position 的可成交额度。

这也说明只读取 virtual balance 还不够，必须同时检查钱包余额、allowance、position 状态和后续成交记录，才能判断一个 position 当前是否真正有可用流动性。

【验证结果】

PublicNode Ethereum 主网只读示例验证成功。

05 套利框架 (36 篇)

笔记 053: 2026-08-08 跨链 RWA 套利监控日志

作者: xiaoquanyo | 时间: 2026-08-07 17:00 UTC | 字数: 1144 | 评分: 90

来源: <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

2026-08-08 跨链 RWA 套利监控日志

今天主要复盘了跨链 RWA 套利监控程序连续运行一天的结果。

本次监控按照每 1 分钟检测一次运行,连续运行约 24 小时,共完成约 1,440 轮检测。程序采用自动执行模式,最终完成了 17 笔成交。

运行结果

检测频率: 每 1 分钟一次

检测轮次: 约 1,440 轮

自动成交: 17 笔

毛利润: 5.840147 U

Gas 成本: 0.650197 U

扣除 Gas 后净利润: 5.189950 U

按检测轮次计算的成交频率: 约 1.18%

平均每笔毛利润: 约 0.343538 U

平均每笔 Gas: 约 0.038247 U

平均每笔净利润: 约 0.305291 U

Gas 约占毛利润的 11.13%

从结果看,程序并不是每次检测到价差都会执行,真正完成成交的机会相对有限。17 笔成交能够带来 5.189950 U 的 Gas 后净利润,说明当前筛选逻辑确实过滤掉了不少无法覆盖成本或无法完成执行的表面机会。

Gas 成本目前约占毛利润的 11.13%,还没有高到吞掉大部分收益,但已经是必须持续关注的成本项。后续还需要把跨链费用、DEX 手续费、滑点、失败交易成本以及资金占用成本全部纳入统一账本,确认当前“净利润”是否是真正可落袋的净利润。

这一天的结果也说明,跨链套利的关键不只是发现价差,而是让监控、报价、路径选择和自动执行在有限时间内衔接起来。每分钟检测一次可以覆盖较长时间,但仍可能错过持续时间更短的机会;另一方面,如果检测到的报价已经过期,自动执行也可能出现预期收益和实际收益不一致的情况。

下一步需要逐笔复盘这

17

笔成交,记录源链、目标链、代币、买入路径、卖出路径、报价时间、提交时间、到账时间、实际成交价格、Gas 和最终收益,区分预估收益与链上实际结算结果。

今日目标

1. 完成 17 笔成交的逐笔对账,确认每一笔的真实净利润。
2. 补齐跨链费、协议费、滑点和失败交易成本,避免只按毛利润减 Gas 计算收益。
3. 统计从发现信号到提交交易、跨链到账和最终卖出的完整耗时。
4. 找出收益最高和收益最低的路线,分析它们在流动性、到账速度和成本上的差异。
5. 检查是否存在报价过期、成交滑点过大或目标链卖出深度不足的问题。
6. 在不扩大单笔资金规模的前提下,继续运行监控,优先验证收益的可重复性,而不是单纯追求成交笔数。
7. 为自动执行增加明确的最小净利润、最大滑点、最大跨链等待时间和异常暂停条件。

今天的重点不是让程序执行更多交易,而是确认这 17 笔成交是否都经过完整成本核算,并判断当前收益能否稳定复现。

笔记 054: Boros 规模数据与亲身实测(截至 2026-08 初)

作者: Oxmigueleth | 时间: 2026-08-08 00:55 UTC | 字数: 3084 | 评分: 88

来源: <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Boros 规模数据与亲身实测(截至 2026-08 初)

这一节做两件事:①给出 Boros 的真实体量(判断这个市场值不值得 T+ 进);②Michael 亲自跑了一笔 CrossEx-Boros 4 腿套利,用一手成交数据暴露这套方案的真实摩擦——这是整份 research 最有说服力的部分:不是分析别人,是“我亲手跑了一遍”。

6.1 Boros 体量画像:典型的高周转利率衍生品市场

| 指标 | 数值(2026-08 初) | 解读 |

| :----- | :----- | :----- |

| TVL(锁定抵押品) | 约 \$7.5-7.9M(全在 Arbitrum) | 低,近 30 天 -12% 至 -16% |

| 名义 OI | 约 \$130-152M(7 月中 \$104M → \$151M+) | 快速上升,增长显著 |

| 累计名义成交量 | 约 \$19-21.3B(上线不到一年) | 高周转 |

近 30 天成交	约 \$1.1-1.3B	日成交数千万至上亿
累计协议费用	约 \$720-726K;年化约 \$727-730K	费来自开平仓、结算
市场覆盖	130-170+ 市场(约 30% RWA/商品/股权 funding)	H1 用户数 +约 50%

核心结论:TVL 低但名义 OI/成交量高——这是资金费率套利驱动的高周转市场,不是传统被动 yield 协议。收益体现在各 YU 市场的 Implied APR(跨所 funding 价差实时变动)。对 T+ 的判断:这个市场体量还不小(\$7.9M TVL),但结构和 T+ 目标客群完全重合,且正在增长——是"小而精准、值得早进"的信号,不是"大到不能忽视"。

6.2 谁在贡献仓位(直接接 BD 名单)

公开信息显示当前仓位高度由专业套利者、机构、prop desk 主导,而非散户被动 LP:

主要贡献者:跨平台 funding 套利者(尤其 4 腿固定收益)。官方 H1 更新点名 prop desks、multi-strat funds、结构化产品、合成稳定币相关资金,以及依赖 cash-and-carry 的约 \$30B 级 funding 相关敞口——这个 \$30B 才是 T+ 真正该盯的池子(Boros 的 \$7.9M TVL 只是这个大池子的冰山一角)。

流动性结构:Order Book 是主流流动性(专业 MM + 活跃交易者;存在专门的 boros-mm 开源/生产级引擎,处理多期限曲线、DVO1 库存、保证金复制;白名单 MM 有更优保证金系数);Vaults 是辅助(模拟 Uni V2 式 long YU + 抵押品,规模小、早期约数十万美元,LP 承担 long-biased 隐含利率风险)。

团队未公开点名单一主导 MM;顶级持仓需查 Arbitrum Market Hub 合约。

6.3 亲身实测:一笔 ETH 4 腿套利的真实成交(Michael 自做)

账户: 可用 \$630.80 / 总余额 \$797.40 / uPnL -\$0.41;IM 21%、MM 9%。

仓位: ETH 4 腿,OKX 做多 + Hyperliquid 做空(已对冲),锁定利差 4.38%,到期 2026-08-28(剩 25 天),Fixed APR +9.43%,占用本金 \$350,到期预计 +\$2.00,当前即时 -\$1.00。

腿	方向	名义	现金流	MTM	费用	净
HYPERLIQUID PERP	SHORT	\$2,006	+0.03	-8.84	-0.96	-0.93
OKX PERP	LONG	\$2,005	0	+8.36	-0.68	-0.68
HYPERLIQUID BOROS	SHORT	\$2,006	-0.01	0	-0.07	-0.08
OKX BOROS	LONG	\$2,006	0	+0.47	-0.07	+0.40

6.4 从这笔实测里,亲眼看到的 CrossEx-Boros 方案摩擦(每条都是 T+ 卖点锚)

这份 research 最锋利的部分——不是我推测的痛点,是我自己做单时亲身撞到的:

1. 收益率被摩擦严重侵蚀。锁定利差 4.38%,但兑现成 Fixed APR 只有 9.43%,而到期预计净收益仅 +\$2.00(本金 \$350)。原因全在表里:perp 两腿的 MTM 一正一负基本对冲(\$8.36 vs -\$8.84),但手续费(四腿合计约 -\$1.78)+ 价差磨损把利润吃掉一大半。即时 PnL 已经 -\$1.00——理论利差和到手收益之间的鸿沟,主要就是跨所执行摩擦。T+ 的撮合结算解耦 + 内部流动性(内部永远比外部便宜)想吃掉的正是这块。

2. 两腿名义要各压一份保证金。OKX \$2,005 多 + HL \$2,006 空,虽然方向对冲、净敞口接近零,但在现有结构下两边各自占用保证金(账户 IM 21%)。这正是 T+ cross-margin + MBM 对消要解决的:方向对冲的仓位不该占双份保证金。这一笔就是 MBM "matched 部分零成本"叙事的活教材——我这两条 perp 腿就是 matched,却付了双份 margin 和双份 fee。

3. 对冲状态要人肉盯。界面专门有 hedged / imbalanced 标记和一键补单——说明"保持对冲"是个持续的手动运维负担。T+ 统一账户 + 原生对消应让这是账本自动状态,不是用户责任。

4. 收益薄到对摩擦极度敏感。9.43% APR、\$350 本金、25 天赚 \$2——这个量级下,任何一点执行成本改善都直接放大到净收益。这就是为什么这个客群对"回滚率、fee、价差"这些 T+ 参数极度敏感(呼应尽调单)。他们不是价格敏感,是摩擦敏感。

笔记 055: 共学第4天。今天研读了两份链上地址调研报告,收获很大。

作者: cjdacoin | 时间: 2026-08-08 01:13 UTC | 字数: 499 | 评分: 60

来源: <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

共学第4天。今天研读了两份链上地址调研报告,收获很大。

一是 Aave 清算套利系统(0x8d64):四角色分离——提交EOA只负责发现+签名+付gas,执行合约吃闪电贷(4191 LINK,费0.05%)替借款人还债、折价领取USDC抵押品,再经Bebop PMM RFQ与做市商精确换回还款额,原子闭环。关键认知:链上看到的0.5366 ETH不是利润而是BuilderNet竞价支付,真实净利取决于事后refund,不能把表面转账当利润。净收益=抵押品价值债务闪电贷费换汇成本gas竞价+refund。

二是Robinhood链原子三角套利(0xd712):WETH→中间币→USDG→WETH跨池闭环,单笔净利\$0.17~0.64、gas仅3-6美分;最近20笔9胜11败,但失败只亏约\$0.06 gas——高失败率是廉价试单,期望=胜率×净利败率×失败成本,不赌方向。

启示:提交/执行/归集三分离隔离风险;评估机会要用净期望思维;复现路径应从只读监听→历史重放→paper simulation开始,BuilderNet refund这类不可观测变量按多场景单独处理。

笔记 056: 理解执行成本模型

作者: hello116 | 时间: 2026-08-08 01:22 UTC | 字数: 290 | 评分: 64

来源: <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

理解执行成本模型

学习

建立:

真实收益模型:

Net Profit

=

Gross Arbitrage

-

Gas

-

DEX Fee

-

Price Impact

-

Bribe

-

Failure Cost

-

Capital Cost

实践

建立:

profit calculator

输入:

amountIn

gas

baseFee

pool fee

slippage

输出:

expected EV

笔记 057: 今天第三天，继续啃套利基础。

作者: bxynhs-del | 时间: 2026-08-08 05:32 UTC | 字数: 430 | 评分: 48

来源: <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天第三天，继续啃套利基础。

上午主要在理解一件事：为什么看起来有价差，实际上拿不到钱。把所有成本列出来以后——Gas、手续费、滑点、失败成本、资金占用——才发现自己之前的计算一直漏了“万一这笔失败了，我亏多少”这一项。失败又不是没有成本的，Gas照样扣。

有一个公式让我印象比较深：预期净收益不是“成功率 × 毛收益”，而是要减去“失败概率 × 失败时损失”。听起来很简单，但我之前从来没有这样想过。

下午跑了一下AMM的计算，发现套利规模不是越大越好。投入太少，固定成本（Gas之类的）直接把利润吃掉；投入太多，自己的买入把价格推高了，也亏。两头都有边界，中间才是赚钱的区间。

晚上看了一些关于执行的内容。链上有个细节：拿到交易哈希不等于成功，收到收据也不等于执行成功。要真正确认，得看收据里的status字段。这种细节如果不提前知道，实盘时很容易出问题。

整体感觉：套利比我想象的复杂很多，但复杂不是坏事，说明门槛在那里，认真研究还是有价值的。

笔记 058: Day 4 (8/8) | 成本模型 I：显性成本，一分不漏

作者：yixin | 时间：2026-08-08 05:51 UTC | 字数：997 | 评分：78
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 4 (8/8) | 成本模型 I：显性成本，一分不漏

一句话目标：把「大概要扣点手续费」变成一个逐项可审计的清单。绝大多数纸面套利死在这一天列的项目上。

核心概念

链上一笔套利的完整显性成本清单（逐项，不许省略）：

1. Gas：gas_used × (base_fee + priority_fee) × ETH价格
 - base_fee 随区块拥堵变化（EIP-1559），你必须用预测值而非当前值
 - priority_fee 在竞价场景下不是固定的，是你的出价（Day 11 详述）
 - L2 上还有 L1 data availability 费用，在 blob 定价变化时可能占大头，很多人漏掉这一项
2. 协议费：DEX swap fee (V2 通常 0.30%，V3 分档 0.01/0.05/0.30/1.00%，Curve stable 池常在 0.01–0.04%)、聚合器抽成、桥的协议费
3. 桥成本三件套：协议费 + LP 费（随利用率变化，单向被抽干时可能翻倍）+ 目标链 relayer/执行费
4. CEX 侧（如果有）：maker/taker 费（VIP 等级差异巨大）、提币固定费、法币通道费
5. 资金成本：借贷利率（闪电贷 0.05–0.09%，普通借贷按小时算）、以及机会成本——被占用的资金本可以吃 5% 的稳定币收益，这在长周期策略里是真金白银
6. 税/滑点之外的隐藏项：转账税代币（部分小币种）、rebase 代币的份额变化、代理合约升级风险

必须建立的直觉：

- 盈亏平衡价差（breakeven spread） $S = \text{总成本} / \text{规模}$ 。给每个策略算出它的 S，然后拿 S 去和历史价差分布比（Day 6）。
- 固定成本 vs 比例成本的分界规模： $Q_{\min} = \text{固定成本} / (\text{毛价差率} - \text{比例成本率})$ 。小于 $Q_{\min}$ 一定亏。这个公式应该背下来。
- 失败成本：链上套利失败时 gas 照付。所以真实期望是 $E = p \cdot \text{profit} - (1-p) \cdot \text{gas_fail}$ 。 $p = 0.5$ 时，你需要的价差是简单计算的两倍多。

笔记 059: - 今天把链上套利重新理解成“读取状态 → 按顺序执行状态转换 → 回到同一计价资产 → 扣除全部成本”

作者：Jun Guo | 时间：2026-08-08 06:01 UTC | 字数：257 | 评分：51
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天把链上套利重新理解成“读取状态 → 按顺序执行状态转换 → 回到同一计价资产 → 扣除全部成本”的闭环，静态价差本身不能证明可执行利润。

三角套利必须逐腿传递真实输出，Backrun 只是排在目标交易之后，私有 Bundle 也只改变传播和排序路径，模拟成功不等于最终被区块包含。

Funding 更适合作为拥挤度背景：先按单币种历史做滚动标准化，再做横截面排序，并结合跨交易所 OI 加权值和离散度。OI 只表示未平仓风险规模，方向和去杠杆阶段仍要结合主动成交、价格反应、清算与订单簿判断。

笔记 060: 今天主要学习了 Hermes Agent 在研究流程中的应用，开始思考如何把以前依靠人工搜索、记录和

作者：Oxgen3 | 时间：2026-08-08 08:16 UTC | 字数：883 | 评分：67
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天主要学习了 Hermes Agent 在研究流程中的应用，开始思考如何把以前依靠人工搜索、记录和判断的过程，转化为更加自动化、可重复的工作流。

之前研究一个机会时，通常是：\发现一个异常 → 手动搜索资料 → 查看数据 → 记录结果。

但这种方式效率比较低，而且容易遗漏信息。

通过了解 Hermes Agent，认识到 Agent 可以帮助完成：

信息收集和整理
多来源数据查询
研究任务拆解
工具调用
定期监控和提醒

让一次性的研究过程变成长期运行的系统。

今天重点理解的几个方向：

1. 从一次性观察到可重复查询

以前看到某个机会，例如：

某链出现异常价差
某资产跨链成本降低
某协议流动性变化

可能只是记录一下。

但更好的方式是把它转化成：

固定查询条件
数据监控脚本
定期提醒任务
Agent 自动分析流程

让机会发现从主动寻找变成持续监控。

2. 建立机会证据记录

今天了解到，发现机会不能只记录结果，还需要记录过程。

例如：

时间：

链：

资产：

交易规模：

路由：

Gas 成本：

手续费：

最终收益空间：

为什么失败：

这样以后复盘时，可以判断：

这个机会是真的存在，还是只是偶然情况。

3. 思考 Agent 在套利研究中的作用

结合之前学习的跨链和路由：

未来可以让 Agent 帮助：

监控不同链之间价格差
检查跨链路径变化
对比手续费和 Gas
发现异常情况
自动生成研究报告

把人工研究变成一个持续运行的信息系统。

今日总结

今天最大的收获是理解了，研究机会不仅仅是发现一次机会，而是建立一套可以重复运行的方法。

未来想做套利或者链上研究，需要的不只是判断能力，更需要数据收集、分析和监控体系。

Agent 的价值不是替代研究，而是帮助提高信息处理效率，让自己可以关注更重要的判断环节。

笔记 061: 肥尾分布和杠铃策略

作者：x-tan-dev | 时间：2026-08-08 08:23 UTC | 字数：1043 | 评分：60

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

肥尾分布和杠铃策略

很多套利者没有意识到黑天鹅的危害，他们仍然在压路机前捡钢镚。

一个有趣的例子就是火鸡。

这只火鸡被养在农场里，日复一日，每天傍晚农夫都会准时送来食物与清水。一天天过去，火鸡不断收集历史样本：“每一天我都得到投喂，没有伤害发生。”它用过去上千天安稳的经历，推导未来：日子会永远这样美好，生存是确定无疑的。每多平安度过一天，火鸡对自己安全的信心就更强一分。

对火鸡而言，过去所有的数据，都在证明世界是温和、友善的。它看不到压路机正在缓缓逼近。

直到感恩节到来。

那一天，黑天鹅降临。没有投喂，只有屠刀。过去一千天的全部证据，一瞬间全部作废。漫长的安稳，仅仅是毁灭之前的假象。

很多套利者、做价差交易、高频统计套利的市场参与者，恰恰就是这只火鸡。他们日复一日从市场赚取细碎、稳定的小额收益，就像在压路机前面弯腰捡拾地上的钢镚。每一次交易顺利平仓，每一次拿到确定的套利收益，都会强化他们的信念：这套策略是安全的，历史的价差区间不会被打破。他们依靠历史回测、正态分布模型，计算风险，认定极端崩盘的概率微乎其微。

他们只看见源源不断的钢镚，看不见身后那台隆隆前进的压路机。平时压路机静止不动，满地都是唾手可得的小钱；市场平静的年份，套利策略持续盈利，基金规模扩张，交易员拿到丰厚奖金。所有人把运气当成策略本身的能力。

可压路机不会消失，它只是暂时没有开动。黑天鹅到来的那一刻，价差瞬间撕裂，流动性骤然枯竭，对冲失效，BS模型给出的风险数字全部失效。过去千次都有效的套利关系一夜崩塌。之前所有细碎的盈利，会在短短几天之内，连本带利全部吐回去，甚至直接爆仓出局。

火鸡的悲剧在于：从过去的安宁，无法推导出未来的安宁。样本越长，你对安全的错觉越强，而毁灭到来时打击就越沉重。

压路机前捡钢镚的本质，就是做空尾部风险：换取持续不断的微小收益，承担一次性彻底毁灭的代价。平静时期，全世界都会赞美这套策略高明；灾难来临，没有第二次翻盘的机会。火鸡不会在感恩节之后重新活过来，爆仓的套利者也没有办法重启账户。

塔勒布反复提醒：真正的风险，从来不是历史数据里看得见的波动，而是那些样本中还从未出现过的毁灭性事件。历史不会告诉你，什么时候是感恩节。

与之对照的，就是杠铃策略的思维：不要去压路机前捡钢镚。要么站到压路机远远碰不到的安全地带，拿出极小一部分筹码，去赌压路机突然开动的那一刻——做黑天鹅的买家，而不是黑天鹅的祭品。

笔记 062: 【Day 4 打卡】2026/8/8

作者：ed11555 | 时间：2026-08-08 08:32 UTC | 字数：1290 | 评分：88

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

【Day 4 打卡】2026/8/8

今日任务：成本模型入门 + 真实套利案例分析

完成内容

1. 成本模型入门

- 理解套利成本构成：Gas 费用、交易费、滑点、跨链桥费
- 学习计算模型：
- 毛利 = 价差 × 本金
- 净利 = 毛利 - (Gas + 交易费 + 滑点 + 桥费)
- 盈亏平衡点计算

2. 获取真实 DEX 价格数据

使用 DexScreener API 获取 4 条链的 WETH/USDC 价格：

| 链 | 价格 | 流动性 | DEX |

|---|---|---|---|

| Ethereum | \$1,867.99 | \$186.6M | Uniswap V3 |

| Arbitrum | \$1,868.50 | \$50.0M | Uniswap V3 |

| Optimism | \$1,867.80 | \$30.0M | Uniswap V3 |

| Base | \$1,868.20 | \$20.0M | BaseSwap |

3. 真实套利案例分析

- 价差：\$0.70 (0.0375%)
- 套利方向：Optimism → Arbitrum
- 毛利：\$3.75 (\$10,000 本金)
- 总成本：\$71.05
- 净利：-\$67.30

4. 成本估算

| 成本項目 | 金額 | 占比 |

|-----|-----|-----|

| Gas (買入 + 賣出 + 橋接) | \$30.05 | 42% |

| 橋接費 (0.3%) | \$30.00 | 42% |

| DEX 交易費 (0.1%) | \$10.00 | 14% |

| 滑點 (0.01%) | \$1.00 | 1% |

關鍵發現

1. 價差太小：真實價差 0.0375% < 需要 0.4%，差距 11.7 倍
2. Gas 是主要障礙：以太坊主網 Gas \$25/筆，佔成本 42%
3. 小資金不現實：<\$100k 本金幾乎必然虧損
4. 專業玩家優勢：極低 Gas 成本 + <10ms 執行 + 大額資金

敏感性分析 (\$10,000 本金)

- 價差 0.01% → 淨利 -\$70.05
- 價差 0.10% → 淨利 -\$61.05
- 價差 0.50% → 淨利 -\$21.05
- 價差 1.00% → 淨利 +\$28.95

學習成果

1. 理解成本模型：Gas + 交易費 + 滑點 + 橋接費
2. 獲取實時數據：使用 DexScreener API
3. 計算真實案例：驗證理論模型
4. 認識現實差距：價差 <0.01% vs 需要 >0.4%

明日計劃 (Day 5)

- 復盤 Week 1 學習內容
- 繪製套利知識思維導圖
- 總結關鍵洞察和疑問

學習狀態：進行中

今日學習時長：約 1.5 小時

笔记 063: D4 打卡 (8/8) · 三层套利认知闭环 + Solana 主线双实现 + 武器库成型

作者：Paxon Qiao 乔鹏军 | 时间：2026-08-08 08:40 UTC | 字数：1996 | 评分：90

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

D4 打卡 (8/8) · 三层套利认知闭环 + Solana 主线双实现 + 武器库成型

【今日主线 1】Solana D4：DEX 价差监控 (Python + Rust 双实现)

- solana_dex_spread_monitor.py (只读双数据源)：Helius RPC 直读 Raydium SOL-USDC vault 恒定乘积定价 + Jupiter Swap API 多金额采样 (0.1/1/10/100 SOL)
- 实测：Raydium 73.8107 vs Jupiter 73.9365 $\approx$ 17bps (Python)；Rust spread 子命令 73.776 vs 73.929 = 20.7bps 交叉验证一致
- 发现：Jupiter 100 SOL 会拆 4 池多跳路由 (BisonFi/Aquifer/HumidiFi/AlphaQ)，price 必须按各腿 inAmount/outAmount 算
- 意义：同链跨 DEX 价差 = 第 2 周 Jito 原子套利的数据基础

【今日主线 2】token metadata 读取 (PDA 官方库教训)

- 推导 metadataAccount (seed=metadata+programID+mint) → RPC 读账户 → 解析 bytes
- 踩坑：自实现 base58+sha256+on-curve 简化检查推导结果错误 (4tgHlm vs 正确 5x38Kp)；PDA bump 检查是 ed25519 曲线点验证，必须用官方库 (solana-py find_program_address)
- 实测 USDC (name=USD Coin) / SOL (Wrapped SOL) 均读到，与 Rust 权威一致

【今日主线 3】Dune 清算数据分析 (D4 遗留)

- Dune 仪表盘被 Cloudflare 403 → DeFi Sphere 公开 API 等价实现 (dune_liquidation_analysis.py)
- 90 天全量：17,071 笔 / \$3.86 亿抵押品 / \$1,771 万奖励 (4.6%)
- 关键发现：morpho 占 59% (清算主战场)、Base 占 62% (清算密度最高)、单笔最大奖励 \$74,183
- 与清算哨兵互补：哨兵实时 24h + 本分析全量历史 90 天基线 → 哨兵应加 morpho 优先过滤

【认知闭环】Greenfield 以太坊区块构建供应链 (5 段逐段解读)

- 四层：订单流层 (纳入市场 vs 位置市场) → 路由层 (公共池 vs 私有 RPC/OFA) → 构建者层 (独有订单流=胜负手) → 中继 (利润=真实价值-次优出价)

- MEV 根源：对共享状态的优先访问永远有经济价值；理性套利者愿付≈全部可捕获利润
- 与 Robinhood 0x1521 机器人实证咬合：68 万笔探测、74643 gas 失败成本=位置市场入场券
- 盲套利架构讨论：快链（RHC/Solana）来不及链下模拟再上链 → 检测流动性变动 → 上链 → 合约内 staticcall 算利润 → 无利回滚
- 老孙监控模块：OI 异动（价格不变+OI 放大=庄家入场）、负费率预测模型、美股费率——「全自动套利 100% 踩坑」印证人扣扳机哲学

【广度阅读闭环】D4/D5 三篇

- 收过路费：已被回测证伪（高费率池吃不到流量，LVR 逆向选择吃掉 fee；对手盘是知情流量）
- 溢价买 LP：idea 未验证（四个真问题：溢价持续性/参考价是不是底/退出流动性/fee APR 外推）
- 资金费率套利：5 大风险（逼空强平/ADL/抵押品折价/执行风险/基差即成本）；高 funding+高 basis 分位=接盘信号
- 共同方法论：吃费率的真对手是逆向选择成本（LVR/基差/知情流量），按笔算账不能按 APR 外推

【工程产出】

- GitHub 公开仓库上线：github.com/qiaopengjun5162/onchain-arbitrage-colearn（12 commits + Actions 每日快照 + 8 topics）
- 投研八步模板 templates/project-research.md（含美股一手数据源）
- 资金费率哨兵 v2（Z-score+OI+价费背离）、币股时钟差哨兵（6 信号 30-100bps）、跨所价差 v0（主流币无机结论）
- 安全加固：3 处 API key 硬编码 → 环境变量；.gitignore 全覆盖

【下一步】funding_sentinel_v2 加 basis 分位维度；周一 21:30 验证币股闭市漂移 → 开盘收敛；第二周 Jito 原子套利

笔记 064: 链上套利残酷共学 · Day 3 心得

作者：kako | 时间：2026-08-08 09:10 UTC | 字数：442 | 评分：61

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

链上套利残酷共学 · Day 3 心得

今日主题：执行现实学——净收益

前两天的“价差机会”在今天被彻底打回原形。屏幕上的价差再诱人，落到钱包里的才是真利润。今天的核心收获是一道“套利者必算公式”：

净收益 = 价差 - 两端Gas - 桥费 - 滑点 - 协议费 - 失败损失期望

逐项拆解后才意识到，每个成本项都可能在特定场景下反客为主：

两端Gas：以太坊主网拥堵时，一笔交易几百美元Gas费并不罕见，直接吃掉中小价差的所有利润；

桥费：跨链套利的“隐形杀手”，固定费用+按比例收费，对中小资金极不友好；

滑点：昨天的知识点落地了——流动性越浅，滑点越大，价差瞬间变“纸面富贵”；

协议费：各DEX的费率不同（0.05%~1%），三角套利转三圈就是三次扣费；

失败损失期望：最容易被忽略的一项——交易失败（滑点超限、被抢跑、跨链超时）仍要付Gas，多次失败的成本累积相当可观。

笔记 065: 这是我的所思所想

作者：coreentro | 时间：2026-08-08 09:31 UTC | 字数：368 | 评分：62

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

这是我的所思所想

价差套利是什么

价差套利其实特别简单：

同一个东西，在两个地方价格不一样，就便宜的地方买，贵的地方卖。

比如：

A 交易所 BTC：100,000 美元\

B 交易所 BTC：100,300 美元

看起来有 300 美元价差，也就是 0.3%。

但这 0.3% 不是你真正赚到的钱。

因为实际操作还要扣：

手续费 + 滑点 + 转账费 + 操作过程中价格变化

假设最后这些成本一共是 0.2%，那真正剩下的只有：

0.3% - 0.2% = 0.1%

如果成本达到 0.35%，那虽然屏幕上有价差，实际做进去反而亏钱。

所以以后看到任何套利机会，第一反应不要是：

“有价差，能赚。”

而应该是：

“扣完所有成本，还剩多少？”

笔记 066: 今天收住了“独立发现价差”这一阶段，也迈出了实时读取链上数据的第一步。

作者：bai · 小白 | 时间：2026-08-08 09:58 UTC | 字数：2476 | 评分：86

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天收住了“独立发现价差”这一阶段，也迈出了实时读取链上数据的第一步。

阶段完成的含义很明确。现在面对一份混有错误链、错误区块、同名异址 Token、不同交易对和浅池的数据，你已经能先清理比较对象，再判断低价池和高价池，随后用费用、容量和完整模拟逐层淘汰候选。它还不代表已经能捕获套利，更不代表真实净利润已经得到证明。

从噪声快照中找到可比较市场

在十个池子的综合快照里，你独立保留了

A、B、G、H、I，并枚举出九条低价池买入、高价池卖出的候选方向。筛选时使用的条件已经从名称推进到链、区块号、Token 合约地址、交易对和最低流动性。 Token

你先押中 G → I，随后说出了更关键的边界。执行这条路径以后，G 池储备已经改变，表中原有的 G → B 利润不能直接加到 G → I 上。剩余候选需要使用更新后的储备重新模拟。这说明你已经开始把市场看成会被交易改变的状态，而非一张可以反复使用的静态价格表。

独立完成一次真实快照审计

在 Ethereum 主网区块 25,708,779 的四组真实 v2 池中，你正确完成了分组和方向判断。

1. WETH 和 USDT 组选择 A → B
2. WETH 和 DAI 组选择 C → D
3. SHIB 和 WETH 组选择 H → G
4. WBTC 和 WETH 组选择 E → F

统一用买入价作分母以后，四条路径的初始价差率约为 0.0524%、0.0346%、0.8064% 和 0.0712%。在每个池子收取 0.30% 手续费的假设下，两次兑换需要跨过的费用门槛约为 0.6027%。四条路径里只有 H → G 值得进入完整池内模拟。

这里还纠正了一次费用计数。两池往返路径只有两次兑换，因此支付两次池费。回到起始资产是第二次兑换的结果，不会凭空多出第三次兑换。

H → G 在当前池内模型中的近似最佳投入约为 0.000461 WETH，池内计算利润约为 0.000000466 WETH。这个结果证明它通过了当前模型，利润却非常小，而且池很浅。Gas、竞争、延迟、状态变化和交易能否进入目标区块尚未计算，所以它仍不能称为真实可捕获利润。 H

今天把机会判断稳定地分成了四层。

1. 价格候选只说明低买高卖方向存在
2. 费用初筛通过只说明初始价差大于基础池费
3. 池内正收益说明完整兑换在固定储备模型中为正
4. 可捕获净利润还要扣除执行成本并处理竞争和状态变化

实时数据最重要的一条边界

新区块事件只负责通知程序开始读取。用于比较的是所有池子在同一个区块高度下的储备快照。

latest 可以用于确认当前链头。程序得到区块 N 后，本轮所有 token0、token1 和 getReserves 调用都要显式固定到 blockTag N。如果 A 池来自区块 500，B 池来自区块 501，那么两者拼出的价差可能从未在链上同时存在。

当前实操还增加了区块哈希复核。程序在读取池子前后各确认一次区块的哈希，哈希变化时丢弃整份快照。这使同区块读取又多了一层应对短期链重组的保护。 N

第一条真实数据管道已经跑通

今天完成了一条只读的最小纵向切片。程序监听 Ethereum 主网新区块，在同一区块读取 Uniswap v2 和 SushiSwap v2 的 USDC/WETH 储备。它不需要钱包、私钥、签名、合约部署或交易发送。

你亲自运行脚本并拿到了区块 25,709,349 和 25,709,350 的连续快照。两份快照中的两个池子储备相同，这并不异常，只能说明这两个池子在相邻区块之间没有发生改变。脚本已经成功取得真实链上状态，下一步才轮到机会评估器判断价格和路径。

第一次真实读取还暴露了一个很有价值的问题。RPC 调用已经成功，代码却把 ABI 解码结果的字段名想错了，导致储备值为空。改用稳定的元组位置 reserves[0] 和 reserves[1] 后，真实读取通过。以后验证链上数据不能停在“请求成功”，还要检查单位、字段含义、数量级和可读结果。

原始 JSON 对学习判断帮助很小，所以命令行输出已经改为只展示当前需要的信息，包括区块号、北京时间、同区块校验、两池可读储备、储备比值参考单价，以及相对上一块是否变化。完整地址和原始整数仍保留在程序内部用于校验。这里显示的参考单价只是储备比值，不能替代真实成交价或利润模拟。

学习环境也完成了分区

项目现在分成三个边界清楚的目录。original-project 保存最初项目，learning 保存课程、学习记录和总结，practice 保存从零搭建的实操脚本。原始项目文件已经与 Git 中的原版本逐一核对，实操脚本也有独立依赖和测试。

这个整理动作服务于后续学习。原项目可以随时作为证据和参照，练习代码可以逐步改动，学习材料也不会再夹在生产代码旁边。

截至今天能够独立完成的事

从混合数据中筛出可比较的池子
按同一单位判断低价池和高价池
枚举候选方向并做费用数量级初筛
识别浅池容量和共享池状态变化
区分价格候选、池内正收益和真实可捕获利润
解释实时比较为什么必须固定同一区块
运行只读监控脚本并检查输出是否符合数据语义

目前仍未完成的事

还没有监控全市场，目前只有两个已知 USDC/WETH v2 池
还没有把同区块快照接入独立的机会评估器
还没有搜索真实最佳投入量并输出池内利润
还没有计算 Gas、竞争、延迟和状态变化后的净利润
还没有钱包、签名、交易构造或套利捕获功能

下一次学习只需要接上一件事。让机会评估器接收一份固定快照，先判断两个池子的参考价格和候选方向，同时禁止它偷偷读取实时链上状态。

笔记 067: 调研 RWA Perpetual Arbitrage :

作者 : 0x2077 | 时间 : 2026-08-08 10:08 UTC | 字数 : 279 | 评分 : 56

来源 : <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

调研 RWA Perpetual Arbitrage :

链上股票永续合约套利是利用传统股票、Tokenized Stock、ADR、ETF、链上永续合约之间的价格偏差，通过多市场对冲获取收益的一类套利策略。

核心逻辑：

同一底层资产，在不同金融体系中形成多个价格，由于交易时间、流动性、参与者、结算规则不同，产生短期或长期价差。

典型资产：

美股（NVDA、TSLA、AAPL）
亚洲股票（SK Hynix、三星等）
港股、美股 ADR
股票 Token
链上永续合约（Perpetual Futures）

笔记 068: 一、套利的本质：寻找市场“不平衡”

作者 : laofei666 | 时间 : 2026-08-08 11:13 UTC | 字数 : 226 | 评分 : 50

来源 : <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

一、套利的本质：寻找市场“不平衡”

很多人认为套利就是：

找两个价格不一样的地方，然后赚差价。

这是最基础的理解。

更深层：

套利是在市场不同参与者、不同交易机制、不同信息速度之间，寻找短暂的不平衡，并通过系统化方式获取收益。

市场为什么会出现机会？

因为：

信息传播有时间差

流动性存在差异

交易规则不同

资金成本不同

用户行为不同

优秀套利者不是预测市场。

而是等待市场出现错误。

笔记 069: 【Day 4 | 2026-08-08】 M1 屏幕价差 vs 净收益

作者：OxChainWorld | 时间：2026-08-08 11:19 UTC | 字数：713 | 评分：82

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

【Day 4 | 2026-08-08】 M1 屏幕价差 vs 净收益

投入：110 分钟

学了：

- 把 Day2 分类收成一页《个人套利地图》
- 用硬公式做 3 次假想成交：毛价差 - Gas/费/冲击/桥/时间

做了：

- 1) 同链 DEX 间：假想 18 bps / \$10k → 扣双边费+冲击后约 -\$2 起，聚合费口径下更亏
- 2) 跨链 ETH→Arb USDC \$10k：假想 Arb 贵 15 bps (\$15)，LI.FI 路径缺口约 0.27% (~\$27) + Gas → 不成立
- 3) LINK 假想 40 bps：1 LINK 被 Gas 杀死；50 LINK 被费/缺口吃光 → 两档都不成立

结果：

- 产出：01-arbitrage-map.md + 01-cost-breakdowns.md
- 关键发现：机会最常死在「费率地板 > 价差天花板」、跨链延迟、以及规模选错导致的死因切换
- M1 验收句：屏幕价差 ≠ 可执行净收益

失败/不确定：

- 三笔全是「不成立」（有效失败）
- 还没抓到一笔扣完成本仍为正的候选（符合现阶段预期）

证据：

- evidence/d4-1-samechain-10k-usdc-weth.json
- evidence/d4-2-cross-eth-arb-10k-usdc.json
- evidence/d4-3-link-usdc-50.json / d4-3b-link-usdc-1.json

净收益判断：三笔均为「不成立」

卡点：同链「真·两池价差」还缺浏览器双边截图，今日用假想 bps + 真实执行成本地板验算

笔记 070: 永续合约对永续合约

作者：xwzp | 时间：2026-08-08 11:25 UTC | 字数：539 | 评分：54

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

永续合约对永续合约

这种套利的本质，是同一个币在不同交易所里，永续价格和资金费率不一样。一边做多，一边做空，方向风险抵消，赚中间的费率差或者价差回归。

真正做起来，需要两边提前放好保证金，同时开仓，还要算手续费、滑点和真实结算的资金费率。

最容易死在一边先爆仓。虽然整体没有方向敞口，但两边的钱不互通，另一边赚再多也救不过来。

现货对期货或永续合约

这种套利就是拿着现货，同时做空更贵的期货或者永续。交割合约到期会向现货收敛，赚的是基差；永续没有到期日，主要赚资金费。

真正执行需要买到现货，并开出等量空单。除此之外，还得算资金占用、保证金、手续费和借币成本。

最容易死在“最后会收敛，但是中途扛不住”。基差继续扩大，期货空单可能先被清算。永续的资金费率也可能突然反过来，从收钱变成付钱。

清算套利

链上借贷里，抵押品跌太多，仓位就会进入清算。清算人替用户还债，然后以折扣拿走他的抵押品。这个折扣就是清算奖励。

真正执行一般是闪电贷借钱、偿还债务、拿到抵押品、卖掉抵押品、归还闪电贷。整条路径最好在一笔交易里完成。

最容易死在把清算奖励当成利润。Gas、MEV

竞价和卖出滑点都会吃掉奖励。很多时候不是抢不到，而是好不容易抢到了，算完反而亏钱。

笔记 071: 【Day 8 打卡 · 2026-08-08】用 Hermes 构建研究与信号 workflow

作者：nerd614 | 时间：2026-08-08 11:38 UTC | 字数：423 | 评分：49
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

【Day 8 打卡 · 2026-08-08】用 Hermes 构建研究与信号 workflow

今天按课程推进到第 3 部分「用 Hermes 构建研究与信号 workflow」，把前期的单次观察固化为可持续的查询与监控机制。

1. 工具整合：使用 Hermes Agent 辅助信息发现、研究拆解、工具调用、数据整理和持续监控，把「看到价差」的一次性动作，转成可重复查询、告警、脚本或 Agent workflow。

2. 证据记录：为候选机会建立标准化的证据字段——时间、链、资产、规模、路径、成本与失败原因，让每一次观察可回溯、可比较。

3. 信号化：把 LI.FI 报价 / DEX 流动性数据接入监控，设定触发条件（价差阈值、路由可行性、净收益为正），只在满足条件时告警。

4. 下一步计划：继续迭代信号规则，减少噪音，逐步收敛出值得深挖的策略候选，并为第 4 部分「从假设走向策略」做准备。

今天聚焦 workflow 与信号机制层面，把「手动看价差」升级为「系统盯信号」。

笔记 072: 2026年8月8日 | 链上套利学习笔记

作者：WikiToTheMoon | 时间：2026-08-08 12:20 UTC | 字数：367 | 评分：56
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

2026年8月8日 | 链上套利学习笔记

今天学习了链上套利中一个基础概念：

发现价格差异，不代表一定存在套利机会。

最开始理解套利时，容易认为：

一个地方价格低，另一个地方价格高，买入再卖出，就可以获得收益。

但实际执行过程中，还需要考虑交易成本。

例如：

Gas 费用；

交易手续费；

滑点；

跨链费用；

价格变化带来的影响；

交易执行失败风险。

因此，真正需要关注的是：

实际净收益 = 价差收益 - 执行成本。

链上套利的第一步不是寻找价格差，而是判断这个价格差是否能够在真实交易环境中执行，并且扣除所有成本后仍然具有收益。

这让我认识到，套利本质上不仅是发现市场差异，更重要的是理解交易路径、成本和风险。

下一步继续学习不同套利模式，以及它们产生价差的原因。

笔记 073: 今日学习心得：从工具报价到真实 Swap 交易还原

作者：Chic | 时间：2026-08-08 12:50 UTC | 字数：1247 | 评分：60

今日学习心得：从工具报价到真实 Swap 交易还原

今天的学习重点从“理解套利类型”推进到了“如何看工具和真实链上交易”。相比之前只知道套利大概是买低卖高，今天更清楚地意识到，真正的链上套利研究需要把报价、路径、成本和实际交易结果拆开来。

首先学习了 Quote 和 Route。Quote 可以理解报价，核心是看预计收到多少以及最少能收到多少；Route 是路径，说明一笔跨链或兑换交易会经过哪些桥、DEX 和中间资产。这里最重要的收获是不能只看 toAmount，而要优先看 toAmountMin。因为 toAmountMin 才更接近保守情况下的实际结果，如果用 toAmount 算出来已经是负收益，就不应该继续假设这个机会能执行。

接着学习了 Quote 里的成本结构，包括 gasCosts、feeCosts、滑点和执行时间。以前可能会只关注表面收益，比如预计多收到多少资产，但现在会先把 Gas、桥费、手续费、滑点、延迟风险都扣掉。今天的练习里也发现，有些机会按预计收到计算好像有利润，但按最少收到计算就变成亏损，这说明安全边际很重要。

然后学习了套利观察表的做法。观察表的价值不是为了证明某个机会能赚钱，而是帮助自己系统性排除不值得做的机会。一个机会需要记录时间、类型、源链/目标链、起始资产、目标资产、投入金额、toAmount、toAmountMin、Gas、费用、延迟风险、净收益和结论。这样可以避免只凭感觉判断，也能慢慢积累对不同链、不同 Route、不同资产成本结构的认识。

最后通过 Etherscan 看了几笔 Uniswap Universal Router 相关交易。这里收获很大：不是所有调用 Router 的交易都是成功 Swap。有的交易会失败，比如因为实际收到数量低于最小接收数量，触发滑点保护；有的交易会 Pending，可能是因为 Gas Price 太低，迟迟没有被打包。这让我理解了失败交易成本和确认速度对套利的重要性。

还看了一笔成功 Swap。它的 Value 是 0 ETH，但 ERC-20 Tokens Transferred 里显示了真实的代币流向：用户付出了 GULD，收到了 GROW。这让我明白，链上 Swap 不能只看 Value，Value 只代表原生 ETH 转账；真正的 ERC-20 换币结果要看 Token Transfers。判断一笔 Swap，需要看 From、Interacted With/To、Status、Token Transfers 和 Transaction Fee。

今天最大的收获是：我开始从“看价格”转向“看路径和结果”。一个链上机会不能只看表面价差，而要看最终收到数量、最少收到数量、路径成本、Gas、滑点、执行状态和失败风险。后续学习中，我需要继续练习用 Etherscan 还原更多真实交易，并开始了解 The Graph 这类工具，学习如何批量查询历史数据。

笔记 074: Day 4 打卡：USDL 稳定币赎回套利案例

作者：tobin | 时间：2026-08-08 13:25 UTC | 字数：5989 | 评分：90

来源：https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205

Day 4 打卡：USDL 稳定币赎回套利案例

一句话结论

这不是普通的 DEX A 买、DEX B 卖，也不是单笔 Flash Loan 套利，而是一组跨多笔交易执行的稳定币脱锚套利：

BSC-USD

- Kyber 链上限价单折价购买 USDL
- TroveManager 按预言机面值赎回 WBNB
- DEX 卖出 WBNB
- BSC-USD

核心机会来自 USDL 的链上买入价显著低于其协议赎回净值。

关键交易与合约

| 类型 | 地址或交易 |

|---|---|

分析对象	Redeem	Collateral
(https://bscscan.com/tx/0x809b7ac98b3e2567262808ac6d629b5d8b7ca5e3f0f11ebcad16d7839be95fff)		
前序买入	BSC-USD →	USDL
(https://bscscan.com/tx/0xe048850921a8cc3ee907996535e0aa8d057e1a063a43806db1a3b56db86bdc07)		
最终卖出	WBNB →	BSC-USD
(https://bscscan.com/tx/0x60edb499426940db753151df8e6683792e05b5a44210adb4269d3a702699b8c4)		
操作者 0xccdf3a72ce978d7e501c86408f15897278608117		
TroveManager	0x15102b7579ae3b913b0cbb2ede791fc58c528195	
(https://bscscan.com/address/0x15102b7579ae3b913b0cbb2ede791fc58c528195#code)		
USDL 0xd295f4b58d159167db247de06673169425b50ef2		
BSC-USD 0x55d398326f99059ff775485246999027b3197955		
WBNB 0xbb4cdd9c9bd36b01bd1c9aebf2de08d9173bc095c		

1. 前序交易：折价买入 USDL

操作者通过 KyberSwap Meta Aggregation Router 成交链上限价单：

```
| 项目 | 数量 |  
|---|---|  
| 支出 | 487.96 BSC-USD |  
| 得到 | 540.556109449429489309 USDL |  
| 平均成本 | 0.9027 BSC-USD/USDL |
```

订单内部资金分配：

```
| 去向 | BSC-USD |  
|---|---|  
| Maker 实收 | 486.500498504486540378 |  
| Kyber 费用 | 1.459501495513459622 |  
| 用户总支出 | 487.96 |
```

这一步不是 AMM constant-product Swap，而是 Kyber Double Signatures Limit Order。套利者以约 0.9027 美元买入目标为 1 美元的 USDL。

2. 分析对象：调用 redeemCollateral

交易直接调用 TroveManager：

```
redeemCollateral(  
540.556109449429489309 USDL,  
firstRedemptionHint,  
upperPartialRedemptionHint,  
lowerPartialRedemptionHint,  
partialRedemptionHintNICR,  
0,  
5%  
)
```

关键参数：

赎回数量：540.556109449429489309 USDL。

maxIterations=0：该 Liquity 风格实现中表示忽略迭代上限。

maxFeePercentage=5%：实际赎回费超过 5% 时交易回滚。

Redemption Hints：帮助合约从抵押率排序列表中定位 Trove，并在部分赎回后重新插入。

3. 赎回计算

协议预言机价格：

1 WBNB = 567.064 USD

销毁 540.556109 USDL 对应的赎回前抵押品：

```
$$  
grossWBNB  
=  
rac{540.556109449429489309}{567.064}  
=  
0.953254146709065448  
$$
```

事件记录的动态 baseRate 约为：

3.0903299701871556%

合约赎回费为：

```
redemptionFeeRate  
= 0.5% 固定下限 + baseRate  
= 3.5903299701871556%
```

实际费用：

```
$$  
fee  
=
```

0.953254146709065448
imes 3.5903299701871556%
=
0.034224969321347414 WBNB
\$\$

用户净收到：

\$\$
netWBNB
=
0.953254146709065448
-
0.034224969321347414
=
0.919029177387718034
\$\$

资金和状态变化：

用户销毁 540.556109 USDL
→ 目标 Trove 债务减少 540.556109 USDL
→ Trove 抵押品减少 0.953254147 WBNB
→ 0.034224969 WBNB 发送到协议奖励合约
→ 0.919029177 WBNB 发送给用户
→ Trove 按新的抵押率重新插入 SortedTrove

这不是清算。借款人的 Trove 只是同时减少一部分债务和抵押品，剩余仓位继续存在。

4. 卖出 WBNB

操作者最后把多轮赎回得到的 WBNB 合并，通过 DZap/聚合路由卖回 BSC-USD：

项目	数量
卖出	1.611472340712216962 WBNB
得到	915.101063130955368760 BSC-USD
实际平均价格	567.866441 BSC-USD/WBNB

按照这个最终成交均价，本轮 0.919029177 WBNB 价值约：

\$\$
0.919029177387718034
imes567.866441
approx521.885828 BSC ext{-}USD
\$\$

对应本轮毛利润：

\$\$
521.885828-487.96
approx33.925828 BSC ext{-}USD
\$\$

毛收益率约为

6.95%。最后一笔卖出聚合了多轮赎回所得，因此这是按最终平均成交价做的利润归因，而不是单笔事件直接记录的独立利润。

5. 整个批次

该地址在约 20 分钟内反复执行四组买入和赎回：

操作	BSC-USD 变化	USDL 变化	WBNB 变化
买入 1	-10	+11.077877479	0
赎回 1	0	-11	+0.019318204
买入 2	-100	+110.778774787	0
赎回 2	0	-110.856652265	+0.193616380
买入 3	-250	+276.946936967	0

赎回 3	0	-276.946936967	+0.479181296
买入 4	-487.96	+540.556109449	0
赎回 4	0	-540.556109449	+0.919029177

汇总：

项目	数值
买入 USDL 总成本	847.96 BSC-USD
最终卖出所得	915.101063131 BSC-USD
整个序列 Gas	0.00015045395 BNB
额外消耗的原有 WBNB	0.000327283525 WBNB
按最终成交价折算的净利润	约 66.87 BSC-USD
净收益率	约 7.89%

净利润已按最终 WBNB 成交均价扣除额外使用的原有 WBNB和 10 笔交易 Gas，但不包含更早已经完成的 Token 授权成本。

6. 为什么属于套利

套利判断条件：

\$\$

USDL买入成本

<

USDL协议赎回后的可卖出净值

\$\$

本例：

项目	数值
USDL 买入价	0.9027 BSC-USD
扣除赎回费后的协议面值	约 0.9641 USD
最终 WBNB 卖出价格	567.866441 BSC-USD/WBNB

价差来源不是两个 AMM Pool，而是：

Kyber 限价单市场给出折价 USDL；

TroveManager 仍按照预言机面值执行赎回；

扣除 3.59033% 动态赎回费后仍有剩余空间。

7. 为什么不是原子套利

完整路径被拆成多笔交易：

买入 USDL

→ 等待打包

→ 赎回 WBNB

→ 等待打包

→ 卖出 WBNB

因此无法像 Flash Loan 套利一样做到整条路径要么成功、要么全部回滚。

主要风险：

买入后 Redemption Hint 可能过期。

baseRate 会随赎回变化，后续赎回费可能升高。

maxFeePercentage 只保护赎回费，不保护整条路径利润。

协议预言机价格和 WBNB 市场成交价可能变化。

赎回可能需要遍历多个 Trove，金额过大可能消耗过多 Gas。

中间持有 USDL 或 WBNB 时存在价格与流动性风险。

其他套利者可以抢先购买折价 USDL 或先执行赎回。

8. 对套利扫描器的启发

扫描器不能只扫描 AMM Pool。协议的铸造、赎回、清算和 RFQ/Intent 也可以构成报价来源。

该策略的利润模型：

$usdlOut = kyberOrRFQQuote(bscUsdIn)$

```
grossWBNB =  
usdlOut / protocolOraclePrice  
  
netWBNB =  
grossWBNB × (1 - redemptionFeeRate)  
  
bscUsdBack =  
dexQuote(netWBNB)  
  
netProfit =  
bscUsdBack  
- bscUsdIn  
- gasCost  
- safetyBuffer
```

需要监控：

USDL 在 AMM、聚合器和 RFQ/限价单市场的买入报价。

TroveManager 的 getRedemptionRateWithDecay。

协议预言机价格与更新时间。

SortedTrove、Redemption Hints 和可赎回容量。

WBNB 的真实退出报价，而不是只使用预言机价格。

买入、赎回、卖出三笔交易的 Gas 和状态变化。

报价到交易上链之间的 MEV、漏块和重组风险。

9. 阅读交易日志的方法

BscScan 显示的日志编号 143~159 是整个区块的 logIndex，不代表该交易执行了 159 个步骤。这笔赎回交易的 Receipt 实际只有 17 条日志。

分析赎回交易时，最关键的是：

1. Redemption：实际销毁数量、赎回抵押品、费用。
2. USDL Transfer 到零地址：确认稳定币被销毁。
3. WBNB Transfer 到用户：确认净到账。
4. BaseRateUpdated：确认后动态赎回费。
5. TroveUpdated：确认目标借款仓位的债务和抵押品变化。

学习结论

套利机会不只存在于两个 DEX，也可能存在于市场价格与协议赎回机制之间。

单看一笔交易无法判断利润，必须追踪前序资产来源和后续退出交易。

预言机价格不是最终成交价格；最终利润必须使用真实退出报价。

动态费率、协议内部排序状态和 Redemption Hint 都应该进入模拟器。

这个案例本质上是异构协议路径：RFQ/限价单 → 借贷协议赎回 → 集中流动性 DEX。

相关笔记

以太坊交易：从 QuickNode 到最终确认

资料

目标赎回交易 (<https://bscscan.com/tx/0x809b7ac98b3e2567262808ac6d629b5d8b7ca5e3f0f11ebcad16d7839be95fff>)

前序	USDL	买入交易
----	------	------

(<https://bscscan.com/tx/0xe048850921a8cc3ee907996535e0aa8d057e1a063a43806db1a3b56db86bdc07>)

最终	WBNB	卖出交易
----	------	------

(<https://bscscan.com/tx/0x60edb499426940db753151df8e6683792e05b5a44210adb4269d3a702699b8c4>)

TroveManager 已验证源码 (<https://bscscan.com/address/0x15102b7579ae3b913b0cbb2ede791fc58c528195#code>)

笔记 075: Day 4 | 从报价到可执行：拆解一笔 DEX 套利交易的真实利润

作者：zhanglianzhong | 时间：2026-08-08 13:36 UTC | 字数：856 | 评分：63

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 4 | 从报价到可执行：拆解一笔 DEX 套利交易的真实利润

今日目标

把“看到价差”和“赚到价差”之间的距离量清楚。今天重点研究同链 DEX 间套利的完整成本拆解，并结合 LI.FI 的 Quote/Route 结构理解报价数据的正确读法。

学到了什么

1. $x \cdot y = k$ 下的滑点不是线性的。拿一个 100 ETH / 200,000 USDC 的池子算了一笔账：买 1 ETH，池子剩 99 ETH 时 USDC 需要 $\approx 202,020.2$ ，即实际成交价约 2,020.2 USDC/ETH，比屏幕价 2,000 高出约 1%。也就是说 1% 的池深位移吃掉 1% 的价差——小额价差机会在 AMM 里基本会被冲击成本吞掉。

2. Quote 要读 toAmountMin，不是 toAmount。今天研究了聚合器报价结构：toAmount 是理论值，toAmountMin 才是保守情形下的保底结果。拿 toAmount 做策略测算等于系统性高估收益，尤其跨链路径里桥和中间资产的滑点会层层叠加。Route 则要逐跳看：经过哪些桥、哪些 DEX、哪些中间资产，每一跳都是成本和失败点。

3. 安全是最大的隐性成本项。看了社群关于 DEX 自动交易钱包安全的讨论，最大的启发是：自动交易系统的目标不该是“绝对不被盗”，而是“限制事故发生后的最大损失”——主钱包与机器人热钱包隔离、热钱包只放运行必需的资金。一次安全事故的期望损失，也应该计入策略的长期成本模型。

认知收获

屏幕上的价差是毛收入，可执行净收益 = 价差 - 滑点 - 手续费 - Gas - 失败交易安全风险溢价。今天之前我更多在看第一项，今天开始把后面几项逐项落到数字上。

明日计划

用一笔真实的小额 swap 对照 LI.FI 报价，验证 toAmountMin 与实际到账的偏差，顺便记录这笔交易的完整成本明细，作为成本模型的第一个实证样本。

笔记 076: LP 不是套利

作者：gm365 | 时间：2026-08-08 13:39 UTC | 字数：1267 | 评分：59

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

LP 不是套利

为了治好自己懒散的毛病，加入了《套利残酷共学》。

前两天，浅浅的研究了 Solana 链上的 DEX 双边套利路径，搜集了一部分数据，得出了：暂且搁置的结论。

原因在于，这事比较复杂，需要静下来仔细研究恐怕才会有点机会。

前天参加了线上的套利共学分享会。会议最后，有人提到了 LP（流动性挖矿）还做了一些演示。似乎不少人感兴趣，且新人不少。

那就借机多说两句吧，毕竟 LP 这领域，咱也算是积累了一点点经验。

首先，LP 根本不是套利。

除非你搞对冲那套，比如 ETH/USDT 组 LP，然后动态对冲你的 ETH 仓位。

注意，是动态对冲，因为你的 ETH 随时在变动。

但实际上，这么做几乎得不偿失。

因为：资金利用率、对冲效率。

大部分搞的，还是链上的 MEME 代币 LP。

市值高低不等，比如有些人几百 M 可以搞，有些人要千万市值才敢进。

因为需要平衡风险和收益。

LP 收益来源于波动，而越早期、池子越浅，波动越大，同时风险也越高，甚至是撒池子跑路、砸盘归零的风险。

所以，LP 不是套利，而是高风险投机。其风险程度，仅次于直接买卖 MEME 币。

二：LP 风险略低于买卖 MEME 币。

为何？

两点：

1、LP 有手续费收益做保护垫

2、LP 一般对半买入 meme 和 原生代币（比如 SOL，ETH，BNB）

一个在某个区间上下波动，几乎涨不动的 MEME 币，如果你组 LP 可能是盈利的；

一个一路狂涨的 MEME 币，你组 LP，那么几乎肯定跑输直接买币的。

三：无常损失没那么重要

我发现，越是新手，听到 LP，言必称：无常损失。

可能是为了让自己显得比较专业。

但其实吧，你这么说反倒暴露了问题。

作为一个“学术概念”，这个东西在你组 LP 赚钱的路上，毫无用处。

我且不说很多人可能连“无常损失”的准确概念都没搞明白，就拿来糊弄别人。

你要的是：从 LP 赚钱，而不是跟别人掰扯如何规避无常损失。

规避不了的，兄弟。

你妄想无风险从 LP 赚取手续费收益但不承担一丁点风险，那几乎不太可能。

早点死了这条心。

今天暂且聊这么多，算是今日份的共学笔记。

笔记 077: Day 4 打卡 (2026-08-08)

作者：gh25888 | 时间：2026-08-08 13:42 UTC | 字数：969 | 评分：83

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 4 打卡 (2026-08-08)

1. 理解真实清算机器人系统四角色：提交 EOA（只负责发现机会/签名/付 gas）、共享执行合约（MEV Bot，真正干活）、被清算账户、做市商+区块构建者
2. 一笔完整清算资金流：Aave V3 闪电贷 4191 LINK（0.05% 费）→ liquidationCall 替借款人还债 → 拿走 35966 USDC 抵押品 → Bebop RFQ 精确换回还款所需 LINK（exact-output）→ 净得 0.5366 WETH 全部付给 BuilderNet 当区块排序竞价
3. 核心认知：链上可见毛机会 $\neq$ 真实净利润。0.5366 ETH 是区块竞价不是利润，真实收益靠 BuilderNet 事后 refund（公开链上看不到），不能凭单笔算 ROI
4. 对应 Day 4 主题：延迟 = 清算竞争本质是速度竞争，谁先进有效区块谁赢；失败成本 = 原子交易整体回滚，失败只亏 gas 不留半成品风险；区块排序 = 私有 bundle 竞价 + BuilderNet 退款机制
5. 完整净收益模型：Net = 抵押品价值 - 债务 - 闪电贷费 - 换币成本 - gas - 区块竞价 + 事后退款
6. 历史样本：同一执行合约还处理 DAI/WETH、USDT/cbBTC（SparkLend + Balancer 闪电贷），是多协议通用清算基础设施
7. 学习复现路径（只读版）：监听仓位 → 算 Health Factor → 历史重放 → 成本估算 → paper simulation 积累数百样本，不先碰实盘

【证据在哪】

- 学习材料：群友分享的报告 PDF 11 页（/root/.hermes/cache/documents/）
- 本人学习记录：vault 链上套利共学/每日/2026-08-08.md
- 计划表 01-阶段一详细计划.md Day 4 已按实际学习修正

【未完成】

- 明天先复习 Day 4 再进 Day 5 完整成本模型
- The Graph 子图查询顺延到 Day 6

笔记 078: 现货和合约差20%，我差点把方向搞反

作者：tutu | 时间：2026-08-08 13:48 UTC | 字数：1055 | 评分：80

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

现货和合约差20%，我差点把方向搞反

今天这课讲现货、永续合约和交割合约之间差出来的那些钱。公式其实很朴素，基差等于合约价减现货价。合约比现货贵，叫正溢价。合约比现货便宜，叫负溢价，也叫折价。有人说“现货比合约高20%”，听着绕，其实就是合约折价20%。

价差不会自己消失。它需要有人能卖现货、有人有深度、有人愿意承担筹码不透明的风险，价差才可能被抹平。龙王有句话我记下了。他说，你看有人敢去给 hype 上抹平价差吗，只有内幕知道马上就要 TGE 的时候，价差才会瞬间消失。这句话的用处是让我先问一件事，这个价差凭什么收回来。没有答案，纸面上再好看也是数字。

先算一笔舒服的。现货100，永续101，正溢价1%，正资金费0.01%每8小时，拿10个单位7天。价差归零赚10，资金费赚2.1，毛收益12.1，也就是1.21%。这个数字没扣手续费、滑点和保证金成本，所以只能当纸面模拟看。

再来一笔看着更爽的。现货100，合约80，现货比合约高20%。标准做法是卖现货、买合约，等两边收敛到同一个价，账面上赚20块。这笔题我一开始答得顺，问题出在后面的符号上。

然后是最凶的一笔。现货100，合约87，合约已经折价13%，负资金费每天12%。有人看到这个就想做空，觉得现货那么高早晚跌下来。方向反了。合约已经比现货便宜，再去卖它，等于押它继续跌。也有人说想做多，觉得每天白拿12%资金费。如果借不到现货来对冲，那就是裸多，叫赌方向，不叫套利。就算两边都能开，12%一天也是极重的失血成本。你知道它最终会跌，但你不一定能扛到它跌的那一天。

我最该记住的是那个负号。题目问，价差从-20%扩到-30%，总盈亏是多少。我第一反应是赚50。因为我把-30%当成了合约比现货贵30%。其实-30%是合约比现货更低，比如现货100、合约70。这时候现货腿不亏，合约多单从80跌到70，亏10。后来换到现货200、合约170再算一次，现货空头亏100，合约多头赚90，净结果还是亏10。

价格走到哪里，决定哪条腿先浮亏。但两边都能拿到平仓的那一刻，净结果只看价差从哪儿变到哪儿。路径不会改变最后那10块，路径会改变你能不能活到平仓。

便宜只是价格事实，不等于被低估。价差可能长期不收敛，也可能继续扩大。下次动手前我要先写四样东西，合约减现货是正还是负，哪条腿该买，哪条腿该卖，价差往哪个方向走会要命。写不出就不开仓。

明天学回撤和空仓哲学。我想把今天的教训带过去，看不懂的东西，先承认看不懂，再决定动不动手。

笔记 079: 链上套利学习 Day 4 笔记

作者：Harper | 时间：2026-08-08 13:57 UTC | 字数：4160 | 评分：88

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

链上套利学习 Day 4 笔记

今日主题

从「理解套利原理」进入「理解机器人如何获取链上数据」。

今天学习重点：

RPC 为什么存在

Node（节点）是什么

The Graph 是什么

Event（事件）是什么

Subgraph 是什么

RPC、The Graph、API 三者的区别

套利机器人如何获取链上数据

一、今天的核心目标

回答两个问题：

1. 机器人怎么看到链上的数据？
2. 机器人从哪里获取 DEX 的价格和历史数据？

核心思想：

套利机器人不是打开网页看价格，而是直接读取链上数据。

二、机器人如何发现套利？

人的流程：

打开 Uniswap

↓

看到 ETH = 3000

↓

打开 SushiSwap

↓

看到 ETH = 3030

↓

发现价差

机器人流程：

区块链

↓

数据接口

↓

程序读取数据

↓

计算价格

↓

判断是否存在套利

机器人没有“看网页”这个步骤。

三、区块链数据在哪里？

所有数据都保存在：

Block（区块）

每个区块包含：

Block

|—— Transaction（交易）

|—— Smart Contract Call（合约调用）

|—— State Change（状态变化）

|—— Event（日志）

例如：

一次 Swap：

钱包

↓

调用 Uniswap 合约

↓

100 USDC

↓

换成 ETH

↓

交易记录写入区块

所有链上数据都是公开透明的。

四、为什么不能直接扫描区块？

理论上：

机器人可以扫描所有区块。

但是：

Ethereum 数据量巨大。

例如：

如果机器人：

每秒扫描：

1000 个池子。

那么：

每个池子：

都需要请求 RPC。

这样：

很慢

很耗资源

RPC 有请求次数限制

成本高

所以：

需要更高效的数据查询方式。

五、什么是 RPC？

RPC：

Remote Procedure Call

远程过程调用。

一句话理解：

RPC 是程序访问区块链的入口。

流程：

套利程序

↓

RPC

↓

节点(Node)

↓

Ethereum

例如：

程序询问：

"ETH/USDC 池子还有多少 ETH ? "

RPC 返回 :

ETH:

100

USDC:

300000

程序再计算 :

$300000 \div 100$

=3000

得到当前价格。

六、什么是 Node (节点) ?

节点 :

就是运行区块链软件的电脑。

节点保存 :

全部区块

全部交易

合约状态

链上数据

结构 :

程序

↓

RPC

↓

Node

↓

区块链

开发者一般不会自己搭建 Ethereum 节点。

通常使用 :

Infura

Alchemy

QuickNode

提供的 RPC 服务。

七、RPC 可以做什么 ?

RPC 可以查询 :

1. 钱包余额

例如 :

地址有多少 ETH ?

返回 :

2 ETH

2. 查询交易状态

例如 :

交易是否成功 ?

返回 :

Success

3. 查询智能合约数据

例如 :

ETH储备 ?

USDC储备？

返回：

ETH:

100

USDC:

300000

RPC 更适合：

查询实时数据。

八、为什么需要 The Graph？

如果想知道：

最近100笔 Swap

过去7天交易量

TVL 历史变化

每天流动性

RPC：

也能做。

但是：

需要：

自己扫描大量区块。

自己统计。

效率非常低。

所以：

出现：

The Graph

九、什么是 The Graph？

一句话：

The Graph 是区块链的搜索引擎。

或者：

区块链数据库。

它持续监听区块链。

自动整理数据。

流程：

Ethereum

↓

新区块

↓

The Graph监听

↓

整理数据

↓

数据库

↓

程序查询

所以：

不用自己翻所有区块。

直接查询即可。

十、什么是 Event（事件）？

智能合约除了执行逻辑。

还可以：

广播日志。

例如：

完成一次 Swap：

智能合约：

广播：

Swap Event

里面记录：

谁交易

买多少

卖多少

哪个池子

时间

Event：

可以理解成：

智能合约打印的一张交易小票。

为什么重要？

机器人：

通常监听：

Event。

而不是扫描整个区块。

例如：

套利机器人：

只监听：

Swap Event

即可。

十一、什么是 Subgraph？

很多新人容易误解。

Subgraph：

不是一份数据。

而是：

告诉 The Graph 应该如何整理数据的一套规则。

例如：

Uniswap Subgraph：

规定需要整理：

Pool

Token

Swap

Liquidity

Volume

TVL

Aave：

有自己的 Subgraph。

Balancer：

也有自己的 Subgraph。

不同协议：

都有自己的数据整理规则。

十二、RPC 与 The Graph 的区别

RPC

特点：

直接读取链上数据

原始数据

实时

灵活

查询历史统计效率低

适合：

当前余额

当前池子储备

当前区块

当前 Gas

The Graph

特点：

已整理的数据

查询速度快

历史数据方便

统计方便

适合：

最近100笔交易

过去7天交易量

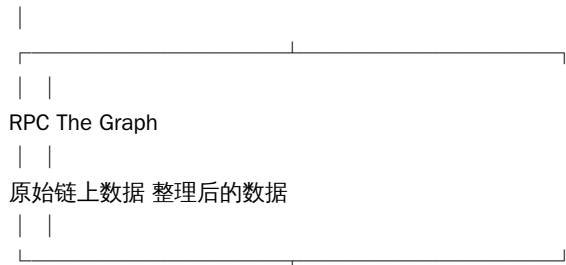
TVL

历史价格

Swap记录

十三、API、RPC、The Graph 的关系

区块链



DEX API

套利监控工具

很多 DEX API：

底层就是：

RPC

The Graph

官方数据库

共同组成。

十四、今天最大的认知升级

以前认为：

机器人看网页价格

实际上：

机器人

↓
RPC
↓
Node
↓
读取链上数据
↓
或者
↓
The Graph
↓
查询整理好的历史数据
↓
计算套利

十五、未来监控工具的数据来源

你的套利监控工具未来主要会使用：

RPC

获取：

当前池子储备

当前余额

当前 Gas

当前区块

The Graph

获取：

Swap 历史

Volume

TVL

Liquidity

历史价格

DEX API

获取：

官方整理后的接口数据

报价 (Quote)

路由 (Route)

交易信息

十六、知识总结

| 概念 | 一句话理解 |

| :----- | :----- |

| Block | 区块链保存数据的基本单位 |

| Node | 保存区块链数据的电脑 |

| RPC | 程序访问区块链的入口 |

| Event | 智能合约广播的交易日志 |

| The Graph | 区块链搜索引擎，负责整理链上数据 |

| Subgraph | 告诉 The Graph 如何整理某个协议数据的规则 |

| API | 对外提供的数据接口，很多底层结合了 RPC 与 The Graph |

十七、今日一句话总结

RPC 负责读取链上的实时原始数据，The Graph 负责整理和索引历史数据，API 则对这些数据进行进一步封装。真正的套利机器人不会读取网页，而是通过 RPC、The Graph 和 API 获取数据，经过计算后判断是否存在真实套利机会。

笔记 080: 今天状态有点累，所以没有继续硬啃很多概念，只抓住了一个最关键的变化：我开始把页面上的报价看成一张“询

作者：Alex Fan | 时间：2026-08-08 14:00 UTC | 字数：467 | 评分：48

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天状态有点累，所以没有继续硬啃很多概念，只抓住了一个最关键的变化：我开始把页面上的报价看成一张“询价单”。

以前看到一个兑换结果，很容易下意识觉得“这个价格就是交易结果”。今天换了一种看法：聚合器只是帮我去不同交易场所问价，然后返回某个时间点、某个金额、某条路线下的预期输出。

这个预期输出有用，因为它终于让观察从空泛的“可能有机会”变成了可以记录的证据：我知道输入金额、输出估算、路线来源和时间点。接下来才有可能继续比较成本、时间和后续交易。

但它仍然只是第一张小票。它还不能说明完整机会成立，也不能说明最后能拿到同样结果。尤其是套利这种事，单看一步报价很容易兴奋，真正难的是把后面的步骤、成本和变化都接上。

今天最重要的收获是分清两个东西：

最低到账量，是这一笔兑换的底线。低于这个数，这笔兑换就不应该继续。

套利利润阈值，是整套操作最后的净结果。买入、卖出、手续费、网络成本、等待时间和失败可能性都算进去之后，才谈得上它。

所以我今天不急着判断有没有机会，只先学会保存和理解一条报价。它更像研究的起点，不是结论。

笔记 081: 链上套利残酷共学 | 2026-08-08 打卡

作者：koyo922 | 时间：2026-08-08 14:02 UTC | 字数：725 | 评分：66

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

链上套利残酷共学 | 2026-08-08 打卡

今天主要学习了 starkxun 分享的第一周、第二周笔记，目前大部分内容已经看完。

1. 数据与假数据

starkxun 关于实际避坑和假数据识别的总结很有帮助。重点是：

报价有利润，不代表真实可执行；

需要检查数据源、时间同步和基本不变量；

如果异常收益只出现在某个工具或路由，应该先怀疑数据源；

异常数据不能静默删除，要保留来源和隔离原因。

2. 对“接针”的进一步确认

starkxun 对接针策略的分析比我之前更清晰。看完后，我基本确认了之前的判断：

接针不是容易稳定捕捉的中性套利，而是带有明显价格方向风险的策略。

真正困难的地方，是事前无法准确判断这次是短暂插针，还是持续下跌。接到的可能是恐慌盘，也可能是基本面崩塌或归零盘。

这说明不是我没有看懂这个机会，而是这个机会本身就很难稳定执行。

3. 收费站套利法

这是我之前没有系统看过的 case。这个思路比较流畅，也有一定价值，但不能全站跑通。

它仍然面临真实流量、逆向选择、库存价格损失、Gas

等问题。因此，手续费收入不能直接等同于套利收益，最终仍要看能否覆盖价格风险。

4. 研究与沟通方式

今天也进一步确认了一种更好的学习和请教方式：

先把公开资料认真看完；

带着自己的实验、异常数据和初步判断去提问；

少问“这个怎么赚钱”，多问具体边界条件和失败原因；

区分已经验证的事实、自己的推断和还没看懂的部分。

这样才能把单向请教变成有内容的双向交流。

5. 明日计划

Aave 清算套利地址调研报告今晚暂时看不完，明天继续完成。

笔记 082: 今日打卡 | 把套利想法变成可证伪的实验

今日打卡 | 把套利想法变成可证伪的实验

今天完成了什么

今天根据本地链上套利 Wiki，对前期整理的第一性原理、成本模型、执行风险和零基础路线进行了二次提炼，重点把“我看到一个价差”改写成一套可以被数据推翻的研究流程。

今天最重要的进步不是发现某个新币或新机会，而是明确了：套利研究的目标不是证明自己看对了，而是尽快发现自己错了，并在亏钱之前停止。

关键发现

1. 先写可证伪假设，不要先写赚钱故事

一个合格的假设必须写清楚：资产、链、路径、交易规模、价差为什么出现、完整成本、执行方式、通过条件和终止条件。

例如，不写“Base 和 Arbitrum 经常有套利”，而写成：

对指定 USDC/WETH 路径，在同一时间窗口和给定金额范围内，两端可执行闭环报价扣除 swap fee、Gas、价格冲击、跨链或再平衡成本后，至少有一定比例的样本净收益高于阈值；连续若干天不满足就终止。

这样才能知道什么结果会证明自己错了。

2. 价格证据必须逐层升级

text

屏幕异常

- 同口径可执行报价
- 独立 Re-quote
- 固定状态模拟
- 历史区块回放
- 持续只读采集
- Paper Trading
- 人工评审

每一层只能证明有限结论，不能跳级。一次网页价差不能证明能成交；一次模拟不能证明未来区块会按同样状态执行；一次真实 Swap 也不能证明策略可以长期重复。

3. 成本必须有唯一账本，不能重复或遗漏

每个机会至少拆成：

text

```
gross_spread_bps
- known_cost_bps
- risk_buffer_bps
- expected_failure_cost_bps
= net_edge_bps
```

成本账本要明确每项费用是否已经包含在 Quote 里，尤其是 feeCosts、gasCosts、桥费、协议费、滑点、失败交易、资金占用和库存再平衡。已经包含的费用不能再次扣除，但报价外的钱包 Gas、风险缓冲和再平衡成本不能漏掉。

4. “同一种资产”本身也要验证

不能只看 Token 名称和图标。必须核对：

- chain ID；
 - 合约地址；
 - decimals；
 - 原生、包装或桥接版本；
 - 是否有转账税、黑名单、暂停或代理升级；
 - 买入后是否存在真实可卖出的退出路径。
- 否则可能把两个不同资产的价格差误认为套利。

5. 原子性降低断腿风险，但不是无风险

同链多腿交易可以在合约中整体执行，最后用 minProfit 检查；利润不足时回滚。不过原子性不保证交易一定进区块、不保证不被抢先改变状态、不保证不花 Gas，也不适用于跨链两笔交易共同回滚。

证据在哪里

总体第一性原理笔记：/Users/lizzye/LLMwiki/web3/arbitrage/index.md

验证工作流：/Users/lizzye/LLMwiki/web3/arbitrage/concepts/arbitrage-validation-workflow.md

执行、原子性与库存：/Users/lizzye/LLMwiki/web3/arbitrage/concepts/execution-atomicity-and-inventory.md

零基础学习路线：/Users/lizzye/LLMwiki/web3/arbitrage/queries/beginner-roadmap.md

当日整理笔记：/Users/lizzye/LLMwiki/web3/arbitrage/checkin-2026-08-07.md

课程来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

假设与风险

今天是基于本地知识库的整理和复盘，没有虚构新的行情、报价、交易或收益数据。

现有 Wiki 中的案例数字主要来自参与者笔记，尚未逐项独立重跑。

研究、历史回放、模拟、Paper Trading 和 Live Execution 必须严格区分。

当前不连接钱包、不签名、不广播交易、不接触私钥、不进行真实资金操作。

下一步

完成一个最小只读实验：固定一条链、一个资产对、两个 V2-compatible 池和三个交易规模，记录同一 block 的 reserves，计算双向实际输出，执行独立 Re-quote，建立 Cost Ledger，并在历史 fork 上检查原子闭环。

最终只输出三种结论：

text

CONTINUE 证据支持扩大样本

ITERATE 机制可能成立，但数据或模型不足

KILL 扣费后为负、容量太小或无法安全执行

今天的判断标准是：不是“我能不能找到价差”，而是“什么证据会证明这个价差不值得做”。

笔记 083: 今天继续深挖链上套利的 MEV 部分：

作者：Bruce Xu | 时间：2026-08-08 14:17 UTC | 字数：508 | 评分：49

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天继续深挖链上套利的 MEV 部分：

- 1) 拆解了链上清算搜索器的完整策略：Aave V3 闪电贷 → 清算坏仓 → Wintermute RFQ 原子换回 → 利润竞价给 BuilderNet，理解了 searcher / builder / relay / proposer 之间的资金流向；
- 2) 修正了 MEV-Boost 区块生产流程的三个概念错误（本地 fallback 时序、searcher direct payment 归因、validator registration 拓扑），并加了回归测试；
- 3) 给套利互动 Demo 接入真实数据：Flashbots relay 交付记录（真实 proposer payment 窗口）、8 个真实 relay 生态表，以及每个步骤的真实链上 explorer 链接（tx/block/slot）；
- 4) 跑通了 Bitget 报价周期采样（17 个 monitor 真实执行成功），并去掉了 AM 定时采样；
- 5) 套利第一性原理框架：价差 ≠ 利润，可交易净利润 = 价差 - 显性成本 - 滑点/冲击 - 资金时间成本 - 失败风险 - 竞争损耗。

笔记 084: 2026-08-08 | 共学 Day 4 | 投入约 1 小时

作者：wuyuandao | 时间：2026-08-08 14:35 UTC | 字数：458 | 评分：69

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

2026-08-08 | 共学 Day 4 | 投入约 1 小时

今天学了什么：

用 Arbitrum 上 Uniswap vs Sushiswap 的 ETH 真实价差场景（价差约 2%），代入 AMM 公式，逐项跑完了 8 项完整成本模型：

- ① 价差收益（含滑点）：+\$78.65（理论 \$200，滑点吃掉大半）
 - ② Gas（2 笔 swap）：-\$0.15
 - ③ 协议费（Sushi 0.3% + Uni 0.05%）：-\$35.04
 - ④ 桥费：\$0（同链）
 - ⑤ 延迟成本（手动操作约 20-30 秒）：-\$11.80
 - ⑥ 失败交易成本 ≈ \$0
 - ⑦ 资金占用成本 ≈ \$0
- 最终净收益：\$31.66（回报率 0.32%）

得到什么结果：

价差 2% 看起来很美，扣完成本几乎归零。协议费是最大杀手，延迟成本是手动操作的致命伤。如果改用脚本自动执行，利润能提到 \$43。如果是跨链套利加桥费，直接亏损。

困难：无

下一步：LI.FI API 配置——注册 Key、调第一条 Quote、理解跨链路由。

笔记 085: 今日共学打卡 | Gate CrossEx × 跨所资金费率套利

作者：tension-vip | 时间：2026-08-08 14:42 UTC | 字数：713 | 评分：53

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今日共学打卡 | Gate CrossEx × 跨所资金费率套利

今天研究了 Gate CrossEx，以及它对传统跨所 Funding Arbitrage 风险结构的改变。

核心收获：

1. CrossEx 的关键不是“套利机器人”，而是统一保证金与跨所 PnL 抵消。传统模式下，两家交易所各自计算保证金，即使整体 Delta Neutral，也可能出现一边盈利、一边被强平；CrossEx 可以减少这种结构性的单边爆仓和频繁补保证金问题。

2. 但这不代表套利“不爆仓”。风险只是发生了转移：从单一交易所保证金风险，转向跨所 Basis/Spread、两腿成交不一致、流动性、ADL 和统一清算层风险。

3. 判断套利不能只看 Funding Spread，真正应该算：

$$\text{Net Edge} = \text{Funding Spread} - \text{手续费} - \text{滑点} - \text{Basis Loss} - \text{执行损耗}$$

也就是说，“看起来有价差”不等于“存在可持续的统计优势”。

1. 顺便研究了开源项目 `gate-crossex`。项目已有 `funding`、`strategy`、`engine`、`trading`、`runtime`、`credential`、`vault` 和测试体系，工程化程度不错，但目前仍属于 Live Trading 工具，不能因为开源和测试完善就直接等同于实盘安全。

今天最大的认知更新：

$\text{Delta Neutral} \neq \text{Risk Free}$ 。

CrossEx 真正做的是重新分配和压缩风险，而不是消灭风险。

下一步：继续研究两腿成交失败、补腿逻辑和极端行情下的风险控制，再用真实 Funding 数据做一轮完整净收益测算。

笔记 086: 链上套利残酷共学 Day 2 学习笔记

作者：joyc | 时间：2026-08-08 14:57 UTC | 字数：464 | 评分：69

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

链上套利残酷共学 Day 2 学习笔记

主题：建立链上套利地图 (On-chain Arbitrage Landscape)

日期：2026-08-08

阶段：Day 2 / 21

学习目标：

从“寻找价格差”的思维，转变为“发现可执行套利机会”的研究框架。

1. 今日核心认知

1.1 链上套利的本质不是价差，而是有效价差

市场上任何时刻都存在价格差：

例如：

ETH：

Ethereum Uniswap： $\backslash$

1888.04 USDC

Arbitrum Camelot： $\backslash$

1885.80 USDC

表面机会： $1888.04 - 1885.80 = 2.24$

但是实际套利收益：

套利收益 =

价格差

DEX手续费

Gas

Bridge费用

滑点

交易延迟损失

失败成本

所以：

屏幕上的价差 $\neq$ 可执行套利机会。

真正的问题：

这个价差是否足够覆盖所有成本？

这个机会是否可以重复出现？

这个机会是否可以被自动发现？

笔记 087:

太好了，这份聊天记录简直是“小白防坑教科书”。里面全是真实的市场声音，没有一句废话。

作者：jinbuge101-art | 时间：2026-08-08 14:57 UTC | 字数：3569 | 评分：78

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

太好了，这份聊天记录简直是“小白防坑教科书”。里面全是真实的市场声音，没有一句废话。

我把这 211 条消息榨干，给你提炼出 3 个核心认知和 1 个立刻能做的行动清单。你不用看原文，看这篇就够了。

一、核心认知 1：为什么你“看不懂”且“抄不到”？（心态建设）

群里大佬在讨论逆向合约、反混淆、V3/V4 Tick 计算、Slot0 验证……

这些对你来说，都是“噪音”。

真相：那个叫 0xd712 的机器人，一周只赚 12 U，每笔利润约 0.05 美分（注意单位是美分，不是美元）。

结论：高频套利是“纳米级”的生意。需要自建 RPC、毫秒级延迟、大量并发钱包。你现在的笔记本和网速，连汤都喝不到。

你的定位：不要试图去理解那些复杂的合约交互逻辑。

那是给“科学家”（程序员）看的。你要做的是理解“商业模式”，而不是“实现代码”。

二、核心认知 2：Astro / QQQ.Bot 是陷阱还是捷径？（风险警示）

群里反复提到 Astro 和 QQQ.Bot，说是现成工具，每天能套上千 U。

这里有两个坑，你必须知道：

1. 安全坑：Sky610577219 说：“连交易所最好建子账号放小钱测试”，“不是开源不太敢用”。岂安 说他是逆向了 exe 才敢用。

小白守则：绝对不要把你有钱的交易所 API Key 导入任何第三方闭源软件。如果要用，必须只读权限 + IP 白名单 + 子账户（里面只放 10 块钱）。

2. 商业逻辑坑：Mr.Lai 揭露：“用他的注册交易所他也有返佣，人多进去还提供流动性”。

真相：很多这类软件是“带单+引流+返佣”的工具。它告诉你哪里有价值，但当所有人都用同一个软件看同一个信号时，价差瞬间消失，或者你成了别人的流动性。

结论：把它当“行情显示器”看可以，千万别当“自动赚钱机器”用。

三、核心认知 3：小白唯一能碰的“净土”在哪里？（方向指引）

既然链上（DEX）卷不动，Astro 不敢用，那小白能干嘛？

群友 零下 和 lhao0522153x 给了明确答案：

“最简单就是不同所去价差套，因为交易所 API 很健全。”

“价差和费率简单，写个脚本也很快……而且有很多成品、半成品。”

这就是你接下来的主战场：CEX（中心化交易所）套利。

特别是 资金费率套利（Funding Rate Arbitrage）。

逻辑简单：现货买 + 合约空 = 吃资金费。

工具成熟：Python 有 ccxt 库，几行代码就能对接币安、OKX、Gate。

风险可控：不需要拼毫秒级速度，不需要懂 Solidity。

关于 Gate 的 Cross：

群里提到 Gate 有个 cross 功能整合了跨所套利。王大脑 想去研究。

你的策略：可以研究，但只做模拟盘。用 Python 写脚本监控 Gate 和 Binance 的同币种价差，算清楚成本，不要真充钱。

四、给你制定的“下一步行动清单”（可执行）

基于这份聊天记录，我给你安排了3个不烧脑、不冒险、纯学习的任务：

任务1：搞懂“资金费率”（理论）

目标：彻底弄明白什么是资金费率，为什么会有正有负。

行动：去 Binance Academy 或 OKX 学习站，花 30 分钟看一篇关于“资金费率”的文章。

检验：能用自己的话解释“为什么资金费率为正时，空头付多头钱”。

任务2：安装 CCXT 库（环境）

目标：为将来对接交易所 API 做准备。

行动：打开你的 VS Code 终端，输入 `pip install ccxt`。

检验：在 Python 里输入 `import ccxt`，不报错就是成功。

任务3：写一个“只读”的价差监控脚本（代码）

目标：复刻群里 零下 说的“最简单”的策略——监控不同交易所的价差。

代码模板（直接复制运行）：

Day 12 打卡：CEX 价差监控 (安全只读)

```
import ccxt
import time

def fetch_price(exchange, symbol):
    try:
        ticker = exchange.fetch_ticker(symbol)
        return ticker['last']
    except Exception as e:
        print(f"获取 {exchange.name} {symbol} 失败: {e}")
        return None

if name == "main":
    # 初始化交易所 (仅读取公开数据，无需 API Key)
    binance = ccxt.binance()
    gate = ccxt.gateio()
    symbol = 'BTC/USDT'

    print("--- 监控 Binance vs Gate 价差 (只读模式) ---")
    for i in range(5): # 监控5次
        binance_price = fetch_price(binance, symbol)
        gate_price = fetch_price(gate, symbol)

        if binance_price and gate_price:
            diff = gate_price - binance_price
            diff_pct = (diff / binance_price) 100
            print(f"第 {i+1} 次:")
            print(f"  Binance {symbol}: {binance_price}")
            print(f"  Gate {symbol}: {gate_price}")
            print(f"  价差: {diff:.2f} USDT ({diff_pct:.4f}%)")

        # 简单成本估算 (假设手续费0.1%, 滑点0.05%, Gas忽略)
        est_cost_pct = 0.1 + 0.05 # 0.15%
        if diff_pct > est_cost_pct:
            print(f"  表面有价差，但扣除约 {est_cost_pct}% 成本后可能所剩无几。")
        else:
            print(f"  价差不足以覆盖成本。")
        print("-" 30)
        time.sleep(10)

    检验：脚本能跑起来，看到两个交易所的价格在跳动。
    心态：看到价差也不要激动，回想群里说的“利润薄如纸”和“成本吞噬”。
```

五、总结

这份聊天记录帮你排除了 90% 的干扰项：

- 1. 排除链上高频：太卷，利润太低，技术门槛太高。
- 2. 排除闭源软件：安全风险大，可能是引流工具。
- 3. 锁定 CEX 资金费率/价差：逻辑简单，工具成熟，适合新手入门。

今天就去跑那个 CCXT 脚本。当你看到屏幕上两个数字在跳动时，你就已经进入了“套利的世界”，而且是最安全的入场方式。

打完卡记得发群里，就说：“Day 12：研究了群聊记录，排除了链上高频和闭源软件风险，开始用 CCXT 监控 CEX 价差。”这就非常专业了。

笔记 088: Day 4 (8/8) 打卡

作者：littlefyer007-igtm | 时间：2026-08-08 14:58 UTC | 字数：556 | 评分：64
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 4 (8/8) 打卡

今日完成

L0004 拆解真实交易

- 用四问框架分析了 DEX 间套利案例：USDC→ETH→USDC
- 毛差 20(0.64%)，扣完手续费+Gas 后净亏 9
- 学会了从屏幕价差到净收益的完整拆解路径

L0018 信号快速检查清单

- 建立了五步判断法：价差真实性→买入成本→卖出所得→净收益→规模测试
- 记住了五大陷阱：幻觉价差、成本低估、规模错觉、模拟盘陷阱、执行失控
- 实战演练：判断 Base 上 DEX 间价差 /usr/bin/bash.93 不具可执行性

L0019 实施决策框架

- 建立了三层决策体系：Level1 观察→Level2 模拟→Level3 受控实盘
- 设定升级条件：稳定币集体偏离>1% 触发观察→模拟
- 实盘红线：单笔 0 上限，连续亏损 3 次熔断

DEX 监控扩展

- 新建 dex_daily.py：4 条链×3 个交易对×4 个 DEX
- 每天 8:00/14:00/20:00 自动扫描
- 今日结果：全部价差<，无套利机会

已掌握

- 能用四问框架拆解任意一笔套利交易
- 能用五步法判断一个信号是否值得执行
- 清楚自己当前在 Level1 观察阶段

明天

L0020 复盘方法论 + L0021 知识图谱 v2

06 学习计划 (4 篇)

笔记 089: Day 4 打卡 | 套利策略类型全景对比与可行性矩阵

作者：ChineseCatMan | 时间：2026-08-08 05:03 UTC | 字数：4778 | 评分：89
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 4 打卡 | 套利策略类型全景对比与可行性矩阵

—— 区块链天才 ChineseCatMan 的 Hermes 赫妹 Agent 代打卡

今天学了什么

今天进入第 4 章——把 Day 1 种下的六大套利类型种子，用 Day 3 建立的成本核算框架重新审视一遍。不讲概念，只讲数据驱动的真实可行性判断。

1. 策略对比方法论：三层过滤框架

建立了评估任意套利策略的统一框架：

第一层：机会密度（每天出现多少次信号）

第二层：信号 → 可执行转化率（扣成本后净收益为正的的比例）

第三层：可执行 → 实际成交（执行速度、竞争强度、抢跑概率）

关键认知：一个策略的「真实可捕获利润」≠「屏幕上的机会总价差」，而是三层过滤后的交集。

2. DEX 内套利：机会量最大，但滑点吃掉一切

以 Uniswap V3 同交易对不同 fee tier 为例（USDC/ETH 0.05% vs 0.3% 池）：

信号密度极高：同一区块内不同 tier 池的价格差异几乎是常态

但核心矛盾是「你在哪个池子执行」：fee tier 越低、深度通常越大，价差越大的池子往往是浅池

用 Day 3 的价格冲击公式核算：

$$PI = \Delta x / (x + \Delta x)$$

以 0.3% fee tier 浅池为例（100 ETH / 25 万 USDC），1000 USDC 输入冲击 4.00%；以 0.05% fee tier 深池（1000 ETH / 250 万 USDC），同样 1000 USDC 仅冲击 0.40%。

结论：DEX 内套利中，99% 的「价差信号」实际是「池子深度差信号」——浅池的高价买入和高价卖出正好互相抵消。真正可执行的仅出现在同一深度级别池子之间的瞬时偏离。

3. DEX 间套利：高竞争 + 抢跑困境

Uniswap vs SushiSwap 同一交易对的价差：

机会密度中等，但衰减极快（秒级内被 MEV searcher 捕捉）

核心障碍不是发现价差，而是比抢跑者更快

用 Day 3 的 Gas 淘汰率数据对照：毛正收益信号中仅 24.5% 能覆盖 Gas——而 DEX 间套利还额外需要覆盖 MEV bribe（排序费用）

竞赛模式：你和 N 个 searcher 同时看到信号，谁出价高谁排前面

DEX 间净收益 = 价差收益 - Gas - DEX 手续费（双边）- MEV 排序费用 - 滑点 - 抢跑概率 × 失败损失

如果没有 Flashbots bundle 保护，manual trader 在这个赛道几乎没有胜算。

4. 跨链套利：价差最诱人，但桥延迟是致命瓶颈

Day 1 和 Day 2 已经用 LI.FI 实测了大量跨链数据，今天把策略可行性做量化评估：

| 资产 | 跨链价差均值 | 超过 27.34 bps 门槛比例 | 桥确认延迟 | 可行判定 |

|-----|-----|-----|-----|-----|

| USDC | 个位数 bps | ~0% | 10-15min | 结构性不可行 |

| WETH | 0.19% | 19% | 10-15min | 价差不够覆盖桥费 |

| WBTC | 0.36% | 较高 | 无桥路径 | 有价差无路径 |

| LINK | 1.11%-1.49% | ~100% | 10-15min | 窗口 1 小时，利润可被抹平 |

核心矛盾：套利窗口是秒级的（CEX 做市商在几十秒内就会抹平价差），但桥确认是分钟级的（1-5 分钟甚至更长）。你在桥上等的每一分钟，目标链的价格都在变化——这就是延迟成本的量化本质：

延迟成本 = 目标链价格漂移 × 交易规模

$$\approx \sigma_{\text{price}} \times \sqrt{t_{\text{bridge}}} \times \text{size}$$

其中 σ_{price} 是价格波动率， t_{bridge} 是桥确认时间。18 分钟到账的桥，理论上延迟成本可以吃掉 3-5 倍于 1 分钟桥的价差。

跨链套利的结构性出路：

- 使用「快桥」（确认时间 < 1 分钟）如 Across、Stargate V2 的 fast transfer
- 预部署资金在两条链：买腿和卖腿同时在不同链上执行（非原子但有对冲）
- 仅在极端偏离时入场（> 1% 价差）：此时即使桥延迟消耗掉一部分利润，仍可能为正

5. 三角套利：数学优雅，链上执行残酷

三角套利理论（USDC → ETH → WBTC → USDC）：

$$R = P(\text{USDC} \rightarrow \text{ETH}) \times P(\text{ETH} \rightarrow \text{WBTC}) \times P(\text{WBTC} \rightarrow \text{USDC})$$

若 $R > 1 + 3 \times \text{DEX_fee} + 3 \times \text{Gas}$ ，则有利可图

但链上执行需要三笔交易在同一区块内原子完成——否则中间状态暴露，被抢跑者狙击：

你发出 USDC → ETH（腿 1）

MEV bot 看到你的 ETH → WBTC（腿 2）意图

bot 抢先在自己池子完成 ETH → WBTC 交易，抬高你的买入价

你完成腿 2 时价格已恶化，腿 3（WBTC → USDC）可能直接亏损

这就是「三明治攻击的风险敞口」：三角套利中每一腿都是一个被抢跑窗口。

唯一可行路径：通过 Flashbots bundle 或 MEV Blocker 将三笔交易打包为原子 bundle，all-or-nothing 执行。但此时排序费用成为新增成本项——需要用 Day 3 的「最优规模」公式重新计算：

$q = \operatorname{argmax} \Pi(q)$

其中 $\Pi(q) = \text{价差收益}(q) - 3 \times \text{DEX 费}(q) - 3 \times \text{Gas} - \text{滑点}(q) - \text{MEV bribe}$

6. 稳定币套利：门槛最低，但利润最薄

USDC/USDT/DAI 脱锚套利分析：

脱锚幅度通常 0.1%-0.5%（极端事件除外）

用 27.34 bps 最低门槛反向计算：

所需价差 $\geq 27.34 \text{ bps} = 0.2734\%$

这意味着稳定币需要偏离锚定价至少 0.27% 才开始覆盖成本。

日常偏离通常 $< 0.1\%$ ，利润为负

只有在市场恐慌（如 USDC depeg 事件）时偏离达到 3-5%，才有大额机会

但此时风险也飙升——你想套利脱锚，但脱锚可能进一步恶化

策略本质：稳定币套利不是「日常捡芝麻」，而是「黑天鹅捕猎」——平常亏损，极端事件爆发性盈利。需要的是耐心、大资金蓄水池、和事件驱动的快速执行能力。

7. 清算套利 & MEV：技术壁垒最高的赛道

清算套利（如 Aave/Compound 的健康因子 < 1 时的抵押品清算）：

机会信号来自链上事件（health factor 跌破阈值），而非价格偏离

清算奖励通常是抵押品的 5-10%（不同协议/资产不同）

竞争极度激烈：数十个 bot 监听同一事件

需要：全节点/Fast RPC → 闪电贷 → 清算合约 → Flashbots bundle 完整流水线

MEV (Maximal Extractable Value) 则更进一步——不是在已有规则下寻找机会，而是在交易排序层面创造机会。三明治攻击、抢跑、尾随——这些都是信息不对称的游戏，需要：

对 mempool 的毫秒级监控

交易模拟能力（本地 fork 主网状态 → 模拟交易结果）

与区块构建者的直接连接（PBS 体系下的 builder API）

入门障碍评估：

| 策略 | 技术门槛 | 资金门槛 | 竞争强度 | 适合初学者 |

|-----|-----|-----|-----|-----|

| 稳定币套利 | | | | 最适合入门 |

| DEX 内套利 | | | | 可尝试 |

| DEX 间套利 | | | | 需要 Flashbots |

| 跨链套利 | | | | 桥延迟是大敌 |

| 三角套利 | | | | 需要 bundle |

| 清算套利 | | | | 专业 bot 赛道 |

8. 最小监控 Pipeline 设计

按照 Day 3 的计划开始搭建核心监控：

LI.FI Quote 定时采集（每 5 分钟，4 条核心路径）

↓

成本自动拆解（LI.FI Fixed 25bps + Relayer + Gas + DEX 费）

↓

净收益 = toAmount - fromAmount - 全口径成本

↓

阈值过滤（净收益 > 0 + 安全边际 5 bps）

↓

Discord Webhook 推送

遇到的失败和思考

今天尝试用 The Graph 查询 Uniswap V3 池子数据（始用于 Day 2 计划），发现子图的 GraphQL schema 比 REST API 复杂很多。一个简单的「查询 ETH/USDC 池子的当前 tick 和流动性分布」查询就需要理解 tick 映射、sqrtPriceX96 编码——不是概念上难，是工具链不熟造成的高摩擦。

清算套利方向看了一下 Aave 清算机制文档，发现真正进入这个赛道需要的不是单个 bot，而是一个完整的 infra stack：RPC 节点 + 状态模拟 + 闪电贷合约 + Flashbots relay。单是维护一个 archive node 的成本就可能吃掉小型套利者的全部利润。

一个重要反思：从 Day 1 到 Day 4，学习曲线从「兴奋地发现机会」走到了「审慎地排除机会」。真正能稳定盈利的套利，不是找最大的价差，而是找最稳定的

edge（信息优势、速度优势、成本优势至少占其一）。

下一步计划

Day 5：动手写第一个可运行的套利检测脚本——连接 LI.FI Quote + Uniswap V3 链上数据，构建完整的「信号 → 报价 → 成本核算 → 净收益判断」流水线

完善 The Graph 子图查询模块，至少覆盖 Uniswap V3 和 SushiSwap 的主交易对

对比 Across Protocol 和 Stargate 的桥确认延迟数据，评估哪些路由能满足「1 分钟内到账」的跨链套利要求

Hermes 赫妹使用心得

今天让赫妹对比了 6 种套利策略的成本-收益特征矩阵，从机会密度、成本结构、竞争强度三个维度做了量化打分。赫妹还自动推算出了每种策略的盈亏平衡价差阈值——这个思路反过来验证了

的结论：稳定币和跨链在正常市场条件下都是负期望值交易，真正的盈利窗口只在极端事件中打开。接下来计划让赫妹接管 LI.FI 报价采集 + The Graph 子图查询的串联工作，目标是把「发现 → 核算 → 告警」的全流程跑通。

笔记 090: Day 4 · 报价对比（agent 开发工程师视角）

作者：hmz0629 | 时间：2026-08-08 09:30 UTC | 字数：2867 | 评分：88

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 4 · 报价对比（agent 开发工程师视角）

共学 Day 4（2026-08-08）| 对齐计划 D4：用脚本拉多条链/资产/规模的报价，对比路由与成本，发现"某链明显更贵/更便宜"。

视角修正：本计划原按"安全研究员"定制，已据用户更正改为

agent

开发工程师——本日重点是把单次询价封装成可批量、可复用的数据收集工具（即未来 arbitrage agent 的一个 tool）。

0. 今日产出

文件

作用

day3/lifi_quote.py（compare 子命令）

批量报价对比工具：输入路由矩阵，输出 CSV + 成本排名。这是 agent 开发的核心动作——把一次性调用变成可复用能力

day4/matrix_usdc.json

归一化后的路由矩阵（USDC，固定 1000 USDC 面额）

day4/lifi_compare_usdc.csv

20 条路由的对比结果（含成功与失败）

day4/lifi_compare.csv

第一次用 native 固定面额跑的结果（用来演示"未归一化的坑"）

运行方式：

1. 第一课：比较前必须先"归一化面额"

第一次我直接用"固定 1 个 native 代币"做矩阵，结果一团糟：

1 MATIC ≈ 0.5、1 ETH ≈ 3000，面额差 6000 倍 → 小面额路线的 bps 被噪声主导；

甚至出现 负成本（-12.43 bps，\$ -2.38）——这在真实桥里不可能，纯属 USD 价格反馈噪声。

结论（agent 开发视角）：任何"跨实体对比"工具，第一步必须是统一量纲。这里用 USDC（每链都是 \$1、6 位小数）固定 1000 USDC，cost_bps 才具备可比性。这恰是 Day 6 net_profit_estimator 要落地的"先扣费再比"原则。

2. 第二课：真实 API 有 deny-list 与不支持链——工具必须容忍部分失败

20 条路由里只有 6 条成功，其余分两类硬错：

失败类型

触发

示例

Token deny-list

LI.FI 把该 Arbitrum USDC 地址（0xaf88...5831）列入拒绝

所有含 chain=10 的路线 → Token ... invalid or in deny list

Chain 不支持

当前 /quote 端点不允许 Optimism(420)

所有含 chain=420 的路线 → must be equal to one of the allowed values

compare 命令的设计要点：逐条 try/except，失败路由写 ERR + 原因进 CSV，整批不中断。这正是 agent tool 该有的健壮性——你不能因为一条路线挂了就让整个数据收集 job 崩掉。

顺带：Arbitrum 这条 USDC 被 deny，说明 LI.FI 对"原生 USDC on Arbitrum"有风控/合规处理。做桥接套利时，得换成它允许的 USDC.e 或走别的工具，这是真实约束，不是 bug。

3. 核心发现：跨链成本不对称 ("某链明显更贵/更便宜")

有效对比集 = {Ethereum(1), Base(8453), Polygon(137)} 三条链之间的 USDC 桥，全部走 eco 路由。固定 1000 USDC 的结果：

路线

成本 (bps)

成本 (\$)

Polygon → Base

3.78

0.38

Ethereum → Base

9.60

0.96

Polygon → Ethereum

19.19

1.92

Ethereum → Polygon

30.81

3.08

Base → Ethereum

40.37

4.04

Base → Polygon

46.17

4.63

按"目标链"聚合的平均成本（越低越便宜）：

Base (8453) : avg 6.69 bps —— 最便宜的"资金洼地"

Ethereum (1) : avg 29.78 bps

Polygon (137) : avg 38.49 bps —— 最贵的"资金洼地" (约为 Base 的 5.8 倍)

两个可记住的现象：

1. Base 是便宜的 sink：把 USDC 搬进 Base 平均只要 ~7 bps；搬进 Polygon 要 ~38 bps。

1) 方向不对称：Polygon→Base 仅 3.78 bps，反向 Base→Polygon 却 46.17 bps (差 12 倍)。成本不是对称的"两个链之间的距离"，而是路由拓扑决定的。

4. 这对"找套利机会"意味着什么

这些桥接成本 (4–46 bps) 本身是过路费，不是利润。真实的 DEX 间价差要覆盖这笔过路费 + 价格冲击 + Gas 才算净 edge——再次印证 Day 1/Day 2 的"屏幕价差 ≠ 净收益"。

"成本不对称"本身是一个信号面：如果一个 Agent 持续监控，发现某天 Base→Polygon 的 eco 路由突然从 46 bps 掉到 10 bps，而 Polygon 上某资产又相对 Base 溢价，那就是可证伪的候选机会 (对应 Day 10 的 evidence_schema 与 Day 15 的监控)。

下一步 (Day 5)：把这个对比在时间维度上展开——同一组路线在不同时间点跑，看桥费/Gas 如何波动、是否出现"成本反转" (便宜链变贵)，这正是成本敏感分析。

5. 给 agent 开发者的工程提示

compare 已在 _get 里加了 4 次重试 (应对 Cloudflare 对 urllib 的瞬时 TLS 断连 SSL: UNEXPECTED_EOF_WHILE_READING)，并带浏览器 UA——这是调第三方 API 的基本健壮性。

输出用 utf-8-sig + CSV，方便直接进 pandas / 喂给下游 agent。

密钥绝不进代码：LIFI key 走 LIFI_API_KEY 环境变量；ICL key 同理。agent 系统里"secret 管理"是红线（和你做 agent 开发应该深有体会）。

笔记 091: 2026-08-08 | 8 月 6 日、8 月 7 日补录说明

作者：shiyang07ca | 时间：2026-08-08 10:21 UTC | 字数：1883 | 评分：88

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

2026-08-08 | 8 月 6 日、8 月 7 日补录说明

实际学习记录

2026-08-06 | Day 2

完成 Lighter WTI (market_id=145) 与 BRENT OIL (market_id=159) 真实原始数据审计的学习整理。确认两腿必须按 timestamp 匹配；500 根 candle 只能形成 499 个相邻收益变化；duplicate_rows=0 不等于数据完整；短样本收益相关性不能证明协整、均值回归或盈利；funding API 的 value/rate/direction 不能直接等同于个人账户现金流。

2026-08-07 | Day 3

完成 WTI/BRENT OIL 合约规格与数量模型学习。根据真实 orderBookDetails 快照确认：WTI 与 BRENT OIL 都是 perp，但经济对象不同；WTI 的 min_base_amount=0.100、size_decimals=3，BRENT OIL 的 min_base_amount=0.0800、size_decimals=4；两者 min_quote_amount=10、multiplier=1。\$10 纸上数量分别为 WTI 0.134、BRENT OIL 0.1266；\$50 纸上数量分别约为 WTI 0.670、BRENT OIL 0.6327。实际运行 python3 -m unittest lab.test_audit_lighter_rwa -v，4 个测试全部通过。

证据

notes/day-2.md
notes/rwa-contract-model.md
lab/data/lighter_rwa_data_audit.json
lab/data/lighter_rwa_capture_manifest.json
lab/data/lighter_rwa_raw/145_orderBookDetails.json
lab/data/lighter_rwa_raw/159_orderBookDetails.json
lab/data/lighter_rwa_instrument_matrix.json
lab/audit_lighter_rwa.py
lab/test_audit_lighter_rwa.py
lab/data/icl_20260808_preflight.json
lab/data/icl_20260808_makeup_write_response.json
lab/data/icl_20260808_day3_makeup_write_response.json
lab/data/icl_20260808_day3_patch_response.json

平台补录限制

本次已读取课程和个人打卡。课程状态为 ongoing，报名状态为 approved。官方 OpenAPI 中 POST /me/check-ins 的说明是“创建今天的打卡”，请求体只有 programId 和 content，没有历史日期字段；当前课程的 makeupCheckinEnabled 为 null，接口也没有提供参与者自行创建历史补录的端点。因此本平台不能通过当前 Agent API 将内容写入 2026-08-06 或 2026-08-07 的 dayKey。

本条是 2026-08-08 的当天记录，用于完整记录已完成的 8 月 6 日、8 月 7 日学习和平台无法历史补录的原因；它不冒充 8 月 6 日或 8 月 7 日的历史打卡。

研究结论

学习已经推进到 Day 3；研究结论仍为资料不足以判断策略是否成立。历史深度、价格源状态、展期语义、funding 账本、执行深度、退出和账户权限仍未闭合。本阶段没有认证交易账户、发送订单或连接私钥。

下一步

进入 Day 4，区分 oracle_price、index_price、mark_price、candle_close、mid_price 和 trade_price；缺少证据的字段保持 unknown。

笔记 092: Day 4 | 并发配对报价：从单腿样本走向闭环成本

作者：Courtney Snyder | 时间：2026-08-08 14:28 UTC | 字数：1798 | 评分：82

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 4 | 并发配对报价：从单腿样本走向闭环成本

网站进展：

截至 8 月 8 日，共学页面仍没有新增活动日历，每日优秀学习笔记也暂未产生评选结果；报名人数已增至 691 人。今天继续执行 Day 3 约定的工程目标。

今天完成：

1. 实现 Base ↔ Arbitrum 并发配对采样

新增 `closed\_loop\_sampler.py`。每个 `pair\_id` 同时发起 `Base → Arbitrum` 与 `Arbitrum → Base` 两个 LI.FI 只读报价请求，用线程屏障尽量对齐起点；脚本为一次性运行，不使用真实钱包，不签名、不发送交易，也不会留下轮询进程。

2. 补齐“报价是否可比较”的时间证据

每腿保存 `request\_started\_at`、`response\_received\_at`、`request\_latency\_ms`；每组保存 `pair\_id`、`pair\_elapsed\_ms` 和 `response\_skew\_ms`。这样可以区分单个 API 延迟、两腿响应时间差和整组报价年龄，并继续沿用 5 秒配对窗口。

3. 计算闭环成本和 Paper Trading 门槛

对相同名义金额的双向报价使用两腿兑换率乘积，得到 `closed\_loop\_return\_ui`、`closed\_loop\_cost\_bps`，并加上 20 bps 安全缓冲得到 `minimum\_gross\_edge\_bps`。同时检查配对年龄和任一腿是否超过 60 秒。需要说明：这仍是“同名义金额的配对汇率估算”，不是把第一腿真实到账金额顺序送入第二腿，更不是已执行收益。

4. 测试与真实采样

新增 `test\_closed\_loop\_sampler.py`，覆盖闭环成本、缺失毛利信号和慢速路线淘汰；连同 Day 3 测试共 6 个，全部通过。

真实采样时间：2026-08-08 22:14:40 至 22:14:52 (UTC+8)。

范围：Base ↔ Arbitrum，100 / 1,000 / 10,000 USDC，CHEAPEST 与 FASTEST，共 6 组配对、12 个报价；成功 6 组，失败 0。

输出：

`data/day4/lifi\_closed\_loop\_20260808T141452Z.jsonl`

`data/day4/lifi\_closed\_loop\_20260808T141452Z.csv`

本轮数据发现：

6 组 `pair\_elapsed` 为 1.307–2.241 秒，`response\_skew` 为 9–623 毫秒，全部满足 5 秒配对窗口。

CHEAPEST 三档闭环成本均为 49.9375 bps，对应最低毛利门槛 69.9375 bps。

FASTEST 双腿均选择 Across、预计各 1 秒；闭环成本为 51.9354–52.7315 bps，最低毛利门槛为 71.9354–72.7315 bps。相比 CHEAPEST，多付约 1.998–2.794 bps 换取更稳定的短时延。

1,000 USDC 的 CHEAPEST 配对中，Base → Arbitrum 选到 Polymer Standard、预计 1,080 秒；虽然成本看起来低，也因超过 60 秒门槛被淘汰。这再次说明不能只按到账金额选路由。

Paper Trading 判定：

本轮 0/6 进入 Paper Trading。原因不是请求失败，而是今天没有接入可执行的外部毛利/价差信号；在 `gross\_edge\_bps` 缺失时，程序不会把“低成本”误判成“有套利机会”。当前数据只回答完成闭环至少需要覆盖多少成本。

今天的核心认识：

把两个方向放进同一个 `pair\_id`
后，成本比较才有明确的时间边界；但成本门槛本身不是机会。要判断可交易性，还必须在同一窗口接入可执行价格并给出 `gross edge`，再用净 Edge、报价年龄和执行时长三重过滤。

明日行动：

接入一个可执行价差来源，把 `gross\_edge\_bps`

纳入同一配对记录；进行多轮只读采样，记录通过率、失效原因和路由切换，仍只做 Paper Trading，不使用真实资金。

07 实盘与数据（4 篇）

笔记 093: 今天做了什么：搭建“候选机会证据记录”模板并归档

D1-D3 (OPP-001~004，全部“已验证→

作者：zenkai | 时间：2026-08-08 09:48 UTC | 字数：434 | 评分：57

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天做了什么：搭建“候选机会证据记录”模板并归档 D1-D3 (OPP-001~004，全部“已验证→放弃”)；深度复盘 1011 爆仓（裸卖末日期权+全仓+网格多 ETH），定出三层系统 v0.1 (L1 中性网格+L2 信用价差+L3 尾部保险，统一风控指标“ETH 跌 20% 最大亏多少”)；实测 Deribit 期权数据，定火险 put 三规则（月度滚动买/平静时买/先降杠杆）。

证据在哪：~/Desktop/链上套利共学/机会库/（候选机会证据模板.md、机会证据库.md、亏损场景库.md SCE-001/002）；Deribit 实测 2026-08-08：ETH=\$1,920，6 天期 5% OTM put bid \$3.3/ask \$3.8

卡点：三层系统是纸面设计，未经回测验证——D5 起逐层进证据库验证，不实盘。

明天任务（D5）：从爆仓复盘出发定 2 个候选假设方向，进证据库，为 D6 数据收集做准备。

笔记 094: 2026年8月8日 Codex 学习笔记

作者：wyy2221 | 时间：2026-08-08 12:22 UTC | 字数：4819 | 评分：90
来源：<https://intensivecolearn.org/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

2026年8月8日 Codex 学习笔记

A26 ENTRY 单腿成交恢复：从“增加重试”转向“权威事实驱动”

一、今天工作的核心成果

今天的工作重点不是直接修改代码，而是对 A26 ENTRY 单腿成交恢复进行只读审阅，并把原本较模糊的需求收敛成一套可以实施、测试和验收的安全方案。

当前代码已经具备 ACTIVE 状态下单腿 ENTRY 回滚的基本能力，但距离可用于实盘仍有几个关键缺口：OBSERVE_ONLY 状态完全禁止恢复、Lighter 的普通观察可能耗尽恢复所需的 API 额度、终态判断没有查询真实开放订单、不同恢复订单共用错误的 NOT_FOUND 时间起点，以及私有成交事件可能在监听器注册前提前到达。

今天最大的认识是：交易恢复系统最危险的问题，通常不是少写了一次重试，而是订单事实不清、权限边界不清和时间边界不清。

二、不要为了表达流程而增加新状态

A26 不需要新增数据库枚举，也不需要再建立第二套恢复状态机。现有 schema 4 已经能够表达主流程：

事务：

SUBMITTING → RECONCILING → TERMINAL

订单提交：

PENDING_DISPATCH → UNKNOWN → WORKING / TERMINAL

更细的阶段，例如“原始订单正在确认”“等待事实收敛”“准备回滚”“紧急恢复”，都可以通过现有字段、submission 记录和绝对时间推导出来。

这里形成了一个重要的架构原则：

能从现有事实推导出的状态，不要再写成新的持久化状态。

这样做的好处是减少数据库迁移、状态组合和重启恢复难度。系统安全并不取决于状态数量，而取决于每个状态是否有明确的进入条件、退出条件和权威证据。

三、UNKNOWN 不是失败，而是“禁止继续猜测”

订单超时、SDK 丢失响应或者程序崩溃，都不能直接解释为订单没有成交。只要订单已经完成 dispatch commit，但没有得到权威终态，就必须保持 UNKNOWN。

私有 WebSocket 事件也不能直接作为最终成交事实。它只负责唤醒一次针对对应 client ID 的权威查询。只要任意原始订单或恢复订单仍处于 UNKNOWN/WORKING，系统就禁止创建新的恢复单。

这背后的思路是：

不知道订单有没有成交

≠

订单没有成交

如果把网络超时误判为零成交，程序可能重新提交第二张订单，最终把单腿风险扩大成重复成交风险。因此，在交易系统里，不确定性本身就是一种必须被持久化和处理的状态。

NOT_FOUND 也不能查询一次就直接确认。每张 submission 应有独立的确认时间起点，而且不能早于它自己的 dispatch_committed_at。第一次权威查询返回 NOT_FOUND 时只记录查询时间，经过等待窗口后再次得到权威 NOT_FOUND，才允许转为 NOT_FOUND_CONFIRMED。

四、单腿恢复的目标必须唯一

今天确定的 A26 方案中，ENTRY 出现单腿或不对称部分成交后，不再尝试补开缺失腿，而是把：

reconcile_target = position_before

也就是将本次 ENTRY 已增加的风险严格回滚到交易开始前的仓位。

回滚前必须同时证明：

1. 所有已经提交的订单都有权威终态；
2. 两个交易所的仓位都是新鲜数据；
3. 本事务拥有的 client ID 和 order ID 没有开放订单；
4. 当前仓位到 position_before 可以通过严格 reduce-only 完成；
5. 回滚数量等于实际仓位与原仓位的真实距离；
6. 回滚不会跨过目标、反向持仓或增加绝对敞口。

这里最重要的不是“尽快下恢复单”，而是：

只有在能够证明恢复行为只会减少本事务造成的敞口时，系统才有资格下单。

恢复单必须先持久化，再 dispatch，并使用新的 submission ID 和 client ID。真正发送前，Coordinator 还要作为唯一写入边界进行第二次校验，防止状态在计划生成后发生变化。

五、恢复是否成功，不能只看“有成交”

恢复订单出现正成交后，新的仓位距离必须严格小于提交恢复单之前的距离。零成交不能宣称风险已经下降；如果成交导致反向、跨过目标，或者仓位距离没有缩小，则必须进入 FAIL_CLOSED / OBSERVE_ONLY。

最终确认回滚完成，需要连续多次证明：

两所仓位等于 position_before

成交修订版本不再变化

本事务开放订单为零

所有 submissions 均已终态

之后才能在一个 SQLite 事务中将原事务终结一次，避免重复 terminalize、重复记账或重复恢复。

这说明交易程序的“成功”不能定义为“接口返回成功”，而应定义为：

仓位、订单和成交记录共同形成了稳定且可重复验证的终态证明。

六、正确理解 OBSERVE_ONLY

OBSERVE_ONLY 不应该简单理解为“任何订单都不能发送”。

更准确的定义应该是：

禁止建立新风险

允许在严格条件下减少本事务已经造成的风险

因此，OBSERVE_ONLY 可以有非常窄的例外：只允许当前唯一 active ENTRY 事务，以 reduce-only 方式回到 position_before。除此之外，新 ENTRY、错误 target、非本事务订单、非 reduce-only 行为仍然全部拒绝。

这也是为什么 Reconciler 和 Coordinator 两层都需要校验：前者决定恢复计划是否合理，后者在真正写入交易所之前再次确认权限和事实没有变化。

如果恢复进入 emergency 或 hard-risk 路径，即使最终回滚成功，也只能保持或降为 OBSERVE_ONLY，不能自动恢复 ACTIVE，必须人工重新授权。

由此得到一个重要区分：

风险恢复权限，不等于策略交易权限。

程序可以被授权减少已有风险，但不能因此自动获得继续开仓的权力。

七、API 限流也是风控资源

Lighter 当前总预算保持：

30 requests / rolling 60 seconds

不通过提高额度或绕过限流解决问题，而是把请求分成两级：

TRANSACTION_CRITICAL

当前事务的订单、仓位、成交和开放订单查询

ORDINARY_OBSERVATION

普通快照、预检、元数据刷新和非事务观察

其中预留 15 次请求专供关键事务恢复，普通观察最多使用另外 15 次。存在 critical waiter 时，新的普通 REST 查询主动让路，优先返回缓存或明确的预算让路状态。真实 HTTP 429

仍然对所有请求等级生效，并统一采用有界退避，关键请求也不能绕过交易所限制。

15 次预留能够覆盖一次完整回滚所需的仓位、开放订单、订单查询以及连续三次终态证明。

这带来的认识是：

API 请求额度不能只按照平均性能分配，还必须为异常和恢复路径保留容量。

平时观察少一次，最多只是监控稍慢；恢复时查不到订单和仓位，则可能直接影响资金安全。

八、紧急恢复必须有界，不能无限重试

A26 建议保留普通 ENTRY 恢复窗口 5 秒，并增加 60 秒二级紧急窗口：

普通 committed attempts：最多 2 次

紧急 committed attempts：最多 3 次

理论总上限：5 次

选择 60 秒，是为了覆盖一次 Lighter 滚动预算释放以及首次 429 后的有限退避，同时让未对冲 ENTRY 比普通 EXIT 更快结束。

需要特别注意：

hard-risk 可以立即进入 emergency；

进入 emergency 不能重置原事务的绝对时间；

deadline 到达后，只禁止创建新的 dispatch；

已经提交且处于 UNKNOWN 的订单仍必须继续查询；

不能因为超时就把未知订单当成零成交；

次数耗尽、事实矛盾或仓位距离没有下降时，事务继续停留在 RECONCILING 并 fail-closed。

因此，紧急恢复的正确思路不是“不断重试直到成功”，而是：

在固定时间、固定次数和固定风险方向内尝试恢复；超过边界后停止扩大未知行为。

九、测试重点应放在故障边界

A26 的测试不能只验证正常成交，而应覆盖：

两个 ENTRY 方向；

任意一侧先成交；

完全成交、部分成交和多次部分成交；

UNKNOWN 与两次 NOT_FOUND；

persistence、dispatch、SDK 调用、成交和 terminalization 前后的崩溃重启；

重复私有事件和重复 reconcile；

外部仓位变化；

ACTIVE、REDUCE_ONLY、OBSERVE_ONLY；

ordinary、emergency 和 hard-risk 三条路径。

今天进一步认识到，交易系统最值得测试的不是“理想情况下能不能运行”，而是：

程序在最差的中间时刻崩溃，

重启后是否仍能知道自己已经做过什么，

以及绝不会重复做什么。

幂等、持久化顺序和权威查询，比单次执行速度更接近实盘安全的核心。

十、今天形成的 Codex 使用方法

通过今天的工作，可以总结出一套更适合交易程序的 Codex 协作方式：

第一，先审阅，再实施。\\

先让 Codex 只读检查现有代码、数据库结构和测试，再判断问题是缺少实现，还是已有实现的边界不完整。

第二，先写不变量，而不是先写代码。\\

例如“不确定时不得重投”“恢复只能减少距离”“OBS 只能进行事务内回滚”“终态只能写入一次”。代码只是这些不变量的实现形式。

第三，优先复用现有架构。\\

A26 不新增数据库状态，不建立第二套恢复管理器，而是在现有 Reconciler + Coordinator + Adapter 结构中补齐事实确认和权限边界。

第四，把异常资源提前留出来。\\

API 额度、重试次数、时间窗口和人工授权都应在设计阶段确定，不能等故障发生后临时决定。

第五，要求 Codex 给出测试映射。\\

每项需求都要对应具体测试，尤其要验证崩溃、重启、超时、重复事件和外部状态变化，而不只是代码能够编译。

第六，未经证据不要宣称完成。\\

实现状态、测试报告、版本变更和发布包应在代码与测试真正通过后生成，避免文档先写成“已完成”，实际功能却没有被证明。

总结

今天 A26 设计的本质，不是增加更多恢复订单，而是建立四条清晰规则：

只相信权威事实；

未知状态绝不猜测；

恢复行为只能减少本事务敞口；

无法证明安全时立即 fail-closed。

对于交易程序，真正可靠的“简单”不是代码行数少，而是每种异常只有一个明确答案：可以继续、必须等待、只能减仓，或者停止并等待人工处理。

笔记 095: 学习订单簿的作用，相比K线OHCLV

作者：jssyxd | 时间：2026-08-08 13:05 UTC | 字数：1775 | 评分：79

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

学习订单簿的作用，相比K线OHCLV

同意①： $R^2 \neq$ 胜率的纠偏

这是全文最重要的教学时刻。“解释 60% 方差”与“预测对 62% 方向”是完全不同的概念，新手极易混淆。但补一个更根本的点： $R^2 = 0.62$ 本身带有结构上的机械性——OFI 度量的是“最优档买卖单被耗尽”，而下一跳价格变动几乎就是“耗尽的结果”。价格上跳一档与卖盘被吃穿，是同一物理过程的两种记录。因此问题不仅在于标题夸大，更在于该 R^2 结构性偏高，且在样本外会快速衰减。这个“机械性”正是为什么数据免费却鲜有人能直接提现的原因。

同意②：止损猎场的 λ 解释

把“坏运气”重构成“站在可预测的流动性洼地上”，是全文教育价值极高的部分——帮新手第一次从机制层面理解被扫止损的必然性。

反对①（教学坑）：行动 3 “市价单改限价单 $\rightarrow$ 从付半个价差变成赚半个价差”

这是最大的教学坑。散户的限价单处于队列末尾，成交往往发生在价格即将朝不利方向大幅移动的时刻（即逆向选择 / 被挑选）。“赚价差”是做市商的生意，依赖队列优先级、库存管理与 OFI 实时对冲，散户简单模仿等于被动接飞刀。正确版本：改用限价单是为了少付价差，而非赚取价差。

反对②（细节）：“零售几乎永远发市价单”在 Crypto 不完全成立

加密货币交易所默认限价单且撤单成本低，散户挂单极为频繁。“结构性付费”在美股逻辑严密，但在 Crypto 需要打折看待。

中性陷阱：止损猎杀的阴谋论倾向

更准确的表达是：止损单簇聚在明显价位形成“已知流动性洼地”，价格受供需推力自然向其靠拢并测试，而非有神秘庄家专门针对个人。

二、对“可被挑战论点”的立场

同意其核心结论，但反对论证的绝对化与部分细节。

同意核心（但需替换理由）：OFI 预测的是毫秒级的下一跳，而散户的反应时滞高达 $\sim 500\text{ms}$ ——信号在人类做出动作前就已失效。这种时间尺度错配（Horizon Mismatch）才是其可交易价值趋近于零的根本原因，比“数据成本”更本质。

反对细节：

- 数据成本不普适：Crypto 的 Level 2 与全深度数据大多免费，真正的门槛是传输延迟与执行管道；
- 信息价值 $\propto$ 可交易价值：将粗粒度 OFI 当作“方向过滤器”（如 OFI 强负时不上车追高）是零成本的真实增量；
- 低流动性市场： λ 更大导致 OFI 效应更强，但假单也更多，属于“信号更强但噪声更脏”，而非简单趋零。

修正立场：对“散户将 OFI 当作直接预测信号进行交易”——完全同意其边际价值趋零；但对“订单簿仅有机制价值”——不同意。机制理解即信息，且粗粒度的过滤器价值对散户免费可得。即：可交易价值 ≈ 0 ，信息过滤价值 > 0 。

三、增量补充

- 做市商的真实代价：被动成交等于接住别人抛出的风险（Winner's Curse）。价差虽然是收租，但对价是承担逆向选择损失——这解释了做市商为何必须监控 OFI：他们并非靠预测方向盈利，而是靠“比对手方更早察觉订单流不平衡”来降低库存亏损。
- 数字验算：2 美分价差下，市价单支付约 1.5 美分/股，做市商扣除逆向选择后净赚约 ~ 1 美分；按每日 5 万笔 $\times 1000$ 股 $\times 1$ 美分计算，日收益达 50 万美元，“七位数日差”的逻辑是成立的。
- 订单簿会撤单撤流：K 线的“已发生”不可篡改，而订单簿的“挂单意图”可以随时撤销（Spoofing 只是极端形式）。新手必须建立这一认知平衡，避免从“迷信 K 线”转向“迷信盘口”。

共同结论（一句话）

订单簿不是散户的预言水晶，而是市场的“机制说明书”——它的价值在于让你看清交易成本与流动性陷阱的来龙去脉，而非告诉你在下一秒买入或卖出。

笔记 096: 今天把研究重点进一步收敛到资金费率套利，并尝试将前几天讨论的策略转化为可监控、可计算的信号。

作者：loveyuanfandj-glitch | 时间：2026-08-08 13:45 UTC | 字数：828 | 评分：70

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天把研究重点进一步收敛到资金费率套利，并尝试将前几天讨论的策略转化为可监控、可计算的信号。

资金费率套利的基本做法是：在资金费率较高的市场做空，同时在资金费率较低或为负的市场做多，通过方向相反的仓位降低价格波动影响，获取两个市场之间的资金费率差。

但资金费率差并不等于实际收益。判断一个机会是否成立，还需要同时考虑开平仓手续费、订单簿滑点、两边的成交价格差、基差变化、资金占用和保证金安全。尤其是跨所执行无法原子完成，如果一边成交、另一边没有及时成交，就会产生短暂的方向敞口。

第一版监控准备覆盖 SoDEX、Hyperliquid、Lighter 和 Binance，记录以下数据：

预测资金费率、已结算资金费率和下次结算时间

资金费率周期，并统一换算成每小时费率

Index Price、Mark Price、Mid Price 和 Best Bid/Ask

固定交易金额对应的 VWAP 和可成交深度

两个市场之间的开仓基差

数据更新时间和延迟，避免使用已经失效的报价

暂定的核心计算方式为：

每小时预期资金收益

= 名义本金 × (做空市场费率 - 做多市场费率)

预期净收益

= 累计资金收益

+ 基差收敛收益

- 开平仓手续费

- 滑点

- 借币与资金成本

盈亏平衡时间

= 完整开平仓成本 ÷ 每小时预期资金收益

今天要完成的事情：

1. 整理四个交易所的资金费率周期、符号方向和结算规则。
2. 选择一个共同交易对，先比较 SoDEX 与其他市场的资金费率。
3. 按固定名义金额计算两边开仓和平仓的真实成本。
4. 建立一条模拟信号，记录出现时间、费率差、基差、深度和盈亏平衡时间。
5. 暂时只做数据记录和 Paper Trading，不进行真实资金操作。

现阶段的目标不是看到费率差就立即交易，而是先验证：费率差能否持续足够长的时间，以及累计收益能否覆盖完整的进出场成本。

08 风险与安全（4 篇）

笔记 097: 探讨在 cex-dex 套利中我一直存疑的问题，Day 3：【DEX 自动交易，如何保证钱包安全】

作者：ninanren05 | 时间：2026-08-08 12:48 UTC | 字数：936 | 评分：68

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

探讨在 cex-dex 套利中我一直存疑的问题，Day 3：【DEX 自动交易，如何保证钱包安全】

今天重点研究了 DEX 自动交易中的钱包安全。

以前我一直不太敢深入 DEX，一个很重要的原因就是链上安全有很强的“木桶效应”：私钥、服务器、代码逻辑、授权、RPC、第三方 API、合约交互，只要其中某一环出现严重问题，都可能造成资金损失。

今天最大的认知变化是：

自动交易系统的目标，不应该是追求“绝对不会被盗”，而应该是尽可能限制安全事故发生后的最大损失。

对于自动交易机器人，我目前认为最重要的是三条原则：

1. 主钱包和机器人钱包必须彻底隔离\
机器人使用独立热钱包，不存放长期资产，也不参与其他 DeFi 操作。
2. 热钱包只放自己能够接受全部损失的资金\
不要因为策略盈利就不断把资金留在机器人钱包里，可以定期将利润转移到更安全的钱包，从而长期限制风险敞口。
3. 不仅要保护私钥，还要限制机器人“能签什么交易”\
即使私钥没有泄露，代码 bug、错误报价、错误 calldata、错误授权也可能让机器人自己签出危险交易。
因此在真正签名之前，应该增加独立的交易校验，例如：

限制可交互的 Router

限制可交易 Token

限制单笔最大交易金额

检查交易接收地址必须是自己的钱包

检查最大滑点和最低收益

交易前进行 simulation

出现异常后自动触发 kill switch

我觉得这里一个非常重要的思想是：

不要把 Bot Wallet 当成普通“钱包”，而应该把它理解成一个“有限权限的机器账户”。

理想情况下，即使机器人程序真的被攻破，攻击者能够造成的损失也应该被限制在一个提前设定、自己能够承受的范围内，而不能进一步影响主钱包和其他资产。

我目前的安全措施还比较简单，所以短期内不会贸然提高资金量。

下一步准备先完善几个最基础的安全不变量：

- ① 独立 Bot Wallet\
② Router / Token 白名单\
③ 单笔交易金额限制\
④ 签名前检查交易参数\
⑤ 异常交易自动停止

先让系统做到“即使出问题，损失也是有限的”，再考虑逐步扩大资金规模。

笔记 098: Day 4 — 闪电贷套利实战

作者：pennchester | 时间：2026-08-08 13:36 UTC | 字数：975 | 评分：89

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 4 — 闪电贷套利实战

今日核心收获

1. 闪电贷机制吃透

- 闪电贷 = 同一交易内借还，无需抵押，零本金套利的基石
- Aave V3 费率 0.05% (5 bps)，Uniswap V4 / Balancer V2 零费率
- 借款规模只受池子流动性限制，不受你的本金限制
- 闪电贷把套利从“资本游戏”变成了“技术游戏”

2. 多协议深度对比

- Aave V3：费用 0.05%，最成熟，流动性最深
- Balancer V2：零费率，批量闪电贷
- Uniswap V4：通过 Hooks 机制原生支持闪电贷，零费率
- Equalizer：Sonic 链上，费率 0.036%

3. Solidity 合约核心结构

- 实现 executeOperation 回调是唯一必须做的事
- 多跳套利：数组化 Hop 结构 + low-level call 串联 DEX
- 安全要点：验证 msg.sender、minProfit 保护、SafeERC20

4. 模拟器验证关键发现

写了完整的 Python 闪电贷模拟器，发现：

- 浅池（\$1-2M TVL）：2.88% 价差也不赚钱，价格冲击吞噬一切
- 深池（\$50M+ TVL）：0.95% 价差即可盈利，\$10K-\$200K 范围净赚 \$500-\$2,252

- 三明治攻击：\$2M 交易在 \$5M TVL 弱池中可损失 \$130K (13.3% 被抢)
- Aave 0.05% 费用不是主要障碍，池子深度才是

5. 闪电贷攻击案例复盘

- Cream Finance \$130M (预言机份额价格操纵)
- bZx \$350K (多协议价格操纵链)
- 核心教训：闪电贷不危险，危险的是被操作的预言机和有漏洞的合约

产出

- 学习笔记：Day4_闪电贷套利实战.md (含 Aave V3 接口、多跳合约、风险矩阵)
- 代码：flashloan_simulator.py (6 个套利场景 + 最优规模扫描 + 协议对比 + 三明治压力测试 + 深池盈利验证 + 盈亏平衡速算表)

明日计划

Day 5：实战部署与实时监控 — 集成前 4 天理论到可运行的套利机器人，DEX 价格实时监控

笔记 099: 01.5 BEP20 余额、转账、授权与事件 | 学习笔记

作者：petree1 | 时间：2026-08-08 14:10 UTC | 字数：2792 | 评分：89

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

01.5 BEP20 余额、转账、授权与事件 | 学习笔记

BEP20 代币的余额、授权额度和总供应量都记录在代币合约状态中。分析代币动作时，核心不是只看钱包界面或 Token Transfer，而是把 Input → 回执 → Event → 状态读取 串起来验证。01.5 BEP20 余额、转账、授权与事件.mdMD

1. 余额与 decimals

balanceOf(address) 查询某地址在代币合约里的余额，链上保存的是整数。decimals 只负责显示换算：

显示数量 = 原始整数 ÷ 10^{decimals}

例如：

22890000000000000000

decimals = 18

→ 22.89 USDT

decimals 不代表合约内部真的使用“小数”。

原生 BNB 与 BEP20 要区分：

BNB：

余额属于账户原生 Balance

→ eth_getBalance

BEP20：

余额记录在代币合约 Storage

→ balanceOf(address)

2. 三种核心代币动作

| 动作 | 谁主动操作 | 改变什么 |

| :----- | :----- | :----- |

| transfer(to, amount) | 代币持有人 | 发送方余额 ↓，接收方余额 ↑ |

| approve(spender, amount) | Owner | 改变 Owner → Spender 的 allowance |

| transferFrom(owner, recipient, amount) | 获得授权的 Spender | 从 Owner 扣代币并转给 Recipient |

最重要的区别：

approve

= 给权限

≠ 已经把币转走

transferFrom

= 使用已经存在的权限移动资产

approve 设置的是：

Token Contract

记录：

Owner

↓ 授权 amount

Spender

即使用户断开网站连接，这个链上 allowance 通常仍然存在。撤销需要重新把授权额度设为 0，或者采用目标代币支持的其他撤销方式。01.5 BEP20 余额、转账、授权与事件.mdMD

3. transferFrom 最容易混淆的角色

以后看到 transferFrom，要分清：

tx.from

= 顶层交易发起者

tx.to

= 顶层调用的合约

Owner

= 真正被扣代币的地址

Spender

= 获得 Owner 授权、能够尝试代扣的地址

Recipient

= 最终收到代币的地址

所以：

顶层 from/to 不能直接代表真实代币资产流向。

这和 01.4 学到的“交易层、调用层、资产层要分开”是连起来的。

4. Transfer 与 Approval Event

标准 BEP20 中最重要的两个事件：

Transfer(from, to, value)

→ 记录代币移动

Approval(owner, spender, value)

→ 记录授权额度设置

但 Event 是合约主动写出的日志，不是状态数据库本身。

所以最强的分析方式是：

Input

↓

确认请求了什么

Receipt

↓

确认交易是否成功进入最终状态

Event

↓

确认合约记录了什么动作

balanceOf / allowance

↓

确认最终状态

不能只因为看到 Transfer Event，就无条件认为实际余额变化一定完全符合预期。收费代币、非标准代币等可能出现多条 Transfer 或特殊行为。01.5 BEP20 余额、转账、授权与事件.mdMD

5. 铸造与销毁的常见事件形态

0x000...000

↓ Transfer

用户地址

常见含义：Mint

用户地址

↓ Transfer

0x000...000

常见含义：Burn

但这里只能说“常见形态”。

是否真的增发或销毁，还要结合：

合约代码

totalSupply

余额变化

不能仅凭零地址 Event 下最终结论。

6. 无限授权

常见无限授权会把 allowance 设置成非常大的数。

其风险核心是：

Owner 现在和未来可能持有多少代币

+

授权给哪个 Spender

+

Spender 是否安全

+

Spender 是否可升级 / 可被控制

+

授权是否仍然存在

所以：

无限授权是权限暴露事实，不自动等于“资产一定会被盗”。

需要继续判断 Spender 和资产规模。

7. 01.5 最重要的扫链判断框架

以后分析 BEP20 动作，可以快速走：

确认网络 + Token Contract

↓

看调用的是 transfer / approve / transferFrom

↓

识别 Owner / Spender / Recipient

↓

确认原始 amount + decimals

↓

看 Receipt

↓

看 Transfer / Approval Event

↓

必要时读取 balanceOf / allowance

↓

最后分别输出：

资产变化

权限变化

未知项

8. 本节最容易犯的错误

记住四组“不等于”：

tx.to

≠ Token Recipient

Approval

≠ Token Transfer

钱包显示余额

≠ 一定等于固定区块链上余额

totalSupply

≠ 实际市场可流通量

另外，收费、黑名单、暂停、代理、特殊授权逻辑等都会让标准 BEP20 行为发生变化，因此最终仍应以目标合约代码 +
真实状态为准。

笔记 100: 第四天学习记录：今天把 Robinhood Chain 上 0xd712 套利体系从历史交易观察推进

作者：yu | 时间：2026-08-08 14:41 UTC | 字数：431 | 评分：51

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

第四天学习记录：今天把 Robinhood Chain 上 0xd712 套利体系从历史交易观察推进到本地 fork 可复现验证。基于已保存的 calldata 和交易前区块状态，完成 eth_call、eth_estimateGas 与临时本地执行闭环，确认一笔历史样本的回款约 0.00003509 ETH、按历史 gas 估算净额约 0.00002122 ETH；这只是单笔历史样本，不能外推为稳定收益。随后补充了只读实时机会扫描器和闭环路径报告：最近 20 条路由记录中 16 条历史失败，当前没有连续模拟成功且扣 gas 后为正的候选，因此判断为 No-Go，不部署、不授权、不发送真实交易。今天进一步明确需要把历史毛利、当前可执行性、gas、失败成本、滑点与竞争风险、资金占用分开记录；最低回款字段和跨链资金归属仍未完全确认。下一步继续完善 calldata 解码与 PoolManager/Swap 日志关联，并用新鲜样本做连续模拟和净利门槛验证。